Contents

Preface **4**
How this book is structured 4
Two ways to read this book 4
The notebooks 5
What we don't promise 5
Who this book is for 5

Unit 1: Logistics **6**
The Hook 6
The Bottleneck 7
The Quantum Angle 8
Worked Example 13
Reality Check 15
Chef's Notes 16

Deep-Dive 1: Building QAOA from Gates **18**
In This Chapter 18

Unit 2: Cryptography **26**
The Hook 26
The Bottleneck 27
The Quantum Angle 28
Worked Example 31
Reality Check 33
Chef's Notes 34

Deep-Dive 2: Inside Shor's Algorithm **36**
In This Chapter 36
The problem we need to solve 36
From CNOT to function evaluation 36
Phase kickback: the trick behind everything 38
Oracles: packaging functions as gates 39
The Quantum Fourier Transform 41
Assembling Shor's algorithm 44
The modular exponentiation circuit 46
What you should take away 47

Unit 3: Drug Discovery **48**
The Hook 48
The Bottleneck 48
The Quantum Angle 50
Worked Example 52
Reality Check 53
Chef's Notes 53

Deep-Dive 3: The VQE Pipeline **55**
In This Chapter 55
Why electrons aren't qubits 55
Second quantisation: the language of electrons 56
Fermion-to-qubit encodings 57

Measuring the energy . . . 58
The ansatz: parameterised chemistry . . . 59
The classical optimiser . . . 60
Putting it all together . . . 61
What you should take away . . . 62

Unit 4: Machine Learning . . . 64
The Hook . . . 64
The Bottleneck . . . 64
The Quantum Angle . . . 65
Worked Example . . . 66
Reality Check . . . 67
Chef's Notes . . . 68

Deep-Dive 4: Quantum Kernels in Detail . . . 69
In This Chapter . . . 69
The geometric picture . . . 69
Designing a feature map . . . 69
Computing the kernel value . . . 70
Building the kernel matrix and training the SVM . . . 71
When does this actually help? . . . 72
What you should take away . . . 72

Unit 5: Finance . . . 73
The Hook . . . 73
The Bottleneck . . . 73
The Quantum Angle . . . 74
Worked Example . . . 75
Reality Check . . . 75
Chef's Notes . . . 76

Deep-Dive 5: Amplitude Estimation from Grover . . . 78
In This Chapter . . . 78
Grover's algorithm: the geometric picture . . . 78
From Grover to amplitude estimation . . . 80
What you should take away . . . 81

Unit 6: Supply Chains . . . 82
The Hook . . . 82
The Bottleneck . . . 82
The Quantum Angle . . . 83
Worked Example . . . 84
Reality Check . . . 84
Chef's Notes . . . 85

Deep-Dive 6: QUBO Engineering . . . 87
In This Chapter . . . 87
From constraints to energy . . . 87
From QUBO to Ising Hamiltonian . . . 89
Quantum annealing: a different paradigm . . . 89
What you should take away . . . 90

Unit 7: Materials Science . . . 92
The Hook . . . 92
The Bottleneck . . . 92

The Quantum Angle . . . 93
Worked Example . . . 95
Reality Check . . . 95
Chef's Notes . . . 96

Deep-Dive 7: QPE and Trotterisation . . . 97
In This Chapter . . . 97
QPE: from phases to eigenvalues . . . 97
Trotterisation: implementing e^{-iHt} . . . 99
Resource estimation . . . 100
What you should take away . . . 101
Beyond Trotter: qubitization . . . 101

Unit 8: Climate & Energy . . . 102
The Hook . . . 102
The Bottleneck . . . 102
The Quantum Angle . . . 103
Worked Example . . . 105
Reality Check . . . 105
Chef's Notes . . . 106

Deep-Dive 8: Quantum Embedding Methods . . . 107
In This Chapter . . . 107
The active space idea . . . 107
DMET: Density Matrix Embedding Theory . . . 108
The complete pipeline . . . 109
What you should take away . . . 109

The Elephant in the Room: Quantum Error Correction . . . 111
Why quantum computers make mistakes . . . 111
The basic idea: redundancy without looking . . . 111
Surface codes: the leading approach . . . 112
LDPC codes: doing more with less . . . 112
Error mitigation: the NISQ bridge . . . 112
Where we are and where we're going . . . 113

What Quantum Computers Are Actually For . . . 114
What's real, what's next, what's unknown . . . 114
The one thing to remember . . . 114
Where to go from here . . . 115
Annotated Bibliography . . . 115

Preface

Most quantum computing books start with a qubit. Then two qubits. Then gates. Then circuits. Somewhere around chapter eight, if you're still reading, you might encounter a real problem.

This book does the opposite.

We start with problems. Real ones, with dollar figures attached. UPS saving $50 million a year by shaving one mile off delivery routes. Banks pricing derivatives with Monte Carlo simulations that take hours. Pharmaceutical companies spending $2 billion and twelve years to bring a single drug to market. Climate scientists searching for catalysts that could pull carbon from the atmosphere.

For each problem, we ask: what makes it hard? Not "hard" in the vague sense, but structurally hard. What is the specific mathematical bottleneck that defeats classical computers? And then: does quantum mechanics offer anything at precisely that bottleneck?

Sometimes the answer is yes. Sometimes the answer is "maybe, if hardware improves by two orders of magnitude." Sometimes the answer is "we honestly don't know yet." We give you all three, clearly labelled.

How this book is structured

The book is organised into eight **units**, each built around one industry problem. Every unit has two parts:

- The **application chapter** tells the story. It opens with the industry context, identifies the computational bottleneck, shows how a quantum algorithm addresses it, walks through a toy example, and closes with an honest assessment of where things stand today. You need no prior quantum knowledge to read these chapters.
- The **deep-dive chapter** teaches the algorithm at the circuit level. It introduces quantum concepts (qubits, gates, interference, entanglement) organically, as the algorithm needs them. These chapters are more technical, but they build everything from scratch.

Two ways to read this book

The fast path. Read only the application chapters (Units 1 through 8). You'll understand what quantum computers could do for logistics, cryptography, drug discovery, machine learning, finance, supply chains, materials science, and climate. You'll know which applications are close to practical and which are speculative. You'll be able to hold your own in any conversation about quantum computing's real-world impact.

The full path. Read the application chapters and the deep-dives. By the end, you'll understand QAOA, Shor's algorithm, VQE, quantum kernels, amplitude es-

timation, QUBO, QPE with Trotterisation, and quantum embedding methods. Not as abstract constructions, but as tools built to solve specific problems.

Either path works. The deep-dives never assume you've read other deep-dives out of order; each one states its prerequisites explicitly.

The notebooks

Every unit has a companion Jupyter notebook that runs the algorithm on a real quantum backend. These aren't toy demonstrations; they're the same algorithms described in the text, shrunk to a size that fits on current hardware. You can run them, modify them, and break them. That's the point.

What we don't promise

We don't promise that quantum computers will revolutionise your industry next year. Some of the applications in this book require fault-tolerant quantum computers that don't exist yet. Every chapter includes a **Reality Check** section that states, plainly, what works today on noisy hardware, what needs error correction, and what remains an open question.

We use "could" and "in principle" where others use "will" and "soon." We cite specific resource estimates. We acknowledge when classical algorithms have closed the gap. This is not a hype document.

Who this book is for

You're curious about quantum computing, but you want to know what it's for before learning how it works. Maybe you're a software engineer evaluating whether to invest time in quantum. Maybe you're a domain expert in finance or pharma who's heard the promises and wants specifics. Maybe you're a student who finds qubits abstract and wants motivation before formalism.

If you're looking for a comprehensive quantum computing textbook, read Nielsen and Chuang. If you want to build quantum circuits from scratch, see our companion project, the Quokka Cookbook. This book is for people who want to start with the "why."

Let's start with a delivery truck.

Unit 1: Logistics

Saving $50 Million One Mile at a Time

The Hook

Every morning, 130,000 UPS drivers leave their depots. Each driver visits between 100 and 200 stops. The order they visit those stops determines how many miles they drive, how much fuel they burn, and whether they make it home for dinner.

In 2012, UPS deployed a system called ORION: On-Road Integrated Optimization and Navigation. Its job: take each driver's list of deliveries and produce a better route. Not a perfect route. Just a better one. ORION shaved an average of one mile off each driver's daily route.

One mile doesn't sound like much. But multiply by 130,000 drivers, 250 working days per year, and the cost of fuel, vehicle wear, and driver time. UPS reported savings of **$50 million per year** from that single mile.

Now consider what a two-mile improvement would be worth. Or five.

The problem is that finding the optimal route isn't just difficult; it's one of the hardest problems in all of computer science. If a driver has 20 stops, there are $20! = 2.4 \times 10^{18}$ possible orderings. That's 2.4 quintillion routes to check. A desktop computer evaluating one billion routes per second would need 77 years. For 50 stops, the number of possibilities exceeds the number of atoms in the observable universe.

This is the **Travelling Salesman Problem** (TSP), and it has tormented mathematicians, computer scientists, and logistics companies for over a century. Every exact algorithm for TSP has a running time that grows at least exponentially with the number of stops. There is strong theoretical evidence (though no proof) that no efficient exact algorithm exists; the problem is NP-hard.

In practice, nobody solves TSP exactly at scale. ORION doesn't find the best route; it finds a pretty good route, using heuristics that have been refined over decades. Branch-and-bound, genetic algorithms, simulated annealing, Lin-Kernighan; these are all sophisticated methods for exploring the vast space of possible routes without drowning in it.

They work remarkably well. But they all share a fundamental limitation: they explore the space of routes one at a time. They walk through the landscape of possible solutions, stepping from one candidate to a neighbouring one, hoping that local improvements lead to global ones. Sometimes they get stuck in valleys; solutions that are better than all their neighbours but far worse than the global optimum.

What if you could explore the entire landscape at once?

The Bottleneck

Let's make the problem concrete. Strip away the geography and the delivery trucks, and what remains is pure combinatorics.

You have a graph; a set of cities (nodes) connected by roads (edges), each road with a cost (the distance). You want to visit every city exactly once and return to where you started, minimising the total cost. That's TSP.

For our purposes, let's start with something even simpler: **MaxCut**. Take a graph and colour every node either red or blue. An edge is "cut" if its endpoints have different colours. MaxCut asks: what colouring cuts the most edges?

MaxCut sounds like it has nothing to do with delivery trucks. But TSP, MaxCut, and virtually every combinatorial optimisation problem share the same deep structure: you have n binary decisions, a cost function that depends on how those decisions interact, and an exponentially large space of possibilities to search. The difference is just what the decisions represent (which city to visit next vs. what colour to assign) and what the cost counts (total distance vs. cut edges). The quantum approach we're about to build works on any problem with this structure. We're using MaxCut because it's the cleanest example — each decision is one qubit, and the cost function translates directly into the language of quantum mechanics. Once you see how QAOA handles MaxCut, the path to TSP, vehicle routing, and scheduling is a change of cost function, not a change of algorithm.

Here's why MaxCut is hard. A graph with n nodes has 2^n possible colourings. For each colouring, you can count the cut edges in $O(m)$ time, where m is the number of edges. So the brute-force algorithm takes $O(m \cdot 2^n)$ time. For $n = 50$, that's roughly 10^{15} operations; doable but slow. For $n = 100$, it's 10^{30} operations. For $n = 300$, you'd need more time than the age of the universe.

The problem is the search space. Every node is a binary decision: red or blue, 0 or 1. The decisions interact; the value of your choice for node i depends on what you chose for nodes j, k, l; every node that shares an edge with i. These interactions create a rugged cost landscape: a function over $\{0, 1\}^n$ with exponentially many local optima.

Classical heuristics navigate this landscape by making local moves. Flip one node's colour. If the cut count improves, keep it; otherwise, try another flip. Simulated annealing adds randomness; occasionally accept a worse solution to escape local optima, gradually reducing the randomness as you "cool down."

These methods are powerful, but they have a fundamental weakness: they explore the landscape sequentially. At any moment, they're sitting at a single point in the space of 2^n possible colourings. They can only see the immediate neighbourhood. A better solution might exist on the other side of a high barrier, and no amount of local flipping will find it.

This is where the story turns quantum.

The Quantum Angle

What if, instead of sitting at one colouring and looking around, you could be at all 2^n colourings simultaneously, and then use interference to amplify the ones that cut more edges?

That's the core idea behind the **Quantum Approximate Optimization Algorithm** (QAOA), introduced by Farhi, Goldstone, and Gutmann in 2014. It's not a magic bullet; it won't solve NP-hard problems in polynomial time (nobody believes that's possible). But it offers a fundamentally different way to explore the solution landscape, one that exploits quantum mechanics in a structural way that classical algorithms cannot mimic.

Before we dive into QAOA itself, let's step back and sketch how quantum algorithms work in general. Nearly every quantum algorithm follows the same three-act structure:

1. **Encode** the problem into a quantum system. Binary decisions become qubits. Costs become operators. The search space becomes the space of quantum states.
2. **Manipulate** the quantum state using carefully chosen operations. This is where the quantum magic happens: interference suppresses bad solutions and amplifies good ones, without ever looking at them one by one.
3. **Measure** the result. Measurement collapses the quantum state to a single classical answer. If the manipulation was done well, that answer is likely to be a good one.

That's the whole recipe. The art is in step 2, and it's different for every algorithm. For QAOA, the manipulation is a rhythmic alternation between "imprint the cost" and "mix the solutions." For Shor's algorithm (Unit 2), it's a Fourier transform. For VQE (Unit 3), it's a chemistry-inspired trial state. But the three-act structure is always the same.

Let's see how it plays out for MaxCut.

Qubits as binary decisions

The first step is encoding. Each node in the graph gets one qubit. A classical bit is either 0 or 1. A qubit is the quantum version: it has two definite states, which physicists write as $|0\rangle$ and $|1\rangle$. The vertical bar and angle bracket are just packaging — the notation is called a "ket," and it's how physicists label quantum states. You can read $|0\rangle$ as "the state labelled zero" and $|1\rangle$ as "the state labelled one." When you measure a qubit, you always get one of these two outcomes: 0 or 1, just like a classical bit. The difference is what happens before you measure — but we'll get to that in a moment.

For MaxCut, $|0\rangle$ means "red" and $|1\rangle$ means "blue." An n-node graph requires n qubits. A specific colouring — say, nodes 1 and 3 are blue, node 2 is red — corresponds to the quantum state $|101\rangle$ (just the individual qubit labels strung together).

So far, this is just notation. The quantum part starts when we use superposition.

> **Superposition.** A qubit doesn't have to be $|0\rangle$ or $|1\rangle$. It can be in a state like $\alpha|0\rangle + \beta|1\rangle$, where α and β are numbers (complex, if you care, but the intuition works with real numbers) satisfying $|\alpha|^2 + |\beta|^2 = 1$. The values $|\alpha|^2$ and $|\beta|^2$ are the probabilities of getting 0 or 1 when you measure. Before measurement, both possibilities coexist. This isn't a metaphor; it's how the physics works. When you put n qubits into superposition, the system represents all 2^n possible states at once. You'll hear people say "the quantum computer tries all answers simultaneously." That's misleading. What really happens is subtler: the 2^n possibilities can interfere with each other, and clever algorithms arrange for the good answers to reinforce and the bad ones to cancel. Deep-Dive 1 makes this precise.

With n qubits in superposition, we have all 2^n colourings present in a single quantum state. We can't look at them all; measurement gives us just one. But between preparation and measurement, we can manipulate the amplitudes to make good colourings more likely to appear. That's the game.

Turning costs into energy

Now we need to tell the quantum computer what "good" means. We need to translate the MaxCut objective; "cut as many edges as possible"; into something a quantum system can work with.

Physicists have a word for "the function that assigns a number to every state of a system": they call it a **Hamiltonian**. In physics, the Hamiltonian describes the total energy, and the system naturally evolves toward its lowest-energy state. We're going to borrow this idea: assign an "energy" to every colouring, where lower energy means more edges cut. Then "find the best colouring" becomes "find the lowest-energy state."

This reframing is not just convenient; it's the key insight that makes quantum optimisation work. Quantum systems are built to find low-energy states. It's what they do naturally. By encoding our problem as a Hamiltonian, we're asking quantum mechanics to do what it already does best.

Here's how we build the Hamiltonian for MaxCut. For each edge (i, j), we need an expression that equals 1 when the edge is cut and 0 when it isn't. The trick uses a quantum operator called Z (the Pauli-Z operator, which gives $+1$ for $|0\rangle$ and -1 for $|1\rangle$):

$$\frac{1 - Z_i Z_j}{2}$$

Does it work? Check all four cases:

q_i	q_j	Same colour?	Z_i	Z_j	$Z_i Z_j$	$\frac{1}{2}(1 - Z_i Z_j)$
$\|0\rangle$ (red)	$\|0\rangle$ (red)	yes	$+1$	$+1$	$+1$	0 — uncut

q_i	q_j	Same colour?	Z_i	Z_j	Z_iZ_j	$\frac{1}{2}(1 - Z_iZ_j)$
$\|0\rangle$ (red)	$\|1\rangle$ (blue)	no	$+1$	-1	-1	1 — **cut**
$\|1\rangle$ (blue)	$\|0\rangle$ (red)	no	-1	$+1$	-1	1 — **cut**
$\|1\rangle$ (blue)	$\|1\rangle$ (blue)	yes	-1	-1	$+1$	0 — uncut

The formula returns 1 exactly when the edge is cut and 0 when it isn't. That's all we need.

The total cost Hamiltonian sums this over every edge:

$$C = \sum_{(i,j)\in E} \frac{1 - Z_iZ_j}{2}$$

The state with the highest value of C is the colouring that cuts the most edges. Equivalently, the **ground state** of $-C$ (the lowest-energy state) is the optimal solution.

This pattern; "encode the objective as a Hamiltonian, then find its ground state"; is the central trick of quantum optimisation, and it recurs throughout this book. In Unit 3, the Hamiltonian encodes molecular energy. In Unit 6, it encodes scheduling constraints. In Unit 7, it encodes the physics of a material. The problems are different, but the strategy is the same: translate "find the best X" into "find the lowest energy."

Reading a quantum circuit

Before we look at the QAOA circuit, a quick orientation on how quantum programs are drawn.

A quantum circuit is read left to right, like sheet music. Each horizontal line (called a **wire**) represents one qubit. Boxes or symbols on the wires represent **operations** (also called gates). A gate on one wire acts on that qubit alone; a gate connecting two wires (drawn with a vertical line between them) acts on both qubits together. At the end, a measurement symbol (M or a meter icon) reads out the qubit's value as a classical 0 or 1.

Here's what that looks like in practice. The circuit below implements **quantum teleportation** — moving the state of one qubit to another using entanglement and two classical bits. You don't need to understand it yet; just notice the structure:

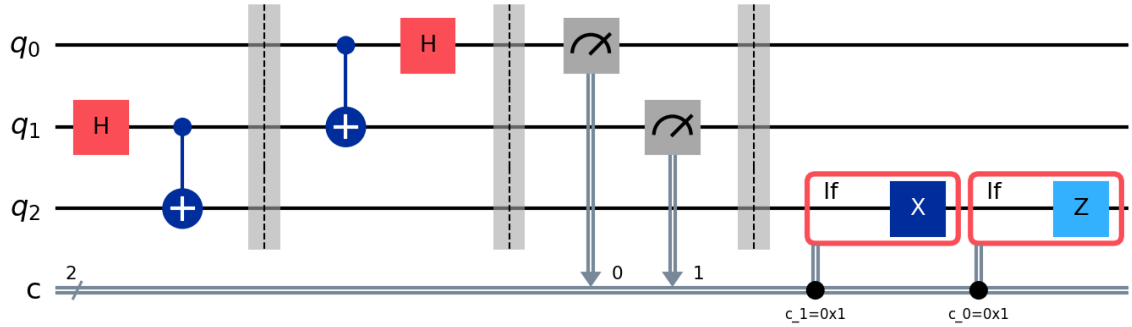

Figure 1. Quantum teleportation circuit: three qubit wires read left to right, with Hadamard and CNOT gates creating entanglement, measurements, and classically controlled corrections

Three wires, three qubits. Gates (H, CNOT) sit on the wires at specific moments in time. The grey arrows carry classical measurement results down to later gates. Read left to right, the circuit tells you exactly what happens and when — just like a musical score tells musicians what to play and when.

We'll return to teleportation in later units. For now, this is the visual language for every quantum algorithm in the book.

> **Unitary.** You'll see quantum operations called "unitary." This is a technical term from linear algebra that means two things: the operation is reversible (you can always undo it), and it preserves probabilities (they still add up to 1 afterwards). Every quantum gate is unitary. Measurement is not; it's the one irreversible step, where the quantum state collapses to a definite classical value. Deep-Dive 1 goes deeper into the mathematics.

A few gates we'll meet repeatedly:

- H (Hadamard): puts a qubit into superposition. Turns $|0\rangle$ into an equal mix of $|0\rangle$ and $|1\rangle$.
- Z (Pauli-Z): you've already met this; it gives $+1$ for $|0\rangle$ and -1 for $|1\rangle$. Related gates $R_Z(\theta)$ rotate by a tunable angle.
- X (Pauli-X): flips a qubit; $|0\rangle \rightarrow |1\rangle$ and vice versa. Related gate $R_X(\theta)$ rotates by a tunable angle.
- CNOT: a two-qubit gate that flips the second qubit if the first is $|1\rangle$. This is how qubits become entangled; correlated in ways that have no classical analogue.

With that vocabulary, let's look at the QAOA circuit.

The QAOA circuit

Here is the entire QAOA algorithm for our three-node MaxCut problem:

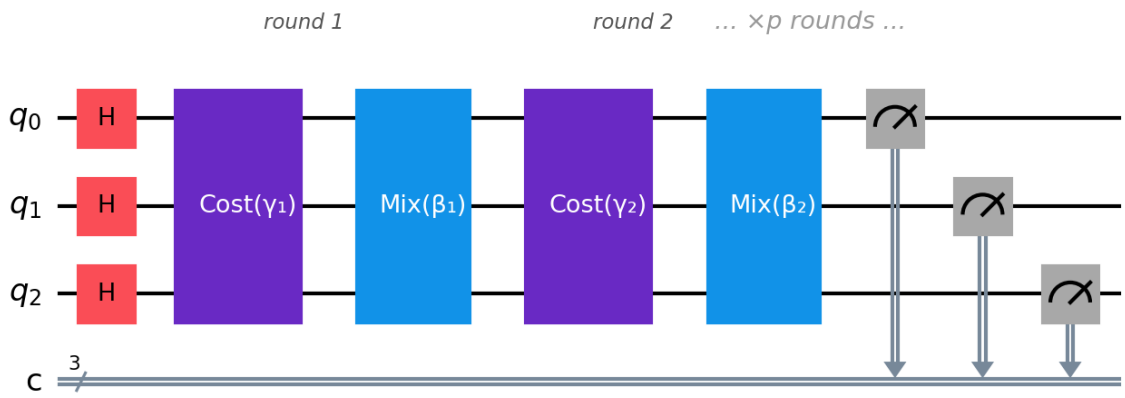

Figure 2. QAOA circuit: Hadamard initialisation, then repeated rounds of Cost and Mix with different angles each round, then measurement

Read left to right. Initialise, then alternate between two boxes — Cost and Mix — for p rounds, each round with its own pair of angles (γ_k, β_k). Then measure. That's the entire algorithm. The angles are different in every round: round 1 uses γ_1, β_1; round 2 uses γ_2, β_2; and so on. There are $2p$ parameters in total, and choosing them well is the whole game. We'll get to that. First, let's open each box.

Box 1: Initialisation. A Hadamard gate H on every qubit. This creates an equal superposition of all 2^n colourings, each with the same amplitude $1/\sqrt{2^n}$. Think of it as a blank canvas: every possible solution is present, and none is preferred. If you measured right now, you'd get a uniformly random colouring. Not useful yet.

Box 2: The cost phase ($e^{-i\gamma_k C}$). This is where our Hamiltonian earns its keep. For each edge (i, j), the circuit applies a rotation whose angle is proportional to γ_k. The effect: every colouring in the superposition picks up a phase that depends on its cut count. Colourings that cut more edges get one phase; colourings that cut fewer get a different phase.

But here's the catch: phase changes alone are invisible to measurement. The probabilities haven't changed at all. If you measured right now, you'd still get a uniformly random colouring. The cost phase has written information in invisible ink: it tagged good solutions differently from bad ones, but the tags are hidden in the phases, and measurement only sees probabilities.

So what good are the phases? This is where interference enters.

> **Interference.** Quantum amplitudes aren't just numbers — they have both a magnitude and a direction (technically, a complex phase). Think of them as arrows. When two arrows point the same way, they reinforce: their magnitudes add. When they point in opposite directions, they cancel: their magnitudes subtract. This is interference, and it's the mechanism that makes quantum computing fundamentally different from classical computing.
>
> Superposition gives you the blank canvas — all 2^n solutions present at once. But superposition alone is useless: you can't look at all the solutions, and a random one is no better than a coin flip. Interfer-

> ence is what sculpts the canvas. It converts the phase information (which solution is good, which is bad) into probability information (which solution you're likely to measure). Every quantum algorithm is, at its core, an interference machine. The details differ — Shor's algorithm uses the Quantum Fourier Transform to create constructive interference at the period. Grover's uses the diffusion operator. QAOA uses the mixer. But the trick is always the same: arrange for good answers to reinforce and bad answers to cancel.

Box 3: The mixer ($e^{-i\beta_k B}$). An $R_X(2\beta_k)$ rotation on every qubit. This "stirs" the superposition, causing amplitudes from different colourings to overlap and interfere. Because the cost phase tagged good colourings with different phases from bad ones, the mixer's stirring produces constructive interference for high-cut colourings and destructive interference for low-cut ones. After the mixer, the probabilities are no longer uniform: good solutions are more likely to appear when you measure.

Then repeat. One round of cost-then-mix shifts the probabilities a little. Two rounds shift them more. Each round uses fresh angles (γ_k, β_k), so the algorithm can tag and stir with different intensities at each step — like a photographer adjusting focus and exposure between shots. As $p \to \infty$, QAOA converges to the exact optimum. In practice, you use a small p and accept an approximate answer — hence "Approximate" in the name.

But who chooses the $2p$ angles? You could try random values, but that's wasteful. Instead, QAOA uses a **variational loop** — a quantum-classical hybrid:

1. Pick initial values of $\gamma_1, \beta_1, \dots, \gamma_p, \beta_p$
2. Run the quantum circuit, measure the output
3. Compute the expected cost (run many times, take the average)
4. Feed this cost to a classical optimiser (COBYLA, Nelder-Mead, gradient descent)
5. The optimiser suggests better angles
6. Repeat until convergence

The quantum computer explores the solution space. The classical computer tunes the exploration strategy. Neither could do the other's job efficiently.

For the gate-level decomposition of each box — how the cost phase becomes a sequence of CNOTs and R_Z rotations, and what R_X does geometrically — see Deep-Dive 1.

Worked Example

Let's solve MaxCut on a triangle; three nodes, three edges. The optimal cut is 2 (colour one node differently from the other two).

The graph

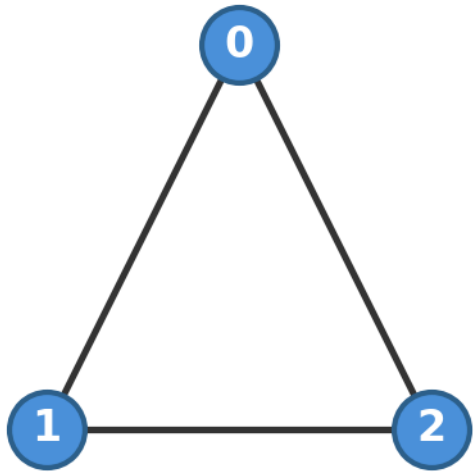

Figure 3. MaxCut triangle graph: nodes 0, 1, 2 with edges (0,1), (0,2), (1,2)

Three nodes, three edges: (0,1), (0,2), (1,2). Each node gets one qubit.

The cost Hamiltonian

$$C = \frac{1 - Z_0 Z_1}{2} + \frac{1 - Z_0 Z_2}{2} + \frac{1 - Z_1 Z_2}{2}$$

What QAOA produces

Using pre-optimised parameters ($\gamma \approx \pi/4$, $\beta \approx \pi/8$), a single round of QAOA produces this measurement distribution:

Outcome	Cut value	Probability
000	0	~3%
001	2	~16%
010	2	~16%
011	2	~16%
100	2	~16%
101	2	~16%
110	2	~16%
111	0	~3%

The colourings with cut value 2 (the optimum for the triangle) are amplified. The "all same" colourings (000, 111) are suppressed. QAOA has shifted probability mass toward the optimal solutions.

→ The next chapter builds this circuit gate by gate, runs it, and shows you the code.

Back to the trucks

We started with UPS drivers and 20-stop routes. We solved MaxCut on a triangle. How do you get from one to the other?

The same machinery applies. TSP and its industrial cousin, the **Vehicle Routing Problem** (VRP), can be written as cost functions over binary variables — which city does driver k visit at time step t? The cost Hamiltonian becomes a sum of distance terms plus penalty terms that enforce constraints (visit every city exactly once, return to the depot, respect vehicle capacity). The encoding is more complex than MaxCut — you need $O(n^2)$ qubits instead of n, and the constraint penalties add terms to the Hamiltonian — but the algorithm is identical: initialise in superposition, alternate cost and mixer phases, measure. QAOA doesn't care whether the cost function counts cut edges or travel distance. It just needs a Hamiltonian.

That's also why we'll see this same pattern in every unit of this book. Drug molecules, financial portfolios, nurse schedules, material properties — they all become Hamiltonians, and the quantum algorithm searches for the ground state. The problem changes. The strategy doesn't.

Reality Check

Let's be honest about where QAOA stands; and where it's heading.

The low-depth story. For depth-1 QAOA on MaxCut, Farhi, Goldstone, and Gutmann (2014) proved a guaranteed approximation ratio of 0.6924 for 3-regular graphs. The best classical polynomial-time algorithm (Goemans-Williamson, 1995) achieves 0.878. So at low depth, QAOA underperforms the best classical method. This is worth stating plainly: if you can only run one or two rounds, you're better off with Goemans-Williamson.

Beyond low depth. QAOA's performance improves monotonically with depth p, and a body of work has made it possible to compute exact QAOA performance at depths that were previously out of reach. MaxCut is the $k = 2$ case of a broader family called Max-k-XORSAT, where each constraint involves k variables connected by an XOR. For D-regular instances, Basso et al. (2021) showed that QAOA's expected performance in the infinite-size limit can be computed via a tree tensor network contraction — a classical calculation whose cost is exponential in p but independent of the graph size. Farhi et al. (2025) pushed this to depth $p = 20$ for MaxCut on 3-regular graphs.

Meanwhile, a different quantum approach has emerged: **Decoded Quantum Interferometry** (DQI), a non-variational algorithm introduced by Jordan et al. (Nature, 2025). DQI combined with Belief Propagation post-processing (DQI+BP) achieves strong results on Max-k-XORSAT without any classical optimiser loop, and was the state of the art for several (k, D) parameter regimes.

Recent work by Azariah and Jordan (2026) extended the tree tensor network approach to Max-k-XORSAT for $k \geq 3$, enabling exact QAOA evaluation through

depth $p = 13$. These calculations show that QAOA at sufficient depth is competitive with DQI+BP across most parameter regimes tested — though each algorithm has its strengths.

The hardware gap. These are theoretical results; computing what QAOA would achieve if you could run it at depth $p = 12$ on a large enough quantum computer. Today's hardware can run QAOA at $p = 1$ or $p = 2$ on a few dozen noisy qubits. The gap between where the algorithm shines ($p \geq 8$) and where hardware can operate ($p \leq 2$) is real. Fault-tolerant quantum computers with $\sim 10^4$ logical qubits would close this gap.

The barren plateau question. McClean et al. (2018) showed that for random quantum circuits, the cost function gradient vanishes exponentially with system size. This is a serious concern for variational algorithms. However, QAOA has far more structure than a random circuit; its alternating problem/mixer layers are determined by the problem graph. Barren plateaus have not been observed in practice for QAOA at moderate depths, and the exact evaluations confirm that the parameter landscape remains navigable through $p = 13$.

What's real today: QAOA's algorithmic performance is now well-characterised for an important family of combinatorial problems. The algorithm is provably competitive with the best known quantum alternatives. The bottleneck is hardware, not the algorithm.

Chef's Notes

- **MaxCut is a gateway drug.** We used it here because it maps directly to qubits ($Z_i Z_j$ interactions), but QAOA applies to any combinatorial optimisation problem you can write as a cost function over binary variables. Scheduling, bin packing, graph colouring, portfolio optimisation; they all fit. MaxCut and k-XORSAT are the problems where we can compute QAOA's exact performance; for other problems, we rely on empirical evaluation.
- **The Max-k-XORSAT family.** MaxCut is the $k = 2$ case of a broader family: Max-k-XORSAT, where each constraint involves k variables connected by an XOR. As k increases, the problem gets harder classically (the random satisfiability threshold drops) but the QAOA analysis extends cleanly; the cost Hamiltonian becomes a sum of k-body Z-interactions instead of 2-body. The same tree tensor network machinery evaluates QAOA performance for any k, with a computational trick (Walsh-Hadamard factorisation) that makes $k \geq 3$ tractable.
- **The cost Hamiltonian pattern recurs everywhere.** The idea of encoding an objective function as a quantum operator whose ground state is the optimal solution appears in Unit 3 (molecular simulation; the Hamiltonian encodes the energy), Unit 6 (QUBO/annealing), and Unit 7 (QPE). Once you see the pattern, you see it everywhere.
- **QAOA vs. quantum annealing.** QAOA explores the cost landscape via alternating unitaries (gate-based). Quantum annealing explores it via adiabatic evolution (slowly changing the Hamiltonian). They're different paradigms for the same goal. We'll see annealing in Unit 6 (Supply Chains).

- **The variational quantum-classical loop appears again in VQE** (Unit 3). If you understood the QAOA loop; quantum circuit produces a cost estimate, classical optimiser updates parameters, repeat; then you already understand VQE's architecture. The only difference is the cost function (molecular energy vs. cut count) and the ansatz (chemistry-motivated vs. graph-motivated).
- **Further reading:**
  - Farhi, Goldstone, Gutmann (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028
  - Basso, Farhi, Marwaha, Villalonga, Zhou (2021). The Quantum Approximate Optimization Algorithm at High Depth for MaxCut on Large-Girth Regular Graphs and the Sherrington-Kirkpatrick Model. arXiv:2110.14206
  - Farhi, Gutmann, Ranard, Villalonga (2025). Lower bounding the MaxCut of high girth 3-regular graphs using the QAOA. arXiv:2503.12789
  - Jordan, Shutty, Wootters, Zalcman, et al. (2025). Optimization by decoded quantum interferometry. Nature (2025) (arXiv:2408.08292)
  - Azariah and Jordan (2026). Exact QAOA on Max-k-XORSAT via generic tree folding. (Working title; in preparation.)

Deep-Dive 1: Building QAOA from Gates

This deep dive teaches the QAOA algorithm at the circuit level — how every piece becomes concrete operations on qubits. It pairs with Unit 1 (Logistics), which introduced the problem and the algorithm conceptually.

In This Chapter

- **What you'll learn:** How to encode a MaxCut problem as a quantum circuit, what each gate does and why, and how the classical optimiser tunes the quantum exploration.
- **What you need:** No prior quantum knowledge. This is the first deep dive; we introduce qubits, gates, superposition, and entanglement here, in context, because the algorithm needs them.
- **Runnable version:** The companion notebook `01-logistics.ipynb` runs the same circuit on a cloud Quokka.

Where we are and what we need

We have a cost function; the MaxCut objective; and we've learned that QAOA encodes it as a quantum operator (the cost Hamiltonian C), applies two alternating unitaries ($e^{-i\gamma C}$ and the mixer $e^{-i\beta B}$), and measures. What we haven't explained is how any of this becomes an actual quantum circuit; specific gates applied to specific qubits in a specific order.

Let's fix that. We'll build the entire circuit for MaxCut on a triangle (3 nodes, 3 edges), gate by gate, explaining why each gate is needed and what it does to the quantum state. By the end, you'll be able to construct QAOA circuits for any graph.

Binary decisions become qubits

The first step is representation. In MaxCut, each node is coloured red or blue; a binary decision. Each binary decision becomes one qubit:

Node	Qubit	$\lvert 0\rangle$ meaning	$\lvert 1\rangle$ meaning
0	`q[0]`	Red	Blue
1	`q[1]`	Red	Blue
2	`q[2]`	Red	Blue

A specific colouring; say, node 0 is blue, nodes 1 and 2 are red; corresponds to the quantum state $|100\rangle$. The set of all possible colourings is the set of all 3-qubit computational basis states: $|000\rangle, |001\rangle, \ldots, |111\rangle$. There are $2^3 = 8$ of them.

So far, nothing quantum. A classical computer could do the same: 3 bits, 8 combinations. The quantum part starts when we put the qubits in superposition.

Creating the starting state

QAOA begins by creating an equal superposition of all possible colourings. We want:

$$|s\rangle = \frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)$$

Every colouring has the same amplitude ($1/\sqrt{8}$), so every colouring has the same probability ($1/8$) of being measured. We're starting from complete ignorance; no colouring is preferred.

How do you create this state? With the **Hadamard gate** (H), applied to each qubit independently:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Apply H to qubit 0, then qubit 1, then qubit 2. Because the qubits are independent at this point, the combined state is the tensor product:

$$H|0\rangle \otimes H|0\rangle \otimes H|0\rangle = \frac{1}{\sqrt{8}} \sum_{x \in \{0,1\}^3} |x\rangle$$

In circuit form:

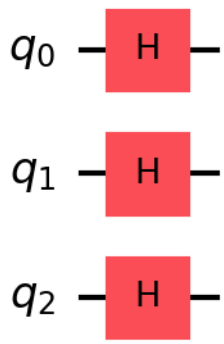

Figure 4. Hadamard gates applied to all 3 qubits

Three gates. That's all it takes to create a uniform superposition of 8 colourings. For n qubits, it's n Hadamard gates and 2^n terms in superposition. The exponential scaling is free.

> **Common Mistake #1:** Students sometimes say "the quantum computer is trying all 8 colourings at once." This is misleading. All 8 colourings are present in the superposition, but you can only read

> out one of them (the one you measure). The power of QAOA is not parallel evaluation; it's interference: manipulating the amplitudes so that good colourings become more likely to be measured.

The problem unitary: turning edge costs into phases

Now we need to "imprint" the cost function onto the quantum state. We want colourings that cut many edges to acquire different phases from colourings that cut few edges. Later, the mixer will cause these phases to interfere, amplifying the good solutions.

The cost Hamiltonian for our triangle is:

$$C = \frac{1 - Z_0 Z_1}{2} + \frac{1 - Z_0 Z_2}{2} + \frac{1 - Z_1 Z_2}{2}$$

Each term contributes 1 when the edge is cut (qubits differ) and 0 when it's not (qubits agree). The total is the number of cut edges.

The problem unitary is $e^{-i\gamma C}$. What does this do? For each computational basis state $|x\rangle$:

$$e^{-i\gamma C}|x\rangle = e^{-i\gamma \cdot \mathrm{cut}(x)}|x\rangle$$

It multiplies each colouring by a phase that depends on how many edges it cuts. The colouring itself doesn't change; only its phase. Colourings with cut value 2 (the optimum for the triangle) get phase $e^{-2i\gamma}$. Colourings with cut value 0 get phase $e^0 = 1$. Different phases, same probabilities (for now).

But how do we implement $e^{-i\gamma C}$ as gates? Since the three edge terms in C all commute with each other (they're all diagonal in the computational basis), we can apply them separately:

$$e^{-i\gamma C} = e^{-i\gamma(1-Z_0Z_1)/2} \cdot e^{-i\gamma(1-Z_0Z_2)/2} \cdot e^{-i\gamma(1-Z_1Z_2)/2}$$

Each factor handles one edge. Let's focus on the first edge, $(0, 1)$.

The ZZ gate: one edge at a time

We need to implement $e^{-i\gamma(1-Z_0Z_1)/2}$. The constant part $e^{-i\gamma/2}$ is just a global phase (it multiplies every state equally), so we can ignore it; global phases are unobservable. What remains is $e^{i\gamma Z_0 Z_1/2}$.

Now, $Z_0 Z_1$ is diagonal in the computational basis. Its eigenvalues are:

q_0	q_1	Z_0	Z_1	Z_0Z_1	Meaning
0	0	+1	+1	+1	Same colour; edge not cut
0	1	+1	−1	−1	Different colours; edge cut

q_0	q_1	Z_0	Z_1	Z_0Z_1	Meaning
1	0	-1	$+1$	-1	Different colours; edge cut
1	1	-1	-1	$+1$	Same colour; edge not cut

So $e^{i\gamma Z_0 Z_1/2}$ applies phase $e^{i\gamma/2}$ when the qubits agree and $e^{-i\gamma/2}$ when they disagree.

How do we build this from standard gates? The trick is the **CNOT sandwich**:

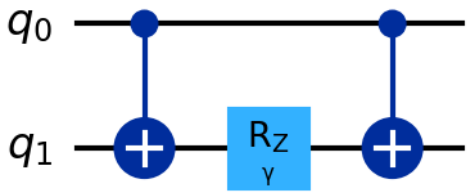

Figure 5. ZZ gate implemented as a CNOT sandwich

Here's what happens step by step:

1. **First CNOT** (`cx q[0], q[1]`): XORs qubit 1 with qubit 0. Now qubit 1 holds the parity of the two original bits: it's $|0\rangle$ if they were equal, $|1\rangle$ if they were different.
2. $R_Z(\gamma)$ **on qubit 1** (`rz(γ) q[1]`): applies phase $e^{-i\gamma/2}$ if qubit 1 is $|1\rangle$ (qubits were different = edge cut), and $e^{i\gamma/2}$ if qubit 1 is $|0\rangle$ (qubits were same = edge not cut). This is exactly the phase we want.
3. **Second CNOT** (`cx q[0], q[1]`): undoes the XOR, restoring qubit 1 to its original value.

Net effect: the computational basis states are unchanged, but each one picks up a phase that depends on whether the two qubits agree or disagree. That's $e^{i\gamma Z_0 Z_1/2}$.

For edge $(0, 1)$, this is the same CNOT sandwich shown above. Repeat for edges $(0, 2)$ and $(1, 2)$, and you've implemented the full problem unitary.

> **Why does this work?** The CNOT "extracts" the parity into a single qubit, the R_Z "acts" on that parity, and the second CNOT "puts it back." This is a general trick: whenever you need a gate that depends on the relationship between two qubits (same? different?), you can use a CNOT to compute that relationship, act on it, and undo the CNOT.

The mixer: where phases become probabilities

After the problem unitary, the quantum state has different phases for different colourings, but the probabilities are still all equal ($1/8$ each). If you measured now,

you'd get a uniformly random colouring — the phases are invisible to measurement. So what was the point?

The point is that phases are potential energy for interference. The mixer is the step that unleashes it.

Let's see this concretely. Before the mixer, our state looks like:

$$\frac{1}{\sqrt{8}}(e^{i\phi_0}|000\rangle + e^{i\phi_1}|001\rangle + \cdots + e^{i\phi_7}|111\rangle)$$

where each ϕ_x depends on the cut value of colouring x. Every amplitude has the same magnitude ($1/\sqrt{8}$) but a different direction (the phase ϕ_x). If you think of amplitudes as arrows, they're all the same length but point in different directions.

Now the mixer acts. It applies $R_X(2\beta)$ to each qubit, which mixes $|0\rangle$ and $|1\rangle$ within each qubit. The effect on the full state is that every colouring's amplitude gets contributions from neighbouring colourings — those that differ by a single bit flip. Each contribution arrives carrying the phase it picked up from the cost step.

Here's the crucial point: when contributions arrive with similar phases, their arrows point roughly the same way and they add up (constructive interference). When they arrive with different phases, their arrows point in opposing directions and they partially cancel (destructive interference).

Let's see this on our triangle. Take the good colouring $|001\rangle$ (cut = 2). Its three neighbours — the colourings one bit-flip away — are:

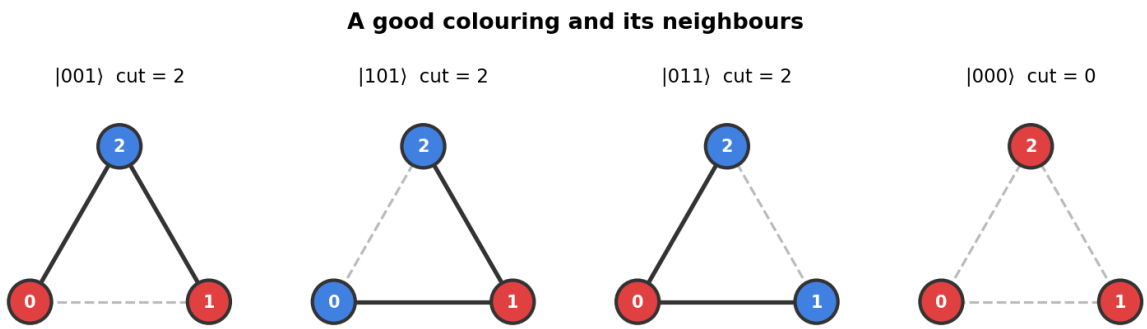

Figure 6. A good colouring (|001⟩, cut=2) and its three neighbours: two also have cut=2, one has cut=0

Two out of three neighbours also have cut value 2 — the same as the original. They picked up similar phases from the cost step, so when their contributions arrive during mixing, the arrows point roughly the same way. Constructive interference: $|001\rangle$'s amplitude grows.

Now take the poor colouring $|000\rangle$ (cut = 0). Its neighbours:

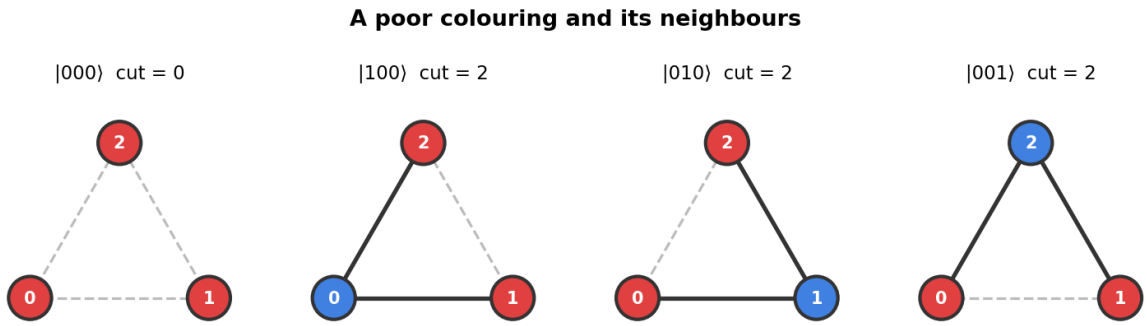

Figure 7. A poor colouring (|000⟩, cut=0) and its three neighbours: all three have cut=2

Every neighbour has cut value 2 — all different from the original's cut value of 0. They carry phases that are very different from $|000\rangle$'s phase, so the arriving contributions point in scattered directions. Destructive interference: $|000\rangle$'s amplitude shrinks.

The result: after the mixer, the probabilities are no longer uniform. Good colourings have grown more likely; poor colourings have shrunk.

This is the heart of QAOA. The cost phase writes information in the phases. The mixer reads it via interference. Neither step works alone — phases without mixing are invisible, and mixing without phases is just random stirring.

The mixer Hamiltonian is $B = X_0 + X_1 + X_2$, and the mixer unitary is $e^{-i\beta B}$. Since X operators on different qubits commute, this factorises:

$$e^{-i\beta B} = e^{-i\beta X_0} \cdot e^{-i\beta X_1} \cdot e^{-i\beta X_2}$$

Each factor is just an R_X rotation: $e^{-i\beta X} = R_X(2\beta)$:

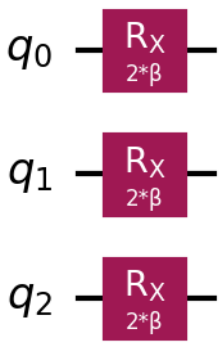

Figure 8. Mixer: Rx(2β) applied to each qubit

What does $R_X(\theta)$ do? It rotates the qubit's state around the X axis of the Bloch sphere. In the computational basis, it mixes $|0\rangle$ and $|1\rangle$:

$$R_X(\theta)|0\rangle = \cos(\theta/2)|0\rangle - i\sin(\theta/2)|1\rangle$$

At $\beta = 0$, the mixer does nothing ($R_X(0) = I$). At $\beta = \pi/2$, it flips every qubit ($R_X(\pi) = -iX$). In between, it partially mixes — allowing amplitude to flow between colourings that differ by one bit flip. This is the quantum analogue of the "flip one node" move in classical local search, but done coherently so that the phases from the cost step drive the interference we described above.

The full circuit, assembled

Putting it all together for the triangle, at depth $p = 1$:

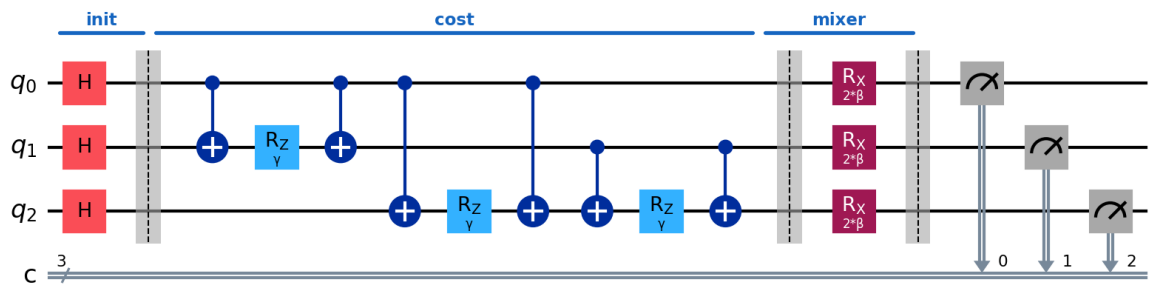

Figure 9. Complete QAOA circuit for MaxCut on a triangle: Hadamard initialisation, three ZZ cost unitaries (one per edge), Rx mixer, and measurement

Gate count: 3 Hadamards + 6 CNOTs + 3 R_Z + 3 R_X + 3 measurements = **18 gates**, of which 6 are the expensive ones (CNOTs). For a general graph with n nodes and m edges at depth p: $n + p(2m \text{ CNOTs} + m\ R_Z + n\ R_X)$ gates.

Running it

With pre-optimised parameters $\gamma \approx \pi/4$, $\beta \approx \pi/8$, the circuit produces the measurement distribution we previewed in Unit 1: the six colourings with cut value 2 each appear with ~16% probability, while the two colourings with cut value 0 (000 and 111) are suppressed to ~3% each. Interference has done its job.

The companion notebook runs this circuit end-to-end — constructing the Hamiltonian, building the QAOA circuit, sweeping the parameter landscape, and comparing with brute-force and random sampling.

→ **See notebook `01-logistics.ipynb` for the runnable version.**

Choosing γ and β: the classical outer loop

The quantum circuit takes two numbers: γ and β; and produces measurement samples. But which values of γ and β make the good colourings most likely?

This is a classical optimisation problem. You run the circuit many times (say, 1,000 shots), compute the average cut value from the measurements, and feed this to

a classical optimiser that suggests better γ and β. Repeat until the average cut value stops improving.

The optimisation landscape; average cut value as a function of (γ, β); is smooth and periodic for small graphs. For the triangle, the optimum is near $\gamma \approx \pi/4$, $\beta \approx \pi/8$. The companion notebook sweeps the full landscape and visualises it as a heatmap.

> **Common Mistake #2:** "QAOA finds the optimal solution by exploration." Not quite. A single run of the QAOA circuit produces a probability distribution over colourings; not a single guaranteed answer. You sample from this distribution. The optimisation loop adjusts γ and β to shift the distribution toward better colourings. The algorithm is probabilistic: you run it many times and take the best sample.

Going deeper: depth $p > 1$

Everything above is depth-1 QAOA. At depth p, you repeat the problem unitary and mixer p times, with independent parameters $\gamma_1, \beta_1, \gamma_2, \beta_2, \ldots, \gamma_p, \beta_p$:

$$|\vec{\gamma}, \vec{\beta}\rangle = \prod_{k=1}^{p} e^{-i\beta_k B} e^{-i\gamma_k C} |+\rangle^n$$

More depth means more expressive circuits; the algorithm can create more complex interference patterns, potentially finding better solutions. But it also means more parameters to optimise ($2p$ in total), deeper circuits (more gates = more noise on real hardware), and a harder classical optimisation problem.

In the limit $p \to \infty$, QAOA can produce any quantum state and find the exact optimum. In practice, $p = 1$ to $p = 20$ are the interesting range. The exact performance of QAOA at each depth; which determines whether it's competitive with classical algorithms; is what the tree tensor network methods described in the Reality Check compute.

What you should take away

1. **Every classical binary decision becomes a qubit.** The encoding is direct: 0/1 → $|0\rangle/|1\rangle$.
2. **The cost function becomes a phase operator.** The CNOT sandwich trick turns pairwise cost terms ($Z_i Z_j$) into phases on the quantum state.
3. **The mixer creates interference.** R_X rotations allow amplitude to flow between colourings, with phases from the problem unitary causing constructive interference on good solutions.
4. **The classical optimiser tunes the circuit.** γ controls how strongly the cost function influences the state; β controls how much mixing occurs.
5. **The circuit is small.** A depth-1 QAOA for n nodes and m edges uses $2m$ CNOTs. This is feasible on current quantum hardware for small-to-moderate graphs.

Unit 2: Cryptography

The Trapdoor Quantum Computers Can Kick Open

The Hook

Every time you type a credit card number into a website, a quiet miracle happens. Your browser and the server agree on a secret key; a number that only the two of them know; even though they've never met before, and even though everything they say to each other travels across a network that anyone can eavesdrop on.

This is **public-key cryptography**, and it protects virtually every financial transaction, every encrypted email, every software update, and every government communication on Earth. The entire system rests on a single mathematical assumption:

Multiplying two large prime numbers is easy. Factoring their product is hard.

Take two 1,000-digit primes p and q. Multiplying them takes microseconds. The product $N = p \times q$ is a 2,000-digit number. Now hand someone N and ask them to find p and q. The best classical algorithms would take longer than the age of the universe.

This asymmetry; easy to combine, hard to separate; is a **trapdoor**. RSA encryption, invented in 1977 and still the backbone of internet security, is built on exactly this trapdoor. If you know p and q, you can decrypt messages quickly. If you only know N, you can't.

For nearly 50 years, this trapdoor has held. Classical computers have gotten faster, and classical factoring algorithms have improved (the General Number Field Sieve, circa 1993, is the current champion). But the best classical algorithms are still sub-exponential; they scale as roughly $e^{n^{1/3}}$ where n is the number of digits. For 2,000-digit numbers, that's far beyond any conceivable classical computer.

In 1994, Peter Shor shattered this assumption.

Shor showed that a quantum computer can factor N in time polynomial in the number of digits; roughly n^3. Not sub-exponential. Not "somewhat faster." Polynomial. For a 2,000-digit number, the difference is between 10^{30} years and a few hours.

Shor's algorithm doesn't just threaten RSA. His 1994 paper actually contains two quantum algorithms: one for **factoring** (which breaks RSA) and one for the **discrete logarithm problem** — given $g^x \bmod p = y$, find x — which breaks Diffie-Hellman (the original public-key protocol) and **elliptic curve cryptography** (ECC), a modern alternative to RSA based on different mathematics. ECC was adopted precisely because its keys are shorter than RSA's for equivalent classical security, but it's in some ways more vulnerable: Roetteler, Naehrig, Svore, and Lauter (2017) estimate that breaking a 256-bit elliptic curve key requires only ~2,330 logical qubits, far fewer than the ~20 million noisy qubits needed for RSA-2048. (We'll define logical vs. physical qubits in the Reality Check.) Both algorithms use the same quantum ingredients: superposition, the QFT, and period-finding.

Let's understand why; and how.

The Bottleneck

Why is factoring hard? It's not enough to say "the search space is big." The search space for sorting a list is big too, but sorting is easy. What makes factoring different is that we don't know how to structure the search.

The trapdoor in detail

RSA works like this:

1. Alice picks two large primes p and q and computes $N = pq$
2. She publishes N (her public key) and keeps p, q secret (her private key)
3. Bob encrypts a message using N; a fast operation
4. Only Alice, who knows p and q, can decrypt it; because decryption requires a key derived from $(p-1)(q-1)$ — the count of integers less than N that share no factor with N

The security assumption: given N, nobody can find p and q efficiently. If they could, they could compute $(p-1)(q-1)$, derive the decryption key, and read everything.

Why classical algorithms struggle

The naive approach; try dividing N by every number up to $\sqrt{N}$; takes $O(\sqrt{N})$ time. For a 2,000-digit N, that's 10^{1000} operations. Hopeless.

The best classical algorithm, the General Number Field Sieve (GNFS), is far cleverer. It exploits algebraic number theory to find special relationships between numbers modulo N. Its running time is roughly:

$$e^{1.923 \cdot n^{1/3} \cdot (\ln n)^{2/3}}$$

where n is the number of bits in N. This is sub-exponential — faster than brute force, but still far slower than polynomial. For RSA-2048 (a 617-digit number), the estimated classical effort is around 2^{112} operations; securely beyond reach for the foreseeable future.

The fundamental issue: factoring has no known polynomial-time classical algorithm. There's no proof that one doesn't exist (factoring is not known to be NP-complete — meaning it hasn't been proven to be among the "hardest of the hard" problems), but decades of effort by the world's best mathematicians and computer scientists have failed to find one.

Factoring reduces to period-finding

Here's the insight that opens the door to quantum computation. Factoring N reduces to finding the **period** of a function.

Pick a random number $a < N$. Define the function:

$$f(x) = a^x \bmod N$$

The notation $\bmod N$ means "take the remainder after dividing by N." So $7^2 \bmod 15 = 49 \bmod 15 = 4$, because 49 divided by 15 leaves remainder 4.

This function is periodic. There exists some integer r (called the **order** of a modulo N) such that $a^r \bmod N = 1$, and therefore $f(x+r) = f(x)$ for all x. The notation $a^r \equiv 1 \pmod{N}$ says the same thing: a^r and 1 have the same remainder when divided by N.

If you can find r, you can (with high probability) factor N. The algebraic trick:

$$a^r - 1 \equiv 0 \pmod{N}$$

$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$$

If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then the **greatest common divisor** $\gcd(a^{r/2} - 1, N)$ — the largest number that divides both arguments — gives a non-trivial factor of N.

So: **if you can find periods efficiently, you can factor efficiently.** Classically, finding the period of $f(x) = a^x \bmod N$ is at least as hard as factoring. Quantumly, it's not.

The Quantum Angle

Shor's algorithm finds the period r of $f(x) = a^x \bmod N$ using three quantum ingredients, each introduced here for the first time:

1. **Superposition**; query f on all inputs simultaneously
2. **The Quantum Fourier Transform**; extract the period from the phase pattern
3. **Interference**; amplify the right answer, suppress the wrong ones

Superposition: querying all inputs at once

In Unit 1, we saw that qubits can be in superposition. Here we use it for something specific: evaluating a function on all inputs in parallel.

Start with two quantum **registers** — groups of qubits that together represent a single number, like how multiple bits form a byte in a classical computer. We need an **input register** of n qubits (to hold x) and an **output register** large enough to hold $f(x)$. Initialise both to $|0\rangle$.

Apply Hadamard gates to every qubit in the input register. The notation $H^{\otimes n}$ is shorthand for "apply H to each of the n qubits":

$$|0\rangle^n \xrightarrow{H^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$

This creates an equal superposition of all 2^n possible inputs. Now apply the unitary that computes f:

$$\frac{1}{\sqrt{2^n}} \sum_x |x\rangle|0\rangle \xrightarrow{U_f} \frac{1}{\sqrt{2^n}} \sum_x |x\rangle|f(x)\rangle$$

With one application of U_f, we've "evaluated" f on all 2^n inputs. The catch: if we measure, we get one random $(x, f(x))$ pair. We haven't gained anything yet. The power comes from what we do before measuring.

The Quantum Fourier Transform: extracting periodicity

The function $f(x) = a^x \bmod N$ is periodic with period r. This means the amplitudes in our quantum state have a hidden structure: for each value y in the output register, the input values x that map to y are evenly spaced with spacing r.

We need to extract r without knowing which y we'll observe. The tool for this is the **Quantum Fourier Transform** (QFT); the quantum analogue of the discrete Fourier transform.

The classical DFT converts a signal from the time domain to the frequency domain. A periodic signal with period r has a frequency peak at $1/r$. The QFT does the same thing to quantum amplitudes.

Applied to the input register, the QFT maps:

$$|x\rangle \xrightarrow{\text{QFT}} \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle$$

The key property: if the input amplitudes are periodic with period r, then after the QFT, the amplitudes are concentrated on values of k that are close to multiples of $2^n/r$. Measuring k and computing $k/2^n$ gives an approximation to j/r for some integer j. A few measurements and some classical post-processing (the continued fractions algorithm) recover r exactly.

Phase kickback: why the QFT "sees" the period

Why does the QFT work here? The deep reason is **phase kickback**; the mechanism by which a function evaluation on a quantum computer imprints information as phases rather than as bit values.

When f is computed in superposition, the output register becomes entangled with the input register. If we were to measure the output register and get some value y_0, the input register would collapse to an equal superposition of all x values satisfying $f(x) = y_0$:

$$\frac{1}{\sqrt{r}} \sum_{j=0}^{r-1} |x_0 + jr\rangle$$

where x_0 is the smallest such x. This state is periodic with period r; and the QFT converts periodicity in position to peaks in frequency.

We don't actually need to measure the output register. The entanglement does the work for us: the QFT on the input register "sees" the period regardless of which y_0 the output register would have yielded. This is the subtlety that makes quantum period-finding work.

Interference: why the wrong answers cancel

The QFT isn't magic; it's interference. Each output state $|k\rangle$ receives contributions from all input states $|x\rangle$, weighted by the complex phase $e^{2\pi ixk/2^n}$. When k is a multiple of $2^n/r$, the phases from the r equally-spaced inputs align; they add constructively. When k is not a multiple of $2^n/r$, the phases point in different directions and cancel; destructive interference.

This is the same physics that creates light and dark bands when light passes through narrow slits: waves that arrive in sync reinforce each other (bright band), waves that arrive out of sync cancel (dark band). The QFT is like a multi-slit experiment with 2^n slits, and the period r determines which bands are bright.

Putting it together

Here is Shor's algorithm as a circuit, drawn at the same box level we used for QAOA in Unit 1:

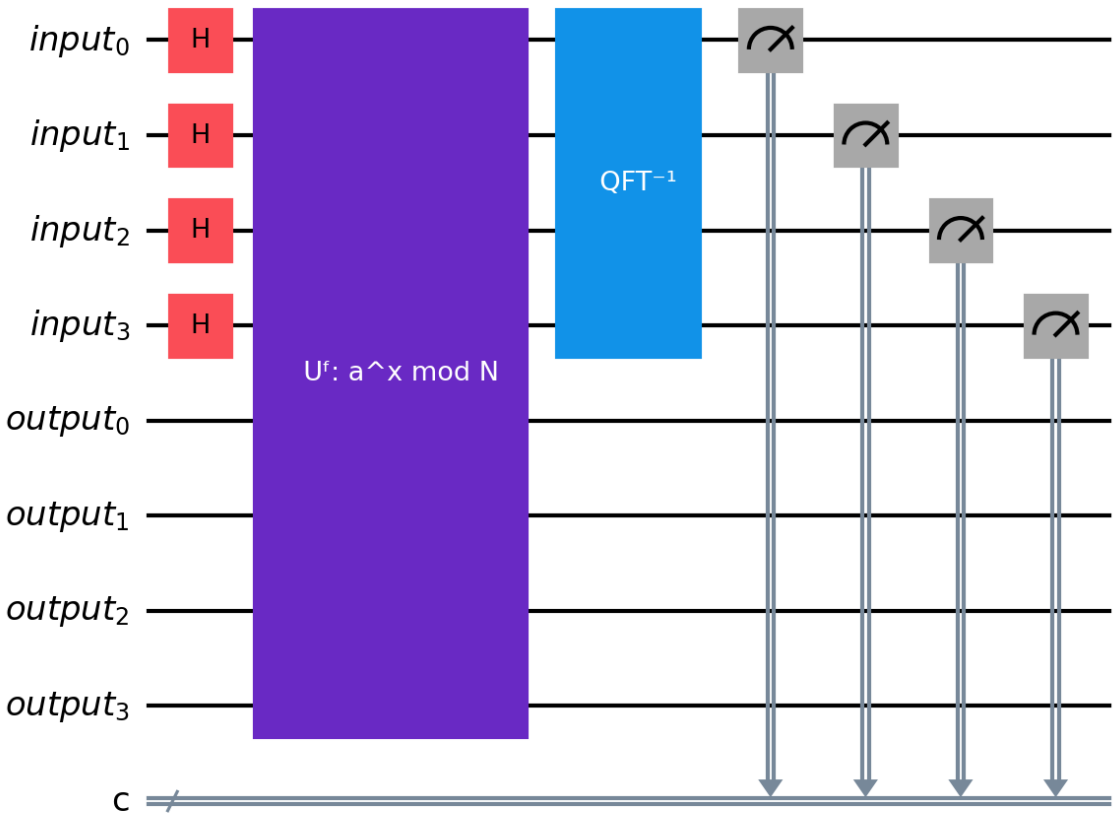

Figure 10. Shor's algorithm circuit: Hadamard on input register, oracle Uᶠ spanning both registers, inverse QFT on input register, measurement

Two registers. The top four wires are the input register; the bottom four are the output register. Read left to right: Hadamard creates superposition, the oracle U_f evaluates $a^x \bmod N$ in superposition, the inverse QFT extracts the period, and measurement reads out a frequency. Only the input register is measured.

Shor's algorithm:

1. Pick a random $a < N$, check $\gcd(a, N) = 1$
2. Prepare a superposition of all inputs: $\frac{1}{\sqrt{2^n}} \sum_x |x\rangle|0\rangle$
3. Compute $f(x) = a^x \bmod N$ in superposition
4. Apply the QFT to the input register
5. Measure the input register → get $k \approx j \cdot 2^n/r$
6. Use the **continued fractions algorithm** — a classical method that finds the simplest fraction j/r close to $k/2^n$ — to extract r
7. Compute $\gcd(a^{r/2} \pm 1, N)$ → factors of N

Steps 1, 6, and 7 are classical. Steps 2–5 are quantum. The quantum part runs in $O(n^2 \log n)$ gates; polynomial in the number of digits.

Worked Example

Let's factor $N = 15$ using Shor's algorithm.

Step 1: Pick a

Choose $a = 7$. Check: $\gcd(7, 15) = 1$. Good.

Step 2–3: Compute $7^x \bmod 15$ in superposition

The values cycle:

x	$7^x \bmod 15$
0	1
1	7
2	4
3	13
4	1
5	7
6	4
7	13

The period is $r = 4$.

Step 4–5: QFT and measurement

Using 4 qubits for the input register ($2^4 = 16$), the QFT concentrates amplitude on multiples of $16/4 = 4$. The possible measurement outcomes are $k \in \{0, 4, 8, 12\}$, each with probability $1/4$.

Measured k	$k/2^n = k/16$	Continued fractions $\to r$
0	0	No information
4	1/4	$r = 4$ ✓
8	1/2	$r = 2$ (try again)
12	3/4	$r = 4$ ✓

Three out of four outcomes give us $r = 4$ (or a multiple).

Step 6–7: Extract factors

With $r = 4$:

$$7^{4/2} = 7^2 = 49$$

$$\gcd(49 - 1, 15) = \gcd(48, 15) = 3$$

$$\gcd(49 + 1, 15) = \gcd(50, 15) = 5$$

And indeed: $15 = 3 \times 5$. ■

This is a toy example; 4 qubits, a number you can factor in your head. But the algorithm scales polynomially: factoring a 2,000-digit number requires roughly 4,000 logical qubits and $O(n^3)$ gates with $n \approx 6,600$ bits. The same algorithm, larger registers, more gates, same polynomial efficiency.

→ The next chapter builds the period-finding circuit from scratch, and shows you the code.

Back to your browser

We started with credit card numbers and the quiet miracle of public-key cryptography. We factored $15 = 3 \times 5$ on a toy quantum computer. What connects the two?

Scale. The same algorithm that finds the period of $7^x \bmod 15$ can find the period of $a^x \bmod N$ for any N. For RSA-2048, N has 617 digits, the input register has ~4,000 qubits, and the circuit has $\sim 10^{10}$ gates. But the structure is identical: superposition, oracle, QFT, measure, continued fractions, gcd. The quantum computer doesn't know whether it's breaking a classroom exercise or a government cipher. It just finds periods.

That's why the world's cryptographic infrastructure is being rebuilt. Not because a quantum computer has broken RSA — none has — but because the algorithm is ready and waiting. When the hardware catches up, it will work.

Reality Check

Shor's algorithm is the most famous quantum algorithm, and its implications are enormous. But let's be precise about what has and hasn't been demonstrated.

What's been factored quantumly. The largest number factored by a genuine implementation of Shor's algorithm on a quantum computer is $21 = 3 \times 7$ (Martín-López et al., 2012, using photonic qubits). A separate experiment factored 15 using NMR (Vandersypen et al., 2001). Various claims of factoring larger numbers use compiled or problem-specific circuits that don't generalise; they exploit known structure of the answer, which defeats the purpose.

In short: Shor's algorithm has never been run, at scale, on a number whose factors were unknown.

Resource estimates for RSA-2048. To make these estimates concrete, two terms matter: a **physical qubit** is one imperfect qubit on real hardware, and a **logical qubit** is an error-corrected qubit built from many physical qubits — hundreds or thousands, depending on the error-correction scheme. Today's machines have physical qubits; Shor's algorithm at scale needs logical ones. A **fault-tolerant** quantum computer is one that uses error correction to run arbitrarily long computations reliably, despite individual gates being imperfect.

Gidney and Ekerå (2021) estimated that factoring a 2,048-bit RSA key would require approximately **20 million noisy physical qubits** (using the surface code, the leading error-correction scheme, with a gate error rate of 10^{-3}) and roughly **8 hours** of computation.

That estimate has dropped dramatically. In February 2026, Webster, Berent, Chandra, and Hockings introduced the **Pinnacle architecture** (arXiv:2602.11457),

which uses quantum LDPC codes — a newer approach that packs more logical qubits per physical qubit — to reduce the cost of breaking RSA-2048 to under **100,000 physical qubits**; a 200× improvement. The largest quantum computers in 2026 have a few thousand physical qubits, so we're still short; but 100K qubits is within plausible reach of hardware roadmaps in the next 5–10 years. The threat timeline just got much shorter.

On the elliptic curve side, Kim, Jang, et al. (ePrint 2026/106) designed new quantum circuits for ECDLP that reduce the total gate count by 52–54% compared to Roetteler et al., bringing the estimated time to break P-224 (comparable security to RSA-2048) down to minutes on a fault-tolerant machine.

The post-quantum migration. The threat is taken seriously. NIST finalised its first set of post-quantum cryptography standards in 2024: CRYSTALS-Kyber (key encapsulation) and CRYSTALS-Dilithium (digital signatures), both based on lattice problems believed to be hard for quantum computers. The U.S. government has mandated that federal systems begin migrating to post-quantum cryptography. Google, Cloudflare, and Apple have already deployed post-quantum key exchange in their products.

The migration timeline is measured in decades, not years. Cryptographic infrastructure is deeply embedded in hardware, software, and protocols. The risk is **harvest now, decrypt later**: an adversary records encrypted traffic today and decrypts it in 15 years when a quantum computer is available. For secrets that must remain confidential for decades (state secrets, medical records, financial data), the threat is already real.

What's certain: Shor's algorithm works. The mathematics is settled. The only question is when quantum hardware will be capable of running it at scale. The cryptography community has decided not to wait for the answer.

Chef's Notes

- **Superposition, interference, and phase kickback.** These three concepts; introduced here for the first time; are the engine of essentially every quantum speedup. Superposition lets you process many inputs at once. Interference lets you amplify the right answers. Phase kickback is the mechanism that converts function evaluations into phases that interference can act on. When you encounter these concepts in later units, you've already met them here.
- **The QFT appears again in QPE** (Unit 7). Quantum Phase Estimation is essentially "Shor's period-finding subroutine applied to a unitary operator instead of modular exponentiation." If you understand why the QFT extracts periodicity, you understand QPE; and QPE is the most important subroutine in fault-tolerant quantum computing.
- **Shor's algorithm via amplitude estimation.** There's a structural connection to Unit 5 (Finance): Shor's period-finding can be viewed as a special case of quantum phase estimation, which is itself related to amplitude estimation. The algorithms look different on the surface but share the same core: extract a number encoded in the phase of a quantum state.
- **Why we don't start with Shor.** Most quantum computing textbooks open with Shor's algorithm because it's dramatic and historically important. We

put it in Unit 2 (not Unit 1) because you need qubits and superposition from Unit 1 to understand it, and because the factoring problem; while economically critical; is less relatable than route optimisation. The UPS driver's $50M is visceral. RSA's factoring trapdoor requires setup to appreciate.

- **Further reading:**
  - Shor (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. arXiv:quant-ph/9508027
  - Gidney and Ekerå (2021). How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. arXiv:1905.09749
  - Webster, Berent, Chandra, Hockings et al. (2026). The Pinnacle Architecture: Reducing the cost of breaking RSA-2048 to 100,000 physical qubits using quantum LDPC codes. arXiv:2602.11457
  - Roetteler, Naehrig, Svore, Lauter (2017). Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms. arXiv:1706.06752
  - Kim, Jang, Wang, Baksi, Song et al. (2026). New Quantum Circuits for ECDLP: Breaking Prime Elliptic Curve Cryptography in Minutes. ePrint 2026/106
  - NIST (2024). Post-Quantum Cryptography Standardization. csrc.nist.gov/projects/pos quantum-cryptography
  - Vandersypen et al. (2001). Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic resonance. Nature 414:883–887 (arXiv:quant-ph/0112176)
  - Nielsen and Chuang (2000). Quantum Computation and Quantum Information. Chapter 5: The quantum Fourier transform and its applications.

Deep-Dive 2: Inside Shor's Algorithm

This deep dive pairs with Unit 2 (Cryptography), which explained why factoring breaks RSA and how period-finding solves factoring. Here we build the period-finding circuit from scratch.

In This Chapter

- **What you'll learn:** How to extract the period of a function using a quantum circuit; the Quantum Fourier Transform, phase kickback, oracles, and why interference makes it all work.
- **What you need:** From Deep-Dive 1 (Building QAOA), you know what qubits are, how Hadamard creates superposition, and how CNOT entangles two qubits. We build everything else here.
- **Runnable version:** The companion notebook `02-cryptography.ipynb` factors 15 on a cloud Quokka.

The problem we need to solve

Unit 2 showed that factoring N reduces to finding the period r of the function $f(x) = a^x \bmod N$. We need a circuit that:

1. Evaluates f on all possible inputs simultaneously (superposition)
2. Extracts the period r from the results (without knowing which specific outputs appeared)

In Deep-Dive 1, we built QAOA from gates. The cost Hamiltonian imprinted phases on the quantum state — different colourings got different phases — and then the mixer caused interference that amplified good solutions. Shor's algorithm has exactly the same shape: a function imprints phases, and interference (via the QFT) extracts the answer. The details are different, but the architecture is the same.

Let's build it up, starting from things you already know.

From CNOT to function evaluation

A function you already know

In Deep-Dive 1, we used CNOT to compute the parity of two qubits as part of the ZZ gate. But CNOT does something simpler and more fundamental than we appreciated at the time: it computes **XOR**.

Here's the full truth table for CNOT, acting on an input qubit and an output qubit:

Input $\lvert x\rangle$	Output before	Output after	What happened
$\lvert 0\rangle$	$\lvert 0\rangle$	$\lvert 0\rangle$	$0 \oplus 0 = 0$
$\lvert 0\rangle$	$\lvert 1\rangle$	$\lvert 1\rangle$	$1 \oplus 0 = 1$
$\lvert 1\rangle$	$\lvert 0\rangle$	$\lvert 1\rangle$	$0 \oplus 1 = 1$

Input $\|x\rangle$	Output before	Output after	What happened
$\|1\rangle$	$\|1\rangle$	$\|0\rangle$	$1 \oplus 1 = 0$

The input qubit is never changed. The output qubit gets XOR'd with the input: $|b\rangle \to |b \oplus x\rangle$. That $\oplus$ is just "add, then take the remainder modulo 2" — the same as flipping the output when the input is 1, which is exactly what CNOT does.

When the output starts at $|0\rangle$, the XOR simply copies: $|0 \oplus x\rangle = |x\rangle$. So CNOT computes $f(x) = x$ (the identity function):

$$\text{CNOT}|x\rangle|0\rangle = |x\rangle|x\rangle = |x\rangle|f(x)\rangle$$

The input qubit is unchanged. The output qubit holds $f(x)$.

This is the pattern for all quantum function evaluation: you have an input register holding $|x\rangle$, an output register initialised to $|0\rangle$, and a circuit U_f that XORs $f(x)$ into the output:

$$U_f|x\rangle|0\rangle = |x\rangle|0 \oplus f(x)\rangle = |x\rangle|f(x)\rangle$$

Why XOR and not just "write $f(x)$"? Because quantum operations must be reversible — you need to be able to undo them. Overwriting a register isn't reversible (the old value is lost). XOR is: apply U_f twice and you get $|b \oplus f(x) \oplus f(x)\rangle = |b\rangle$, back to where you started. A single CNOT is the simplest oracle. Larger oracles — like the one that computes $a^x \bmod N$ — are built from networks of CNOTs and Toffoli gates, but they all follow this same XOR-into-the-output pattern.

For the identity function, U_f is a single CNOT. For $f(x) = a^x \bmod N$, U_f is a much larger circuit (we'll see how it's built later in this chapter). But the interface is always the same: input in, function value XOR'd into the output, input unchanged.

What happens in superposition

Now put the input in superposition. Apply Hadamard to the input qubit before the CNOT:

$$H|0\rangle \otimes |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle$$

Then apply U_f (the CNOT):

$$\frac{1}{\sqrt{2}}(|0\rangle|f(0)\rangle + |1\rangle|f(1)\rangle) = \frac{1}{\sqrt{2}}(|0\rangle|0\rangle + |1\rangle|1\rangle)$$

We've evaluated f on both inputs with one operation. But if we measure, we get one random pair: either $(0, 0)$ or $(1, 1)$. No speedup. The information is there — both answers are in the superposition — but measurement destroys it.

This is the fundamental challenge: superposition lets you evaluate f on all inputs at once, but measurement gives you only one answer. The trick is to extract structural information about f (like its period) without looking at individual values. That trick is called **phase kickback**.

Phase kickback: the trick behind everything

The problem with reading the output

When we evaluate f in superposition, the result sits in the output register as a computational basis state — a "classical" value that we could read out. But reading it gives us one random answer. We need a way to extract information about the pattern of all the answers at once.

In QAOA, we faced a similar problem. The cost Hamiltonian didn't write "this colouring cuts 3 edges" into a register. Instead, it imprinted the cost as a phase on the quantum state — invisible to measurement but available for interference. Can we do the same thing with function evaluation?

Yes. Here's how.

The $|-\rangle$ trick

First, two useful shorthands. Applying a Hadamard to $|0\rangle$ gives $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ — equal superposition with a plus sign. Applying X then H to $|0\rangle$ gives $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ — equal superposition with a minus sign. The only difference between $|+\rangle$ and $|-\rangle$ is one sign. That sign is about to do all the work.

Instead of initialising the output register to $|0\rangle$, initialise it to $|-\rangle$. Now watch what happens when we apply U_f (for a single-bit function f):

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$$

The output register doesn't change — it comes back as $|-\rangle$ every time. But the input register picks up a phase of $(-1)^{f(x)}$: a plus sign when $f(x) = 0$, a minus sign when $f(x) = 1$. The function value has been "kicked back" from the output register into the phase of the input register.

Why does this happen?

Let's trace it. We established that U_f XORs $f(x)$ into the output: $U_f|x\rangle|b\rangle = |x\rangle|b \oplus f(x)\rangle$. Now expand $|-\rangle$ and apply U_f to each term:

$$U_f|x\rangle|-\rangle = U_f|x\rangle \cdot \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

$$= \frac{1}{\sqrt{2}}(|x\rangle|0 \oplus f(x)\rangle - |x\rangle|1 \oplus f(x)\rangle)$$

Here $\oplus$ means XOR (addition modulo 2): $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

If $f(x) = 0$: XOR with 0 changes nothing. We get $\frac{1}{\sqrt{2}}(|x\rangle|0\rangle - |x\rangle|1\rangle) = |x\rangle|-\rangle$. Phase $= +1$.

If $f(x) = 1$: XOR with 1 flips both bits. We get $\frac{1}{\sqrt{2}}(|x\rangle|1\rangle - |x\rangle|0\rangle) = -|x\rangle|-\rangle$. Phase $= -1$.

The output register returns to $|-\rangle$ in both cases — it's a catalyst, not a container. The information about $f(x)$ lives entirely in the phase of the input register. This is **phase kickback**.

> **Why this matters:** Phases are invisible if you measure immediately: $|+1|^2 = |-1|^2 = 1$. But phases interfere. If we apply the right transformation after the phase kickback, different phases will add constructively (amplifying some states) or destructively (suppressing others). This is how quantum algorithms extract information that's hidden in the phases. It's the same principle as QAOA — the cost phase was invisible until the mixer turned phase differences into probability differences.

Phase kickback for general functions

Phase kickback isn't limited to single-bit functions. For the period-finding problem, $f(x) = a^x \bmod N$ produces multi-bit outputs. The mechanism is more general: instead of $(-1)^{f(x)}$, the phase encodes the eigenvalue of the oracle operator. (An eigenvalue is the factor by which an operator scales a particular state: if $U|v\rangle = \lambda|v\rangle$, then λ is the eigenvalue and $|v\rangle$ is the eigenvector.) We'll see this in detail when we build the full period-finding circuit below.

The pattern generalises. Deutsch-Jozsa, Bernstein-Vazirani, Simon's algorithm, Grover's search, and Shor's algorithm all use phase kickback. Master it once, and you've understood the engine of quantum speedup.

Oracles: packaging functions as gates

What an oracle is

We've been calling the function-evaluating circuit U_f. In quantum computing, this is called an **oracle** — a black box that computes a function f reversibly. You don't need to know how it works internally; you only need to know what it computes.

In Deep-Dive 1 (QAOA), we didn't use oracles — the cost function was built directly as a sum of ZZ interactions, gate by gate. In period-finding, the function $f(x) = a^x \bmod N$ is packaged as an oracle U_f. The quantum algorithm doesn't need to know how modular exponentiation is implemented; it just calls U_f and exploits phase kickback.

The Deutsch-Jozsa pattern

To see phase kickback in its simplest complete form, consider this problem: you're given a function $f : \{0,1\} \to \{0,1\}$ (meaning f takes a single bit as input and returns a single bit), and you want to know if $f(0) = f(1)$ (constant) or $f(0) \neq f(1)$ (balanced). Classically: two queries. Quantumly: one.

The circuit:

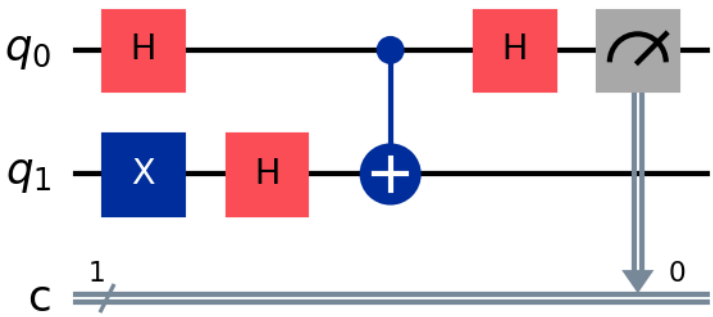

Figure 11. Deutsch-Jozsa circuit

1. Prepare $|0\rangle|-\rangle$ (the second qubit — called an **ancilla**, a helper qubit used as scratch space — is set to $|-\rangle$ via X then H)
2. Apply H to the input qubit → superposition $|+\rangle|-\rangle$
3. Apply the oracle → phase kickback: $\frac{1}{\sqrt{2}}((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle)|-\rangle$
4. Apply H to the input qubit → interference
5. Measure

Step 4 is where interference happens, so let's slow down and watch it. After the oracle, the input qubit is in:

$$\frac{1}{\sqrt{2}}((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle)$$

Now recall what H does to $|0\rangle$ and $|1\rangle$:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Applying H to our state, each term produces a contribution to $|0\rangle$ and to $|1\rangle$:

If f is constant ($f(0) = f(1)$, so both phases are the same): the $|0\rangle$ contributions add up (both point the same way), while the $|1\rangle$ contributions cancel (one $+$, one $-$). Result: $|0\rangle$ with certainty. Constructive interference on $|0\rangle$, destructive on $|1\rangle$.

If f is balanced ($f(0) \neq f(1)$, so the phases are opposite): now the $|0\rangle$ contributions cancel and the $|1\rangle$ contributions reinforce. Result: $|1\rangle$ with certainty.

This is interference in its purest form: two amplitudes arriving at the same state, either reinforcing or cancelling depending on their relative phase. The Hadamard is the "beam splitter" that makes them overlap so the phases can interact. Without it, the phases would sit on $|0\rangle$ and $|1\rangle$ separately, invisible to measurement. The Hadamard forces them to interfere — and the outcome tells you whether the phases were the same or different.

One query. The phase kickback converted the function's output into a phase, and the Hadamard converted the phase difference into a measurable bit. This is the template for everything that follows — and the QFT generalises this from "same or different?" to "what's the period?"

From one bit to n bits

The Deutsch-Jozsa algorithm generalises: for $f : \{0,1\}^n \to \{0,1\}$ (a function that takes an n-bit string and returns a single bit), promised to be constant or balanced, one quantum query suffices (vs. $2^{n-1} + 1$ classically). The circuit is the same: $H^{\otimes n}$, oracle, $H^{\otimes n}$, measure; and the mechanism is the same: phase kickback + Hadamard interference.

Bernstein-Vazirani extends this: the oracle encodes a hidden string s, and the same circuit recovers all n bits of s in one query. Simon's algorithm goes further: for a function with a hidden period s (where $f(x) = f(x \oplus s)$, with $\oplus$ meaning bitwise XOR), $O(n)$ queries suffice quantumly vs. $O(2^{n/2})$ classically. Each of these is a stepping stone toward Shor's algorithm — we won't detail them here, but they all use phase kickback and Fourier-type interference, applied to increasingly structured problems.

The Quantum Fourier Transform

Why we need it

Shor's algorithm needs to extract the period of a function; not a single bit of information (constant vs. balanced) but a number r that could be anywhere from 1 to N.

The Hadamard transform, which we used in Deutsch-Jozsa, is actually a 1-qubit Fourier transform. To extract a period from a multi-qubit state, we need the full **Quantum Fourier Transform**; the generalisation of the Hadamard to n qubits.

What the QFT does

The classical Discrete Fourier Transform converts a sequence of numbers from the "time domain" to the "frequency domain." A periodic signal with period r has energy concentrated at frequency $1/r$.

The QFT does the same thing to quantum amplitudes. The factor $e^{2\pi i\theta}$ is a complex number of magnitude 1 that encodes an angle θ — it's the mathematical language for phases:

$$\mathrm{QFT}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle$$

If the input state is periodic; amplitudes concentrated on values equally spaced by r; then the output state is concentrated on multiples of $2^n/r$. Measuring the output gives a multiple of $2^n/r$, from which we can extract r.

How the QFT is built

The remarkable thing about the QFT is that it factorises. The output state can be written as a tensor product of single-qubit states:

$$\mathrm{QFT}|x\rangle = \bigotimes_{\ell=1}^{n} \frac{1}{\sqrt{2}} \left(|0\rangle + e^{2\pi i x/2^\ell}|1\rangle\right)$$

Each qubit in the output depends on the input through a phase $e^{2\pi i x/2^\ell}$. This factorisation means the QFT can be built from:

- **Hadamard gates** (creating the $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$ superpositions)
- **Controlled phase gates** R_k: a **controlled gate** applies an operation to one qubit only when another qubit is $|1\rangle$ — CNOT is a controlled-X; here R_k is a controlled rotation that adds a phase of $e^{2\pi i/2^k}$

For 3 qubits, the circuit is:

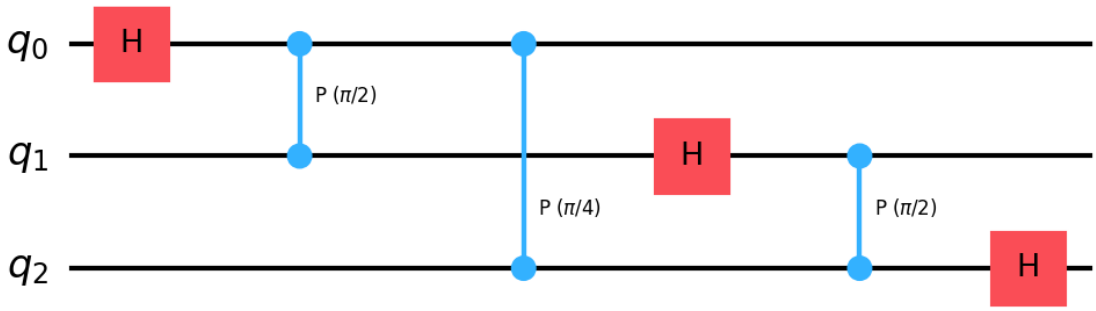

Figure 12. 3-qubit Quantum Fourier Transform circuit

where R_k applies a controlled phase of $e^{2\pi i/2^k}$. The QFT naturally produces its output qubits in reverse order, so a final layer of SWAP gates puts them right:

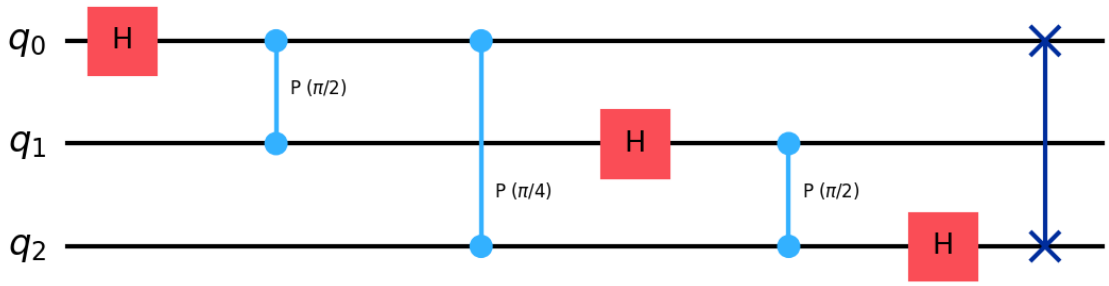

Figure 13. 3-qubit QFT with bit-reversal swap

Gate count: n Hadamards + $n(n-1)/2$ controlled rotations = $O(n^2)$ gates. Compare with the classical Fast Fourier Transform (FFT): $O(n \cdot 2^n)$ operations. The QFT is exponentially faster; but you can't read out the full Fourier transform (measurement collapses it to one value).

> **Common Mistake #1:** "The QFT gives an exponential speedup for Fourier transforms." Not exactly. The classical FFT transforms a vector of 2^n numbers and lets you read all of them. The QFT transforms 2^n amplitudes but only lets you sample one outcome. The speedup comes from the combination of QFT with specific problem structure (like periodicity), not from the QFT alone.

Why the QFT "sees" the period: phase arrows

The QFT works because of interference — the same mechanism we just watched in Deutsch-Jozsa, scaled up. In Deutsch-Jozsa, we had two amplitudes arriving at $|0\rangle$, either reinforcing or cancelling. In Shor's, we have many amplitudes arriving at each output $|k\rangle$, and the question is: do their phases line up?

Each output state $|k\rangle$ receives contributions from all the periodic input states, each carrying a phase $e^{2\pi i x k/2^n}$. Think of each contribution as an arrow on the unit circle, pointing in the direction given by its phase. The total amplitude at $|k\rangle$ is the sum of all these arrows.

For our $N = 15$ example (period $r = 4$, register size $2^n = 16$), the four periodic inputs $x = 0, 4, 8, 12$ each contribute an arrow:

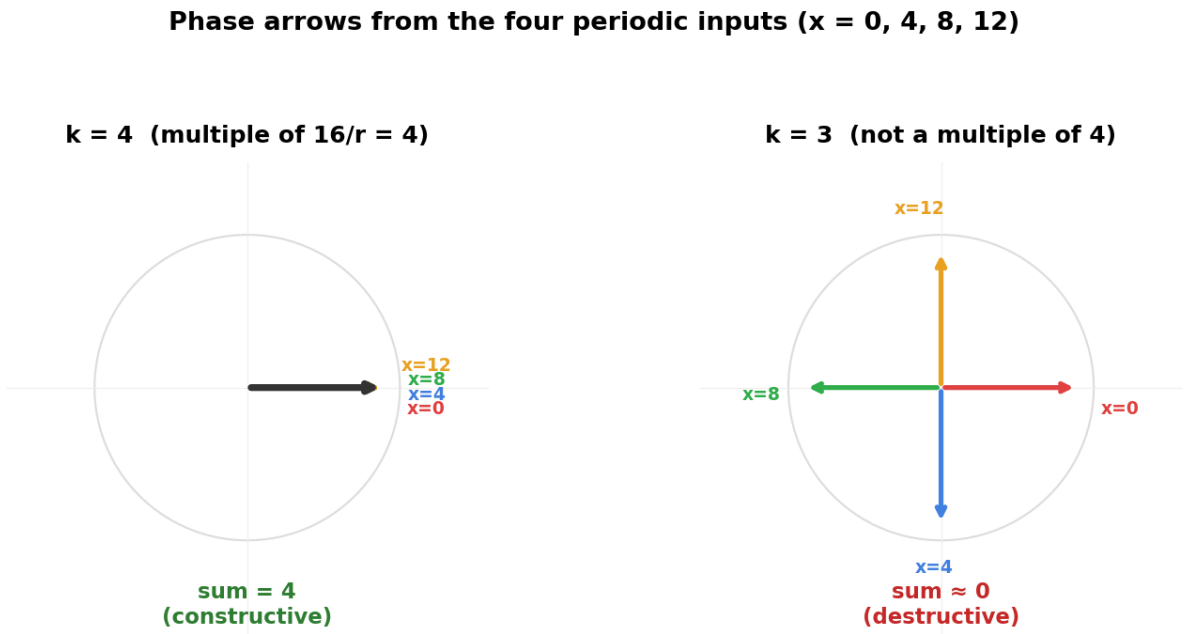

Figure 14. Phase arrows: at k=4 (a multiple of 16/r), all four arrows align constructively; at k=3, they point in four different directions and cancel

Left ($k = 4$): The four arrows point at 0°, 0°, 0°, 0°. Why? Because the phase for input x is $e^{2\pi i \cdot x \cdot 4/16} = e^{2\pi i \cdot x/4}$, and when x is a multiple of 4, the exponent is a whole number of turns around the circle — every arrow lands back at 0°. They all add up: constructive interference. High probability.

Right ($k = 3$): The phase is $e^{2\pi i \cdot x \cdot 3/16}$. For $x = 0$: 0°. For $x = 4$: $3/4$ of a full turn = 270°. For $x = 8$: $3/2$ turns = 180°. For $x = 12$: $9/4$ turns = 90°. The arrows point in four different directions and cancel almost perfectly. Destructive interference. Near-zero probability.

The pattern: when k is a multiple of $2^n/r = 4$, the periodicity of the inputs guarantees all arrows align. For any other k, the arrows spread around the circle and cancel. This is Deutsch-Jozsa's "same phase = reinforce, different phase = cancel" principle, but now applied across r contributions at each of 2^n frequency bins. The QFT is the multi-frequency generalisation of the Hadamard beam splitter.

Assembling Shor's algorithm

Now we have all the pieces. Let's put them together for factoring $N = 15$ with $a = 7$.

Step 1: Superposition

Apply Hadamard to 4 input qubits:

$$|0000\rangle \xrightarrow{H^{\otimes 4}} \frac{1}{4} \sum_{x=0}^{15} |x\rangle$$

We've created a superposition of 16 input values. This is identical to what we did in Deep-Dive 1 for QAOA; the Hadamard creates uniform superposition.

Step 2: Oracle (modular exponentiation)

Compute $f(x) = 7^x \bmod 15$ in an output register. The state becomes:

$$\frac{1}{4}\sum_{x=0}^{15} |x\rangle|7^x \bmod 15\rangle$$

The output register creates entanglement between input values that produce the same output. Since $7^x \bmod 15$ has period 4, the inputs $\{0, 4, 8, 12\}$ all produce output 1, inputs $\{1, 5, 9, 13\}$ all produce output 7, etc.

Step 3: Inverse QFT

Apply the **inverse QFT** to the input register. Since every quantum gate is reversible, we can run the QFT backwards to convert the periodic structure of the amplitudes (spacing = 4) into concentrated frequency peaks (multiples of $16/4 = 4$).

Step 4: Measure

We get one of $\{0, 4, 8, 12\}$ with equal probability.

Step 5: Classical post-processing

From the measured value k and the known register size $2^n = 16$:

$$k/16 \approx j/r$$

The **continued fractions algorithm** extracts r from this rational approximation. For $k = 4$: $4/16 = 1/4$, so $r = 4$. For $k = 12$: $12/16 = 3/4$, and the continued fraction gives denominator 4, so $r = 4$ again.

Step 6: Factor

$a^{r/2} = 7^2 = 49$. Compute $\gcd(49-1, 15) = \gcd(48, 15) = 3$ and $\gcd(49+1, 15) = \gcd(50, 15) = 5$.

$15 = 3 \times 5$. Done.

The cost

For an n-bit number N: $O(n)$ qubits for the registers, $O(n^2)$ gates for the QFT, and $O(n^2 \log n)$ gates for the modular exponentiation (the expensive part). Total: **polynomial in** n; exponentially faster than the best classical factoring algorithm.

The companion notebook runs this circuit end-to-end — factoring 15 on a cloud Quokka, stepping through the superposition, oracle, QFT, and classical post-processing.

→ See notebook `02-cryptography.ipynb` for the runnable version.

> **Common Mistake #2:** "Shor's algorithm finds factors by trying all possible factors simultaneously." No. It finds the period of a specific function, using the QFT to extract that period from a superposition. The period is then used classically (via gcd) to find the factors. The quantum computer never "sees" the factors; it sees a frequency.

The modular exponentiation circuit

The QFT is elegant and compact ($O(n^2)$ gates). The modular exponentiation; computing $a^x \bmod N$ reversibly; is where the circuit gets large.

Repeated squaring

$a^x \bmod N$ is computed using the **binary expansion** of x. Any integer can be written as a sum of powers of 2: $x = x_{n-1}2^{n-1} + \cdots + x_1 \cdot 2 + x_0$, where each x_k is 0 or 1. This lets us decompose:

$$a^x = a^{x_0 \cdot 1} \cdot a^{x_1 \cdot 2} \cdot a^{x_2 \cdot 4} \cdots a^{x_{n-1} \cdot 2^{n-1}} \pmod{N}$$

We pre-compute $a^1, a^2, a^4, a^8, \ldots \pmod{N}$ by squaring at each step (hence "repeated squaring"), then multiply together only the terms where $x_k = 1$.

Each factor is a **controlled multiplication**: if input bit $x_k = 1$, multiply the output register by $a^{2^k} \bmod N$; otherwise, do nothing. This is a controlled operation; controlled by qubit k of the input register.

Modular arithmetic on a quantum computer

Multiplication modulo N is built from:

1. **Quantum adders** (Draper's QFT-based adder, Cuccaro's ripple-carry adder, or Gidney's optimised adder)
2. **Modular reduction** (subtract N if the result exceeds N, using a comparison and controlled subtraction)
3. **Uncomputation**: quantum operations must be reversible, so any scratch work must be run backwards after use. Leftover scratch qubits become entangled with the result — called garbage entanglement — which ruins interference.

Each controlled multiplication costs $O(n^2)$ gates. There are $2n$ such multiplications (one per input qubit). Total: $O(n^3)$ gates for the naive version. Gidney and Ekerå (2021) brought this down to $O(n^2)$ **Toffoli gates** — a Toffoli flips a target qubit only when two control qubits are both $|1\rangle$, making it the quantum AND gate.

For $N = 15$ (our toy example), the modular exponentiation is small enough to compile by hand. For RSA-2048, it's $\sim 10^{10}$ gates; feasible on a fault-tolerant machine, far too deep for today's noisy hardware (sometimes called NISQ — Noisy Intermediate-Scale Quantum).

What you should take away

1. **Phase kickback** converts function values into phases. The output register is a catalyst; it enables the computation but returns to its starting state. The information lives in the phases of the input register.
2. **The QFT** converts periodicity in amplitudes to peaks in frequency. It's built from Hadamards and controlled rotations: $O(n^2)$ gates for n qubits. It's the engine that extracts the period.
3. **Oracles** abstract away function computation. You don't need to know how f is implemented; just that it can be called as a quantum operation. This separation of concerns is what makes quantum algorithms modular.
4. **The cost is dominated by the oracle.** The QFT is cheap ($O(n^2)$). The modular exponentiation is expensive ($O(n^2)$ to $O(n^3)$). For practical factoring, the modular arithmetic is where all the engineering effort goes.
5. **Shor's algorithm is not trying all factors.** It's using interference to extract a number-theoretic property (the period) that happens to reveal the factors. The quantum computer sees frequencies, not factors.

Unit 3: Drug Discovery

$2 Billion and 12 Years Per Drug

The Hook

In 2020, the pharmaceutical industry spent approximately $83 billion on research and development. The average cost to bring a single new drug to market: **$2.6 billion**. The average time: **12 years**. And roughly 90% of drug candidates that enter clinical trials fail.

Why is it so expensive? The cost isn't mostly in the lab work itself. It's in the failures. For every drug that reaches your pharmacy shelf, hundreds of candidates were synthesised, tested, found wanting, and discarded. Many of those failures could have been predicted; if we could accurately simulate how a drug molecule interacts with its biological target.

The bottleneck is **molecular simulation**. To predict whether a candidate drug will bind to a protein target, you need to compute the electronic structure of the drug-protein complex — the arrangement of electrons that determines binding affinity, reaction rates, and molecular stability. This is a quantum mechanical calculation. The molecule doesn't know classical physics. Its electrons obey the **Schrödinger equation** — the fundamental law governing quantum systems, which relates a system's energy to the quantum states its particles can occupy — and the only way to predict the molecule's behaviour accurately is to solve that equation.

For small molecules — up to about 50 electrons — classical computers can do this well enough. **Density functional theory** (DFT) — which approximates the many-electron problem by working with the electron density rather than tracking every electron individually — handles molecules of this size in minutes. For a drug candidate docked to a protein binding site — hundreds or thousands of electrons — DFT is too inaccurate and exact methods are unaffordable.

The exponential wall is not a metaphor. Each electron in a molecule occupies an **orbital** — a quantum state describing where the electron is likely to be found and what energy it has (a **spin-orbital** adds the electron's spin: up or down). The full quantum state of N electrons occupying M orbitals lives in a space of dimension $\binom{M}{N}$ — the number of ways to choose which N orbitals are occupied. This space of all possible quantum states is called a **Hilbert space**. For a modest active site — say, 30 electrons in 60 orbitals — that's $\binom{60}{30} \approx 1.2 \times 10^{17}$ dimensions. Storing a single quantum state requires more memory than exists on Earth. No amount of classical computing power will cross this wall. The problem is quantum, and the solution might be too.

The Bottleneck

Let's understand what makes molecular simulation hard; and why it's a fundamentally different kind of hardness from the combinatorial problems in Unit 1.

The electronic structure problem

Given a molecule — its atoms and their positions — compute its **ground-state energy**: the lowest-energy arrangement of its electrons.

The ground-state energy determines everything chemists care about: whether a reaction is energetically favourable, how tightly a drug binds to a receptor, what shape a protein folds into, how strong a chemical bond is. It is arguably the single most important number in chemistry.

In the language of quantum mechanics, the ground-state energy is the smallest eigenvalue of the molecular Hamiltonian H. You've met Hamiltonians before: in Unit 1, we built a cost Hamiltonian for MaxCut. Here the Hamiltonian encodes real physics — all the forces between electrons and atomic nuclei — and the Schrödinger equation $H|\Psi\rangle = E|\Psi\rangle$ says that the allowed energy states are precisely the eigenvalues of H. The **wavefunction** $|\Psi\rangle$ is the complete quantum description of every electron in the molecule — the multi-electron generalisation of the qubit states $|\psi\rangle$ you already know.

Why electrons aren't qubits

There's a crucial complication. Electrons are **fermions** — particles that obey the Pauli exclusion principle: no two electrons can occupy the same quantum state. This means the wavefunction must be antisymmetric: swapping any two electrons flips the sign of $|\Psi\rangle$. (This is why atoms have shells and chemistry exists — if electrons could pile into the same state, every atom would behave the same.)

This is different from qubits. If you swap two qubits on a quantum computer, nothing special happens — no minus sign appears. A quantum computer built from qubits doesn't natively enforce fermionic antisymmetry. To simulate electrons on qubits, you need a **fermion-to-qubit encoding** — a mathematical map that translates between the language of electrons and the language of qubits, preserving the crucial minus signs.

The classical approximation hierarchy

Classical computational chemistry has developed a remarkable hierarchy of approximation methods:

Method	Accuracy	Scaling	What it does
Hartree-Fock (HF)	~99% of total energy	$O(N^4)$	Each electron moves in the average field of all others (no correlation)
DFT	Good for weak correlation	$O(N^3)$	Works with electron density instead of the full wavefunction
MP2	Perturbative correction	$O(N^5)$	Adds a first-order correction for electron-electron interaction

Method	Accuracy	Scaling	What it does
CCSD(T)	"Gold standard"	$O(N^7)$	Systematically accounts for how pairs (and triples) of electrons interact
Full CI	Exact (within basis)	$O(e^N)$	Tries every possible electron arrangement — brute force

The pattern: more accuracy costs exponentially more computation.

The simplest method, Hartree-Fock, treats each electron as if it moves through the average field created by all the others — a **mean-field** approximation. This ignores the fact that electrons actively dodge each other: when one electron moves left, its neighbours respond instantly. The difference between this real, correlated dance and the mean-field approximation is called **electron correlation**. Hartree-Fock misses it entirely. Each step up the ladder captures more correlation, but at drastically higher cost. Full Configuration Interaction (Full CI) is exact but exponential.

The gap between CCSD(T) (the best practical classical method, scaling as N^7) and Full CI (exact but exponential) is where quantum computing enters. For molecules with **strong electron correlation** — where the mean-field picture breaks down entirely, such as transition metal complexes (molecules with partially filled d-orbitals) and reaction transition states — CCSD(T) fails and Full CI is unaffordable. These are exactly the molecules that matter most for drug design.

The Quantum Angle

A quantum computer can represent the quantum state of N electrons directly, using qubits in place of orbitals. The key algorithm is the **Variational Quantum Eigensolver** (VQE); a hybrid quantum-classical algorithm for finding the ground-state energy of a molecular Hamiltonian.

From electrons to qubits

To put a molecular problem on a quantum computer, we need to translate the language of electrons into the language of qubits. The key idea: each orbital becomes one qubit ($|1\rangle$ = occupied, $|0\rangle$ = empty), and the electron-electron interactions become a sum of Pauli operators — just like the cost Hamiltonian in QAOA, but with coefficients that come from physics instead of a graph.

The translation isn't trivial because of the fermion sign rule. Different **encodings** (Jordan-Wigner, Bravyi-Kitaev, and others) handle the minus signs differently, making different tradeoffs between qubit count and circuit depth. Deep-Dive 3 walks through the Jordan-Wigner encoding in detail; for the full story, see our companion book From Molecules to Qubits.

After encoding, the molecular Hamiltonian becomes a weighted sum of Pauli operators — terms like Z_0Z_1, X_0X_1, and so on — with real-valued coefficients computed from quantum chemistry. This is the input to the quantum algorithm.

VQE: the variational quantum eigensolver

VQE uses the **variational principle**: for any trial state $|\psi(\theta)\rangle$, the expected energy is an upper bound on the true ground-state energy. (The notation $\langle\psi|H|\psi\rangle$ means "the expected value of measuring H in state $|\psi\rangle$" — the quantum analogue of a weighted average.)

$$E_0 \leq \langle\psi(\theta)|H|\psi(\theta)\rangle$$

The algorithm:

1. **Prepare a trial state** on the quantum computer using a parameterised circuit (the **ansatz** — German for "initial guess," meaning the family of trial states the circuit can reach): $|\psi(\theta)\rangle = U(\theta)|0\rangle$
2. **Measure the energy** by decomposing H into a sum of Pauli operators and measuring each term. The total energy is the weighted sum of these measurements.
3. **Optimise classically**: feed the measured energy to a classical optimiser, which suggests new parameters θ.
4. **Repeat** until the energy converges.

This is the same variational loop as QAOA (Unit 1); quantum circuit produces an estimate, classical optimiser updates parameters, repeat. The difference is the cost function (molecular energy instead of cut count) and the ansatz (chemistry-inspired circuits instead of alternating problem/mixer layers).

The ansatz: encoding chemical intuition

The quality of VQE depends heavily on the ansatz; the family of quantum states the parameterised circuit can reach. Two common choices:

UCCSD (Unitary Coupled Cluster): inspired by the classical CCSD method. It applies single and double excitations; moving one or two electrons between orbitals; with tunable coefficients. Chemically motivated, but the circuits can be deep.

Hardware-efficient ansätze: generic circuits (layers of rotations and entangling gates) that are short enough to run on noisy hardware. Less chemically motivated, but feasible. The risk: they may not contain the true ground state, or they may suffer from **barren plateaus** — regions of parameter space where the energy landscape is exponentially flat, leaving the optimiser with no useful gradient to follow.

Measurement overhead

There's a catch. The molecular Hamiltonian, after encoding, becomes a sum of many Pauli terms: $H = \sum_i c_i P_i$. For H_2 with Jordan-Wigner, there are about 15 terms. For a drug-sized molecule, there could be millions. Each term requires separate measurements, and the statistical error shrinks as $1/\sqrt{N_{\text{shots}}}$. This **measurement overhead** is one of the main practical challenges of VQE.

Worked Example

Let's compute the ground-state energy of **H_2**; the hydrogen molecule; at various bond lengths, building the potential energy surface.

H_2 has 2 electrons and (in a minimal basis) 4 spin-orbitals → 4 qubits after Jordan-Wigner encoding. This is small enough to run on any quantum computer (or simulator), but large enough to show the real VQE pipeline.

The pipeline

1. **Compute molecular integrals** (h_{ij}, h_{ijkl}) using a classical chemistry package
2. **Build the qubit Hamiltonian** via Jordan-Wigner encoding
3. **Choose an ansatz** (UCCSD or hardware-efficient)
4. **Run VQE**; optimise parameters to minimise energy
5. **Sweep the bond length** from 0.2 Å to 2.5 Å and plot the potential energy surface

The equilibrium bond length (the minimum of the potential energy curve) is where H_2 is stable. The dissociation limit (large bond length) is where it breaks into two hydrogen atoms. Getting this curve right; especially the shape near the minimum; is the test of a good quantum chemistry method.

VQE with a 4-qubit UCCSD ansatz reproduces the exact Full CI result for H_2 to **chemical accuracy** (1.6 milliHartree ≈ 1 kcal/mol — the threshold at which energy differences become chemically meaningful; the Hartree is the atomic unit of energy, roughly 27.2 eV). At the equilibrium bond length (~0.74 Å, where 1 Å = 10^{-10} m), the computed energy matches the exact value to several decimal places. As we stretch the bond toward dissociation, the correlation between the two electrons grows stronger, and this is precisely where VQE outperforms Hartree-Fock.

→ The next chapter builds the full VQE pipeline — from molecular integrals to circuit to energy — and shows you the code.

Back to the pharmacy

We computed the energy of H_2 — the simplest molecule there is. How does this help find a $2.6 billion drug?

The pipeline is identical for any molecule. Only two things change: the number of qubits (one per orbital) and the number of Pauli terms in the Hamiltonian (which scales as $O(M^4)$ for M orbitals). H_2 needs 4 qubits and ~15 terms. Caffeine would need ~100 qubits. A drug-sized active site might require several hundred. The algorithm doesn't change — you still prepare a trial state, measure Pauli terms, optimise classically, repeat.

The practical strategy, which we'll develop fully in Unit 8, is to use a classical computer for the "easy" electrons and a quantum computer only for the strongly correlated ones in the active site. This hybrid approach is how quantum computing will likely enter the pharmaceutical pipeline — not replacing classical chemistry, but solving the hardest sub-problems that classical methods can't.

Reality Check

What's been demonstrated. VQE has been run on real quantum hardware for small molecules: H_2 (2 qubits, various groups since 2014), LiH (4 qubits), BeH_2 (6 qubits), and H_2O (up to 12 qubits with **tapering** — a technique that exploits molecular symmetries to remove qubits from the Hamiltonian). Google's 2020 Hartree-Fock experiment on 12 qubits was a milestone, though it used a non-variational approach.

The gap to drug discovery. A drug-sized active site might require 50–100 qubits (after encoding and tapering) and circuits with thousands or millions of gates. Current NISQ devices can handle at most ~20 qubits with meaningful accuracy. **Error mitigation** techniques (like zero-noise extrapolation, which runs circuits at artificially increased noise levels and extrapolates back to zero) help but don't close the gap for industrially relevant molecules.

The measurement problem. For a molecule with M orbitals, the number of Pauli terms in the Hamiltonian scales as $O(M^4)$. Each term needs many measurement shots for acceptable statistics. For 100 orbitals, you might need 10^{10} measurements. Techniques like **classical shadow tomography** (which extracts many properties from relatively few randomised measurements) and grouping of commuting terms reduce this, but it remains a bottleneck.

What would change the picture. Fault-tolerant quantum computers would enable **quantum phase estimation** (Unit 7) instead of VQE; giving exact energies rather than variational upper bounds, without the measurement overhead. But QPE requires much deeper circuits and is a post-NISQ algorithm.

The hybrid approach works. The real near-term strategy is **active-space methods**: use a classical computer for the "easy" electrons (core, weakly correlated) and a quantum computer for the "hard" ones (strongly correlated, in the active space). This is the approach we'll develop fully in Unit 8 (Climate & Energy).

Chef's Notes

- **This chapter introduces the molecular Hamiltonian pattern.** In Unit 1, we encoded an optimisation objective as a cost Hamiltonian. Here we encode the physical Hamiltonian of a molecule. The mathematical structure is the same; a sum of Pauli operators; but the origin is different: physics rather than combinatorics.
- **VQE is QAOA's cousin.** Same variational loop, different cost function, different ansatz. If you understood the QAOA loop in Unit 1, VQE is a natural extension.
- **Fermion-to-qubit encodings are a deep topic.** We've given the 30-second version (Jordan-Wigner, Bravyi-Kitaev). For the full story; including parity encoding, ternary tree encoding, tapering, and the algebraic structure behind all of them; see our companion book: From Molecules to Qubits.
- **H_2 is the "Hello World" of quantum chemistry.** It's the simplest molecule with a non-trivial electronic structure — small enough to verify classically, rich enough to show the full VQE pipeline, and pedagogically indispensable. Our encodings book uses H_2 throughout for the same reason.

- **The path from here to real drug discovery** runs through Unit 8 (Climate & Energy), where we'll use quantum embedding methods to handle realistic molecular sizes; computing the active-site electrons quantumly and the environment classically.
- **Further reading:**
  - Peruzzo et al. (2014). A variational eigenvalue solver on a photonic quantum processor. Nature Communications 5:4213 (arXiv:1304.3061)
  - McArdle, Endo, Aspuru-Guzik, Benjamin, Yuan (2020). Quantum computational chemistry. Reviews of Modern Physics 92:015003 (arXiv:1808.10402)
  - Google AI Quantum (2020). Hartree-Fock on a superconducting qubit quantum computer. Science 369:1084–1089 (arXiv:2004.04174)
  - Azariah et al. From Molecules to Qubits. github.com/johnazariah/encodings-book
  - Szabo and Ostlund (1996). Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory. The classical reference for computational chemistry foundations.

Deep-Dive 3: The VQE Pipeline

This deep dive pairs with Unit 3 (Drug Discovery), which explained why molecular simulation is hard and how VQE addresses it. Here we build the VQE pipeline from molecule to energy, step by step.

In This Chapter

- **What you'll learn:** How electrons become qubits; second quantisation, fermion-to-qubit encodings, Pauli-basis measurement, and the variational optimisation loop that finds the ground-state energy.
- **What you need:** From Deep-Dive 1, you know qubits, superposition, CNOT, and the variational optimisation loop (QAOA). From Deep-Dive 2, you know phase kickback and the idea that quantum operations encode information in phases. Here we add new physics: fermions.
- **Runnable version:** The companion notebook `03-drug-discovery.ipynb` runs H_2 VQE on a cloud Quokka.

Why electrons aren't qubits

In Chapters 2 and 4, qubits were abstract binary variables; node colours in MaxCut, input bits in Shor's algorithm. We could assign any meaning we wanted to $|0\rangle$ and $|1\rangle$.

Electrons are different. They obey rules that qubits don't, and ignoring these rules gives wrong answers. The central rule is:

Electrons are fermions. The wavefunction of a multi-electron system must be antisymmetric — swapping any two electrons flips the sign:

$$\Psi(\ldots, \mathbf{r}_i, \ldots, \mathbf{r}_j, \ldots) = -\Psi(\ldots, \mathbf{r}_j, \ldots, \mathbf{r}_i, \ldots)$$

This minus sign is the Pauli exclusion principle: two electrons can't be in the same state (if they were, swapping them would change nothing, but the minus sign says the wavefunction must flip; the only function equal to its own negative is zero).

Qubits don't have this rule. If you swap two qubits, no minus sign appears — the state is unchanged. If we naively map electrons to qubits — electron in orbital i → qubit i is $|1\rangle$, not in orbital i → qubit i is $|0\rangle$ — we get the right occupations but the wrong phases. The standard encoding produces correct energies for H_2, but for larger molecules the missing signs lead to completely wrong results.

> **Common Mistake #1:** "Qubits are quantum, electrons are quantum, so mapping is trivial." No. Swapping two qubits produces no sign change. Swapping two electrons produces a minus sign. The encoding must bridge this gap. This is the entire subject of the companion book From Molecules to Qubits.

Second quantisation: the language of electrons

Occupation numbers

Instead of tracking where each electron is (first quantisation), we track which orbitals are occupied (second quantisation). For a molecule with M spin-orbitals, the state is described by an occupation vector:

$$|n_0, n_1, \dots, n_{M-1}\rangle$$

where $n_i = 0$ (orbital empty) or $n_i = 1$ (orbital occupied). For H_2 in a minimal basis: 4 spin-orbitals, 2 electrons. A possible state: $|1, 1, 0, 0\rangle$ (the two lowest orbitals occupied).

This looks like qubits! And it is: each orbital maps to one qubit, with $|1\rangle$ meaning "occupied." But the antisymmetry rule means the ordering of orbitals matters. $|1, 1, 0, 0\rangle$ and $|0, 0, 1, 1\rangle$ are not related by a simple swap; the sign depends on how many occupied orbitals you "pass through" to move an electron.

Creation and annihilation operators

The tools that respect antisymmetry are **creation** ($a_i^\dagger$) and **annihilation** (a_i) operators. The symbol † ("dagger") denotes the Hermitian conjugate — roughly, the quantum version of transposing and complex-conjugating a matrix:

- $a_i^\dagger$ creates an electron in orbital i (flips qubit i from $|0\rangle$ to $|1\rangle$, with sign adjustments)
- a_i removes an electron from orbital i (flips $|1\rangle$ to $|0\rangle$, with sign adjustments)

The key property — the thing that encodes the minus sign — is the **anticommutation relation**:

$$\{a_i, a_j^\dagger\} \equiv a_i a_j^\dagger + a_j^\dagger a_i = \delta_{ij}$$

The curly braces $\{A, B\} \equiv AB + BA$ denote the anticommutator — unlike the commutator $[A, B] = AB - BA$ you may know from Pauli algebra. And δ_{ij} is the Kronecker delta: 1 when $i = j$, 0 otherwise.

What does this say? Creating electron j then destroying electron i is not the same as destroying i then creating j — they differ by a sign (unless $i = j$). This is the mathematical expression of the Pauli exclusion principle.

The molecular Hamiltonian

In second quantisation, the molecular Hamiltonian is:

$$H = \sum_{ij} h_{ij}\, a_i^\dagger a_j + \frac{1}{2} \sum_{ijkl} h_{ijkl}\, a_i^\dagger a_j^\dagger a_k a_l$$

The first sum is the **one-electron terms** (kinetic energy + electron-nucleus attraction). The second sum is the **two-electron terms** (electron-electron repulsion). The coefficients h_{ij} and h_{ijkl} are called **molecular integrals** — they come from classical quantum chemistry computations and depend on the molecule's geometry and its **basis set** (the finite set of mathematical functions used to approximate orbitals, like choosing how many Fourier terms to keep in a series).

For H_2 in STO-3G (a minimal basis set: 3 Gaussian functions per orbital): there are about 15 unique molecular integrals. For a drug-sized molecule: millions. But the Hamiltonian always has this same structure — sums of products of creation and annihilation operators, weighted by integrals.

Fermion-to-qubit encodings

The problem

We have a Hamiltonian written in terms of $a_i^\dagger$ and a_i (which obey anticommutation rules). We need to express it in terms of Pauli operators X, Y, Z, I (which obey commutation rules). The encoding translates between these two algebras.

Jordan-Wigner: the simplest encoding

The **Jordan-Wigner** encoding (1928; older than quantum computing!) maps each orbital to one qubit. The occupation number is stored directly: orbital i occupied → qubit i is $|1\rangle$.

The creation operator becomes:

$$a_i^\dagger \to \frac{1}{2}(X_i - iY_i) \otimes Z_{i-1} \otimes Z_{i-2} \otimes \cdots \otimes Z_0$$

Here $\otimes$ is the **tensor product** — it combines operators on separate qubits into one multi-qubit operator. (The $Z_0 Z_1$ you saw in QAOA is shorthand for $Z_0 \otimes Z_1$.) The factor $\frac{1}{2}(X_i - iY_i)$ is called the **raising operator** — it flips $|0\rangle$ to $|1\rangle$ (creating an electron) while leaving $|1\rangle$ unchanged.

The string of Z operators is the price we pay for antisymmetry. Each Z_k checks whether orbital k is occupied; if it is, it contributes a minus sign. This enforces the rule that creating an electron "past" occupied orbitals picks up the right number of minus signs.

Example: $a_2^\dagger$ in a 4-qubit system is $\frac{1}{2}(X_2 - iY_2) \otimes Z_1 \otimes Z_0$. If orbitals 0 and 1 are both occupied, the two Z operators each contribute -1, and the total sign is $(-1)^2 = +1$. If only orbital 0 is occupied, the sign is $(-1)^1 = -1$.

What the Hamiltonian looks like after encoding

Each fermionic term $a_i^\dagger a_j$ becomes a sum of Pauli strings. For H_2 with Jordan-Wigner, the full Hamiltonian decomposes into about 15 terms:

$$H = g_0 I + g_1 Z_0 + g_2 Z_1 + g_3 Z_0 Z_1 + g_4 X_0 X_1 + g_5 Y_0 Y_1 + \cdots$$

Each g_k is a real number computed from the molecular integrals. The Hamiltonian is now a weighted sum of Pauli operators; exactly the kind of thing we know how to measure on a quantum computer.

The locality problem

The Z string in Jordan-Wigner means a single fermionic operator can touch all qubits. For orbital i, the string has length i. For the last orbital in a 100-qubit system, the string is 100 qubits long.

Bravyi-Kitaev encoding improves this: the longest string is $O(\log n)$ instead of $O(n)$. The tradeoff: the mapping between occupation numbers and qubit states is more complex (it uses a binary tree structure). Other encodings (parity, ternary tree, compact) make different tradeoffs.

For the full story; including the algebraic structure behind all encodings, tapering to reduce qubit count, and benchmarks; see From Molecules to Qubits.

Measuring the energy

The variational principle

VQE uses the **variational principle**: for any quantum state $|\psi\rangle$, the expected energy is an upper bound on the true ground-state energy:

$$E_0 \leq \langle\psi|H|\psi\rangle$$

Equality holds only when $|\psi\rangle$ is the ground state. So: prepare trial states, measure their energy, and minimise. The minimum you find is your best estimate of E_0.

Decomposing the energy measurement

The Hamiltonian is $H = \sum_k g_k P_k$ where each P_k is a Pauli string (like $Z_0 Z_1$ or $X_0 X_1$). The expected energy is:

$$\langle H\rangle = \sum_k g_k \langle P_k\rangle$$

We measure each $\langle P_k\rangle$ separately. The total energy is the weighted sum.

Measuring Pauli strings

How do you measure $\langle Z_0 Z_1\rangle$? Run the circuit, measure both qubits in the computational basis, compute $z_0 \cdot z_1$ where $z_i = +1$ if you measured $|0\rangle$ and $z_i = -1$ if you measured $|1\rangle$. Average over many shots.

How do you measure $\langle X_0 X_1 \rangle$? You can't measure X directly in the computational basis. But applying a Hadamard before measurement rotates X into Z:

$$HXH = Z$$

So: apply H to both qubits, then measure in the computational basis. The result gives $\langle X_0 X_1 \rangle$.

How do you measure $\langle Y_0 Y_1 \rangle$? Apply $S^\dagger H$ to each qubit, then measure. The S gate adds a phase of i to $|1\rangle$; its inverse $S^\dagger$ subtracts it. Together with H, this rotates the Y-measurement basis into the computational basis:

$$S^\dagger H \cdot Y \cdot HS = Z$$

Each distinct measurement basis requires a separate circuit execution. This is the **measurement overhead**; for T Pauli terms in the Hamiltonian, you need at least T different circuit runs (reducible by grouping commuting terms).

> **Common Mistake #2:** "VQE measures the energy with one circuit." No. The Hamiltonian is a sum of terms. Each term (or group of commuting terms) requires its own measurement circuit. For molecules with thousands of Pauli terms, this is a major cost; potentially requiring billions of total measurement shots for acceptable statistical precision.

The ansatz: parameterised chemistry

What an ansatz is

The **ansatz** is the parameterised quantum circuit that prepares trial wavefunctions. In QAOA (Deep-Dive 1), the ansatz was the alternating problem/mixer layers with parameters γ, β. In VQE, the ansatz is motivated by chemistry.

Starting point: the Hartree-Fock state

The simplest starting point is the **Hartree-Fock state** — the best approximation that treats each electron as independent. For H_2: two electrons in the two lowest spin-orbitals:

$$|\text{HF}\rangle = |1100\rangle$$

In circuit terms: `x q[0]; x q[1];` (flip qubits 0 and 1 from $|0\rangle$ to $|1\rangle$).

Adding correlation: excitations

The Hartree-Fock state ignores electron correlation; it treats each electron as if it moves in the average field of all others. To capture correlation, we apply **excitation operators**: controlled rotations that mix in states where electrons have been moved to different orbitals.

A **single excitation** $a_i^\dagger a_j$ moves one electron from orbital j to orbital i. A **double excitation** $a_i^\dagger a_j^\dagger a_k a_l$ moves two electrons. The UCCSD (Unitary Coupled Cluster with Singles and Doubles) ansatz applies all possible singles and doubles with tunable amplitudes:

$$|\psi(\theta)\rangle = e^{\sum_{ij} \theta_{ij}^{(1)}(a_i^\dagger a_j - \text{h.c.})} \cdot e^{\sum_{ijkl} \theta_{ijkl}^{(2)}(a_i^\dagger a_j^\dagger a_k a_l - \text{h.c.})} |\text{HF}\rangle$$

Here "h.c." stands for **Hermitian conjugate** — the dagger of the preceding term. Writing $A - \text{h.c.}$ means $A - A^\dagger$, which makes the exponent anti-Hermitian and ensures $e^{A-A^\dagger}$ is unitary (a valid quantum gate). The exponential of an operator is defined by its Taylor series: $e^A = I + A + A^2/2 + \cdots$

After encoding, each excitation becomes a sequence of CNOTs and parameterised rotations.

For H_2: one parameter

H_2 has only one relevant double excitation — moving both electrons from the **bonding orbital** (lower energy, electron density concentrated between the two nuclei) to the **antibonding orbital** (higher energy, a node between the nuclei). So the entire ansatz is:

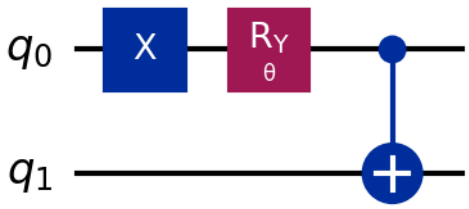

Figure 15. H_2 VQE ansatz circuit: X gate for Hartree–Fock state, Ry(θ) rotation, and CNOT

One parameter θ, one CNOT. The simplest possible VQE circuit; but it captures the essential physics: at the optimal θ, the energy matches the exact result to chemical accuracy.

The classical optimiser

The same variational loop from Deep-Dive 1 (QAOA):

1. Choose parameters θ
2. Prepare $|\psi(\theta)\rangle$ on the quantum computer
3. Measure each Pauli term → compute $\langle H \rangle$
4. Feed $\langle H \rangle$ to a classical optimiser:
   - **COBYLA** (gradient-free — doesn't need derivatives)
   - **L-BFGS** (gradient-based — faster when gradients are available)
   - **SPSA** (stochastic gradient estimate — popular on noisy hardware because it needs only two circuit evaluations per step)
5. Optimiser suggests new θ
6. Repeat until convergence

The variational principle guarantees $\langle H \rangle \geq E_0$; every measurement is an upper bound. The optimiser's job is to tighten the bound.

For H_2 (one parameter), the landscape is a smooth function of θ — a simple 1D minimisation. For larger molecules with hundreds of parameters, the landscape is much rougher, and gradient-based optimisers need the **parameter-shift rule** to estimate gradients from circuit measurements: evaluate the circuit at two shifted parameter values ($\theta + \pi/2$ and $\theta - \pi/2$) and take the difference.

Putting it all together

The complete VQE pipeline:

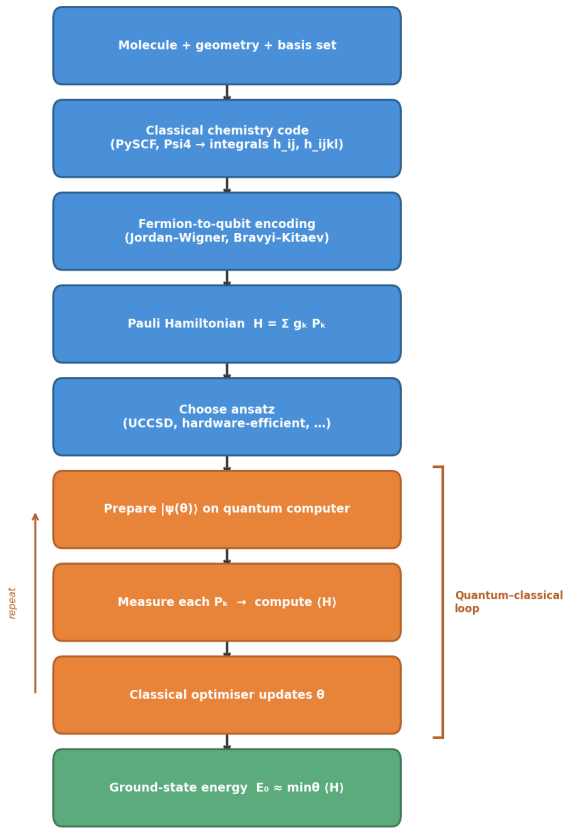

Figure 16. The VQE pipeline: from molecule to ground-state energy via a quantum–classical optimisation loop

Every step is well-defined. The pipeline works for any molecule; only the integrals and the circuit size change. H_2 needs 2–4 qubits. Caffeine needs ~100. A drug-protein complex needs thousands; beyond current hardware but within the scope of fault-tolerant machines.

The companion notebook runs the full H_2 pipeline end-to-end — computing molecular integrals, building the qubit Hamiltonian, constructing the VQE ansatz, and sweeping the bond length to produce the potential energy surface.

→ See notebook `03-drug-discovery.ipynb` for the runnable version.

What you should take away

1. **Electrons are fermions, not qubits.** The antisymmetry of the fermionic wavefunction requires an encoding (Jordan-Wigner, Bravyi-Kitaev, etc.) that maps fermionic operators to Pauli operators with the correct signs.
2. **The Hamiltonian becomes a sum of Pauli strings.** After encoding, the molecular energy is just a weighted sum of measurable quantities; each Pauli string can be estimated from repeated measurements.

3. **Measurement has a cost.** Each distinct Pauli basis requires a separate circuit execution. For large molecules, the measurement overhead can dominate the total computation time.
4. **The ansatz encodes chemical intuition.** Good ansätze (like UCCSD) start from the Hartree-Fock state and apply physically motivated excitations. Bad ansätze miss the ground state or suffer from **barren plateaus** — regions where the energy landscape is exponentially flat and the optimiser has no useful gradient to follow.
5. **VQE is the same variational loop as QAOA.** If you understood Deep-Dive 1, you understand VQE's architecture. The difference is the cost function (molecular energy, not graph cuts) and the ansatz (chemistry-motivated, not graph-motivated).
6. **For the full encoding story,** see From Molecules to Qubits.

Unit 4: Machine Learning

10^8 Features and Counting

The Hook

In 2006, Netflix offered a $1 million prize to anyone who could improve their movie recommendation algorithm by 10%. The winning team needed three years and a blend of over 100 different algorithms to get there. The core challenge wasn't the prize; it was the **dimensionality** of the problem.

Netflix had roughly 480,000 users and 18,000 movies. Each user-movie pair is a potential data point. The "feature space" that recommendation algorithms operate in has dimensions proportional to the number of users times the number of features describing each user's preferences. Modern recommendation engines at companies like Spotify, TikTok, and Amazon operate in feature spaces with 10^8 or more dimensions.

Machine learning at scale is fundamentally a problem of high-dimensional geometry. A classifier draws a boundary in feature space separating "cat" from "dog," "spam" from "not spam," "buy" from "don't buy." The more dimensions, the more expressive the boundary can be; but the harder it is to find.

Kernel methods are the mathematical engine behind this. A kernel function $K(x, x')$ computes the similarity between two data points — effectively computing an inner product in a high-dimensional (possibly infinite-dimensional) feature space without ever explicitly constructing that space. This is the famous **kernel trick**, and it's what makes **Support Vector Machines** (SVMs) powerful: an SVM finds the boundary that maximises the margin (the gap between the boundary and the nearest data points of each class), and it needs only kernel values — pairwise similarities — not the raw feature vectors.

But some useful feature spaces are so large that even the kernel trick breaks down. The computation of certain kernels scales poorly with the number of features or requires evaluating functions that are classically intractable.

A quantum computer operates natively in an exponentially large space; the Hilbert space of n qubits has 2^n dimensions. What if that's the feature space?

The Bottleneck

The kernel trick is effective: instead of mapping data into a high-dimensional space $\phi : x \mapsto \phi(x)$ and computing inner products $\langle \phi(x), \phi(x') \rangle$ explicitly, you compute $K(x, x') = \langle \phi(x), \phi(x') \rangle$ directly. The classifier never sees the high-dimensional representation; it only needs the kernel matrix $K_{ij} = K(x_i, x_j)$ over the training set.

The kernel trick works well when K is cheap to compute. But consider:

- A feature map into a space of dimension 2^n (exponential in the number of input features)

- A kernel that requires sampling from a distribution that's hard to sample classically
- A learning task where the relevant structure lives in a space that has no efficient classical description

In these cases, the kernel trick doesn't help; you can't compute $K(x, x')$ efficiently, even though you never explicitly construct $\phi(x)$.

The question becomes: **are there feature maps whose kernels are hard to compute classically but easy to compute quantumly?** And if so, do they correspond to learning tasks anyone actually cares about?

The Quantum Angle

The Hilbert space as a feature space

The key geometric insight. A quantum computer with n qubits operates in a Hilbert space of dimension 2^n. Preparing a quantum state $|\phi(x)\rangle$ from classical data x is a **quantum feature map**; it maps data into an exponentially large space, natively.

The inner product between two quantum states $|\phi(x)\rangle$ and $|\phi(x')\rangle$ is:

$$K(x, x') = |\langle\phi(x')|\phi(x)\rangle|^2$$

This is just the **Born rule** — the principle we've relied on since Unit 1: the probability of a measurement outcome equals the squared magnitude of its amplitude. Here, it gives us the probability of measuring $|\phi(x)\rangle$ and finding it in state $|\phi(x')\rangle$. A quantum computer can estimate this probability by preparing $|\phi(x)\rangle$, applying $U^\dagger_{\phi(x')}$, and measuring. The result is a kernel value.

If the feature map $|\phi(x)\rangle$ has no efficient classical simulation; if there's no way to compute $\langle\phi(x')|\phi(x)\rangle$ efficiently on a classical computer; then this quantum kernel is something genuinely new. The quantum computer is computing a similarity measure that a classical computer cannot.

Quantum kernel estimation

The algorithm is straightforward:

1. **Encode** classical data point x into a quantum state $|\phi(x)\rangle$ using a parameterised circuit $U_\phi(x)|0\rangle^n$
2. **Compute the kernel** $K(x_i, x_j) = |\langle 0^n|U_\phi(x_j)^\dagger U_\phi(x_i)|0^n\rangle|^2$ by preparing $U_\phi(x_i)|0\rangle$, applying $U_\phi(x_j)^\dagger$, and measuring the probability of getting $|0\rangle^n$
3. **Build the kernel matrix** K_{ij} over the training set (the labelled examples the algorithm learns from)
4. **Train a classical SVM** using this kernel matrix; the classical optimisation is the same as for any kernel SVM

The quantum computer is only used for step 2 — computing kernel values that are classically intractable. Everything else is classical. This encode → measure → classically-optimise pattern is the same variational architecture we saw in QAOA

(Unit 1) and VQE (Unit 3) — the quantum computer evaluates a function that a classical computer cannot, and classical methods do the rest.

When does this actually help?

This is the most contested question in quantum machine learning. There are three scenarios:

Provable advantage exists for constructed problems. Huang, Broughton, et al. (2022) showed that there exist classification tasks where quantum kernels achieve exponentially better accuracy than any classical learner using the same data; but these tasks are specifically designed around the structure of quantum circuits. They're real proofs of separation, but the tasks are artificial.

Dequantisation results limit the advantage. Tang (2019) showed that several claimed quantum ML speedups (for recommendation systems, principal component analysis — extracting the most important axes of variation in high-dimensional data) can be matched classically under certain conditions. The classical algorithms are "quantum-inspired"; they use similar sampling techniques. This doesn't kill quantum ML, but it narrows the territory where advantage can exist.

The data loading bottleneck. To use a quantum kernel, you need to encode classical data into quantum states. If the data has N features, encoding typically requires $O(N)$ or $O(\text{poly}(N))$ gates. For $N = 10^6$ features (common in practice), this circuit is too deep for near-term hardware. Quantum ML advantage, if it exists for practical problems, likely requires either quantum data (data that's already quantum) or very compact encodings.

What's the honest picture?

Quantum kernel methods are the most rigorous approach to quantum ML; they come with provable guarantees under specific conditions. But the conditions are narrow, and it's an open question whether natural (non-constructed) datasets exhibit the kind of structure that quantum kernels exploit.

The alternative: **variational quantum ML** (parameterised quantum circuits trained end-to-end as classifiers); is more heuristic. It's easier to run on near-term hardware but has weaker theoretical foundations and faces barren plateau problems at scale.

Worked Example

We build a quantum kernel classifier for a synthetic 2D dataset: two classes arranged in a pattern that's separable by a quantum feature map but not by a linear classifier.

1. Generate synthetic data (two interleaved spirals)
2. Define a quantum feature map: encode each 2D point (x_1, x_2) into a 2-qubit state using parameterised rotations and entanglement
3. Compute the quantum kernel matrix by running circuits on Quokka
4. Train a classical SVM with the quantum kernel

5. Compare the decision boundary with a classical RBF (radial basis function) kernel, which measures similarity as a Gaussian that decays with distance: $K(x, x') = e^{-\gamma\|x-x'\|^2}$

→ The next chapter builds the quantum kernel pipeline from scratch, and shows you the code.

Back to the recommender

We started with Netflix's 480,000 users and 10^8-dimensional preference space. Could a quantum kernel capture similarity between users that classical kernels miss?

In principle, yes — a quantum feature map encodes each user's preference vector into an exponentially large Hilbert space, and the Born rule computes a similarity measure that may be classically intractable. In practice, the obstacle is **data loading**: encoding a preference vector with 10^5 features requires 10^5 gates, far too deep for current hardware. For quantum kernels to reach Netflix scale, we'd need either much deeper circuits (fault-tolerant hardware) or data that's already quantum.

The more likely near-term path is smaller, harder problems: classifying outputs of quantum experiments, detecting phases of matter in quantum simulations, or analysing molecular fingerprints (connecting back to Unit 3). In these settings, the data has quantum structure that classical kernels can't access — and the feature vectors are naturally compact.

Reality Check

The quantum ML landscape is contentious. Unlike factoring (where Shor's algorithm gives a clear exponential speedup) or molecular simulation (where the problem is inherently quantum), ML advantage is much harder to establish. The data is classical, the metrics are empirical, and classical baselines keep improving.

What's been demonstrated. Quantum kernel classifiers have been run on real hardware for small datasets (up to ~20 features, ~100 data points). Results match classical kernel SVMs; no advantage has been demonstrated on a practical dataset.

The most promising direction may not be kernel methods at all, but using quantum computers to generate or model data distributions that are hard to sample classically. **Generative models** — models that learn to produce new data resembling the training distribution, rather than classify existing data — are a natural fit. A **Born machine** treats the measurement probability distribution of a parameterised circuit as the model's output distribution; the Born rule becomes the generative engine. These can sample from distributions that require exponential classical resources, though whether those distributions are useful for real-world tasks is an open question.

What would change the picture: Discovery of a natural, practically important dataset where quantum kernels provably outperform all classical methods. Or fault-tolerant quantum computers that eliminate the data-loading bottleneck.

Honest assessment: Quantum ML is the most speculative application area in this book. We include it because the ideas are beautiful (Hilbert space as feature

space, Born rule as kernel evaluator) and because the theoretical framework is sound; but the path to practical advantage is less clear than for any other unit.

Chef's Notes

- **Hilbert space as feature space** is a genuinely deep idea. It reframes quantum mechanics geometrically: a quantum state is a point in a high-dimensional vector space, and measurement is an inner product. If you've ever wondered "what's quantum mechanics good for besides physics?"; this is the cleanest answer: it gives you a feature space that's exponentially large and natively available.
- **Connection to Unit 1 (Logistics).** The quantum kernel circuit — parameterised gates, entanglement, measurement — is structurally identical to a QAOA layer. In Unit 1, we optimised the parameters; here the parameters are fixed by the data. Same hardware, different purpose.
- **The Born rule as a kernel evaluator.** In Unit 2, we used measurement to extract periodicity. In Unit 3, we used it to estimate energy. Here we use it to compute similarity. Same physical operation (measurement), different computational purpose. This is the versatility of quantum mechanics as a computational resource.
- **Connection to Unit 5 (Finance).** Quantum sampling; preparing and measuring quantum states to sample from hard distributions; appears in both quantum ML (generative models) and quantum finance (Monte Carlo). The underlying resource is the same: quantum mechanics lets you sample from distributions that are classically intractable.
- **The SPEC.md open question.** Whether to keep this unit was flagged as an open question. We keep it; with strong caveats; because (a) ML is what most readers will want to hear about, (b) the theoretical framework is honest and rigorous, and (c) the Reality Check is where we earn trust by not overpromising. A book that dodges ML looks like it's hiding something.
- **Further reading:**
  - Havlíček, Córcoles, Temme, et al. (2019). Supervised learning with quantum-enhanced feature spaces. Nature 567:209–212 (arXiv:1804.11326)
  - Huang, Broughton, Mohseni, et al. (2022). Power of data in quantum machine learning. Nature Communications 12:2631
  - Tang (2019). A quantum-inspired classical algorithm for recommendation systems. STOC 2019 (arXiv:1807.04271)
  - Schuld and Killoran (2019). Quantum Machine Learning in Feature Hilbert Spaces. Physical Review Letters 122:040504 (arXiv:1803.07128)
  - Mohanty, Behera & Ferrie (2024). Solving the vehicle routing problem via quantum support vector machines. Quantum Machine Intelligence 6:27
  - Nair & Ferrie (2025). Local surrogates for quantum machine learning. arXiv:2506.09425

Deep-Dive 4: Quantum Kernels in Detail

This deep dive pairs with Unit 4 (Machine Learning), which explained why quantum feature maps could provide computational advantage for classification. Here we build a quantum kernel classifier from scratch.

In This Chapter

- **What you'll learn:** How to design a quantum feature map, compute kernel values from circuit measurements, build a kernel matrix, and train a classical SVM; the complete quantum kernel pipeline.
- **What you need:** From the earlier units, you know qubits and measurement (Unit 1), parameterised rotations (R_Y, R_Z) and CNOT (Deep-Dive 1), and the variational loop (Units 1 and 3). Here we use measurement in a new way: the probability of a specific outcome is the kernel value.
- **Runnable version:** The companion notebook `04-machine-learning.ipynb` trains a quantum kernel SVM on a cloud Quokka.

The geometric picture

In Unit 1, we encoded binary decisions as qubit states: red/blue → $|0\rangle/|1\rangle$. Here we encode continuous data; real-valued feature vectors; into qubit states. The quantum state space becomes a feature space for machine learning.

A single qubit lives on the **Bloch sphere**; a 2D surface parameterised by two angles (θ, ϕ). A data point (x_1, x_2) can be encoded as a point on this sphere using rotation gates: $R_Y(x_1)R_Z(x_2)|0\rangle$.

Two qubits live in a 4-dimensional complex space ($\mathbb{C}^4$). With entanglement, the accessible states fill a much richer space than two independent Bloch spheres; the entangled states have no classical analogue. This is where quantum kernels get their power: the feature space grows exponentially with qubit count.

For n qubits: the feature space has 2^n complex dimensions. A classical computer can't even store a point in this space for $n > 50$. A quantum computer represents it naturally in the physical state of n qubits.

Designing a feature map

A **quantum feature map** is a parameterised circuit $U_\phi(x)$ that maps a classical data point x to a quantum state $|\phi(x)\rangle = U_\phi(x)|0\rangle^n$.

The simplest feature map: product encoding

Encode each feature independently:

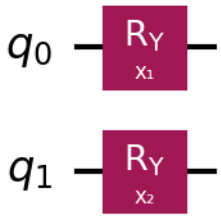

Figure 17. Product encoding: independent Ry rotations on each qubit

This produces a **product state** — a tensor product of independent single-qubit states — with no entanglement. The feature space is just two independent Bloch spheres. The kernel $K(x, x')$ is the product of two cosines; equivalent to a classical kernel. No quantum advantage.

Adding entanglement: the ZZ feature map

Entanglement is what makes quantum feature maps non-classical. Add a CNOT and an interaction term:

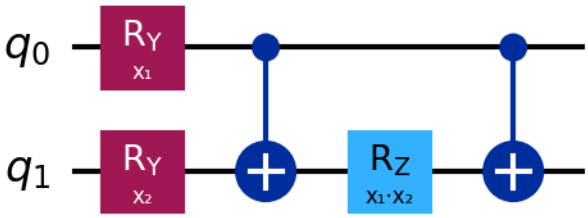

Figure 18. ZZ feature map: Ry encoding + CNOT–Rz–CNOT entangling block

The $R_Z(x_1 \cdot x_2)$ gate, sandwiched between CNOTs, is exactly the ZZ interaction from Deep-Dive 1 (QAOA). But here, instead of encoding a graph edge cost, it encodes a nonlinear feature interaction. The kernel now depends on products of features; it goes beyond what any linear or product encoding can represent.

> **Design principle:** The expressiveness of the feature map comes from entanglement and nonlinear encoding. More layers (repeated encoding + entanglement blocks) create richer feature spaces; but also deeper circuits and more noise.

Computing the kernel value

The kernel between two data points x and x' is:

$$K(x, x') = |\langle \phi(x') | \phi(x) \rangle|^2$$

This is the squared overlap between the two quantum states; the probability of measuring one state and finding it in the other. The Born rule gives us this for free.

The kernel circuit

To compute $K(x, x')$:

1. Apply $U_\phi(x)$; encode data point x
2. Apply $U_\phi(x')^\dagger$; the adjoint (reverse order, negate all angles)
3. Measure; the probability of getting $|00 \ldots 0\rangle$ is $K(x, x')$

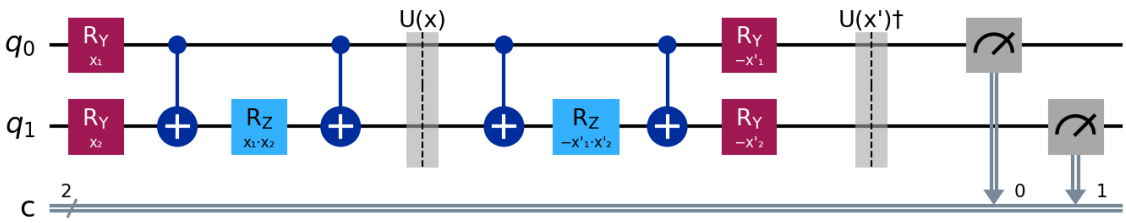

Figure 19. Kernel circuit: U(x) followed by U(x')† and measurement

Run this N_{shots} times. Count how often you get $|00\rangle$. That fraction is $K(x, x')$.

If $x = x'$: the adjoint perfectly undoes the encoding → you always get $|00\rangle$ → $K(x, x) = 1$.

If x and x' are very different: the states don't overlap much → $|00\rangle$ is rare → $K(x, x') \approx 0$.

Building the kernel matrix and training the SVM

For m training points, compute all m^2 pairwise kernel values (using the circuit above for each pair). This gives the **kernel matrix** K_{ij}. Then train a classical SVM with this precomputed kernel; `SVC(kernel='precomputed')` in scikit-learn.

The quantum computer is only used for kernel evaluation. The SVM training, prediction, and cross-validation (testing the trained model on held-out data to estimate real-world accuracy) are entirely classical. This clean separation means quantum ML inherits all the theory and tooling of classical kernel methods.

The companion notebook trains a quantum kernel SVM on a synthetic dataset end-to-end — defining the feature map, computing the kernel matrix, training the SVM, and comparing with a classical baseline.

→ See notebook `04-machine-learning.ipynb` for the runnable version.

When does this actually help?

The honest answer: we don't know for natural datasets.

Provable advantage exists for constructed problems (Huang et al. 2022): classification tasks designed so that the quantum kernel captures structure that no classical kernel can. These proofs are real but the tasks are artificial.

For natural data, the question reduces to: does the data have structure that lives naturally in the entangled quantum feature space? If the data is essentially low-dimensional (as most real datasets are), a classical kernel might capture the same structure with fewer resources.

The data loading bottleneck: encoding d-dimensional classical data requires $O(d)$ gates per data point. For $d = 10^6$ (common in practice), the circuit is too deep for near-term hardware.

The most promising near-term application may be **quantum data**; data that is already quantum (e.g., outputs of quantum experiments or quantum simulations). For such data, classical kernels can't access the relevant feature space at all.

What you should take away

1. **Quantum feature maps encode data as quantum states.** The feature space is the Hilbert space of n qubits; exponentially large and natively available.
2. **The kernel is a measurement probability.** $K(x, x') = \Pr(|0\rangle^n)$ after applying $U_\phi(x)$ then $U_\phi(x')^\dagger$. The Born rule is the kernel evaluator.
3. **Entanglement is the source of power.** Without entanglement, quantum feature maps reduce to classical ones. The CNOT + R_Z interaction (the ZZ gate from Deep-Dive 1) is reused here for a completely different purpose.
4. **The quantum computer computes kernel values; everything else is classical.** SVM training, prediction, and interpretation use standard classical tools.
5. **Advantage is not guaranteed.** Quantum kernels provably help for some problems, but whether natural datasets benefit is an open question.

Unit 5: Finance

Pricing the Unpriceable

The Hook

In 2008, the financial crisis taught the world that mispriced **derivatives** — financial contracts whose value derives from an underlying asset like a stock, a bond, or an interest rate — can destroy economies. At the heart of the crisis were exotic options: contracts that give the holder the right to buy or sell an asset at a fixed price on a future date, but with complex conditions that make them hard to value.

Pricing a derivative means answering one question: **what is the expected payoff** — the profit (if any) the contract delivers at maturity? For a simple **European call option** (the right to buy a stock at a fixed price, exercisable only at maturity), there's an analytical formula (Black-Scholes, 1973). For exotic options — path-dependent, multi-asset, with early exercise features — there is no formula. The standard method is **Monte Carlo simulation**: generate millions of random market scenarios, compute the payoff for each, and take the average.

Monte Carlo works, but it's slow. The statistical error (the uncertainty in your price estimate) shrinks as $1/\sqrt{N}$ where N is the number of samples. To get one more digit of accuracy, you need **100 times more samples**. For complex derivatives that take seconds per sample, this translates to hours or days of computation. Banks run massive compute clusters to price their portfolios overnight. Goldman Sachs reportedly runs over a billion Monte Carlo simulations per day.

The $1/\sqrt{N}$ convergence rate isn't a software limitation. It's a mathematical theorem: for classical random sampling, you cannot do better than $1/\sqrt{N}$. It's a fundamental limit.

Unless you're quantum.

The Bottleneck

The mathematical structure is clean. You have a **random variable** X — a quantity whose value depends on a random outcome; here, the derivative's payoff under a randomly sampled market scenario — with expected value $\mu = \mathbb{E}[X]$ (the probability-weighted average of all possible outcomes). You want to estimate μ.

Classical Monte Carlo: draw N independent samples $X_1, \dots, X_N$ and compute the sample mean $\bar{X} = \frac{1}{N}\sum_i X_i$. The error in this estimate shrinks as:

$$|\bar{X} - \mu| \sim \frac{\sigma}{\sqrt{N}}$$

where σ is the **standard deviation** (a measure of how spread out the payoff values are). To halve the error, quadruple N. To gain a factor of 10 in precision, multiply N by 100. This $1/\sqrt{N}$ convergence rate is a mathematical theorem, not a software

limitation — for any estimator based on independent classical samples, you cannot do better.

For a derivative that needs 6 digits of accuracy (10^{-6} relative error) with $\sigma \sim 1$: you need $N \sim 10^{12}$ samples. At 1 microsecond per sample, that's 12 days.

The Quantum Angle

Amplitude amplification: Grover as a subroutine

Before we get to financial pricing, we need the tool that makes it possible: **amplitude amplification**, which is Grover's search algorithm generalised.

Grover's algorithm (1996) solves unstructured search: given a function $f : \{0,1\}^n \to \{0,1\}$ with M solutions among $N = 2^n$ inputs, find a solution. Classically: $O(N/M)$ queries. Quantumly: $O(\sqrt{N/M})$ queries. A quadratic speedup.

The key operation is the **Grover iterator** $G = S_0 \cdot S_f$, where S_f flips the phase of solutions and S_0 reflects about the initial superposition. Geometrically, G is a rotation in the 2D subspace spanned by "solutions" and "non-solutions." Each application rotates the state vector by an angle 2θ toward the solution space, where $\sin\theta = \sqrt{M/N}$.

After $k = O(\sqrt{N/M})$ iterations, the overlap with the solution space is close to 1. Measure, and you get a solution with high probability.

Quantum amplitude estimation

Now the key insight: the angle θ encodes M/N; the fraction of inputs that are solutions. If you can measure θ precisely, you can estimate M/N without ever finding a specific solution.

Quantum Amplitude Estimation (QAE) does exactly this. It uses the same controlled-powers-plus-inverse-QFT pattern we built in Deep-Dive 2 for period-finding — that pattern is called **Quantum Phase Estimation** (QPE), and it works for any unitary operator, not just the modular exponentiation oracle from Shor's algorithm. Here we apply QPE to the Grover iterator G. The eigenvalues of G are $e^{\pm 2i\theta}$, and QPE extracts θ with precision $O(1/N_{\text{queries}})$ — that's $1/N$, not $1/\sqrt{N}$.

Applied to Monte Carlo:

1. **Encode** the probability distribution as a quantum state: prepare $\sum_x \sqrt{p(x)}|x\rangle$
2. **Compute** the payoff function: mark states where the payoff exceeds a threshold (this is the "oracle" for Grover)
3. **Run QAE**: extract the amplitude $a = \sqrt{\Pr[\text{payoff} > \text{threshold}]}$
4. The expected payoff is encoded in a, estimated to precision ϵ using $O(1/\epsilon)$ queries

Classical Monte Carlo: precision ϵ requires $O(1/\epsilon^2)$ samples. Quantum amplitude estimation: precision ϵ requires $O(1/\epsilon)$ queries.

Same accuracy, quadratically fewer samples. For the derivative that needs 10^{12} classical samples: quantum estimation needs 10^6 queries. At the same time per query, that's 1 second instead of 12 days.

When does $\sqrt{}$ matter?

A quadratic speedup sounds modest compared to Shor's exponential speedup. But for Monte Carlo, quadratic is transformative. The reason: the baseline is already polynomial (Monte Carlo is $O(1/\epsilon^2)$), and the constant factors are large. Going from 10^{12} to 10^6 is the difference between "we need a compute cluster" and "we need a laptop."

Worked Example

Price a European call option using quantum amplitude estimation. Compare the convergence rate with classical Monte Carlo.

Setup: - Stock price $S_0 = 100$, strike price $K = 105$ (the fixed price at which the option can be exercised), volatility $\sigma = 20\%$ (the annualised standard deviation of the stock's returns — how wildly the price swings), maturity $T = 1$ year (the date when the option expires), risk-free rate $r = 5\%$ (the return on a riskless investment, like a government bond) - Payoff: $\max(S_T - K, 0)$ - Black-Scholes analytical price: ~$8.02

We discretise the stock price into 2^n bins, encode the **log-normal distribution** (stock prices are modelled so that their logarithm follows a normal distribution, ensuring prices can't go negative) as a quantum state, and use the Grover oracle to mark states where $S_T > K$.

→ The next chapter builds the amplitude estimation circuit from Grover's algorithm, and shows you the code.

Back to the trading floor

We priced a European call option — the simplest derivative there is. How does this help Goldman Sachs price exotic instruments?

The pipeline is identical. A more complex derivative has a more complex payoff function (path-dependent, multi-asset, with barriers and early exercise), but the quantum algorithm doesn't care: it just needs an oracle that marks states where the payoff exceeds a threshold, and QAE estimates the probability. The quadratic speedup applies regardless of the payoff's complexity. For Goldman's billion-simulation-per-day workload, the gap between 10^{12} classical samples and 10^6 quantum queries is the gap between a compute cluster and a single machine.

Reality Check

What's been demonstrated. Goldman Sachs and IBM published a series of papers (2019–2021) on quantum amplitude estimation for derivative pricing, demonstrating the algorithm on simulators for simple options. Actual quantum hardware

experiments have been limited to toy models (2–4 qubits, highly simplified distributions).

The depth problem. QAE requires quantum phase estimation, which requires deep circuits: $O(1/\epsilon)$ applications of the Grover iterator. For useful precision ($\epsilon \sim 10^{-3}$), that's thousands of sequential operations, each involving the full oracle circuit. On NISQ devices with short coherence times, this is currently infeasible.

Approximate QAE. Variants like iterative QAE (Suzuki et al. 2020) and maximum likelihood QAE reduce the circuit depth at the cost of more measurements. These are more NISQ-friendly but haven't demonstrated practical advantage.

What would change the picture. Fault-tolerant quantum computers with $O(10^3)$ logical qubits and the ability to run circuits of depth $O(10^4)$. The financial industry is actively investing: JP Morgan, Goldman Sachs, BBVA, and others have quantum computing research teams focused on this exact application.

The quadratic wall. A quadratic speedup means quantum advantage arrives only above a certain problem size, where the improved scaling overcomes the overhead of quantum error correction. For Monte Carlo, estimates suggest this crossover requires 10^3–10^4 logical qubits; ambitious but within the scope of hardware roadmaps.

Chef's Notes

- **Grover's algorithm appears here, not as a standalone topic.** Most books introduce Grover as "quantum search"; finding a needle in a haystack. That's correct but unrelatable. Here, Grover appears as a subroutine inside amplitude estimation, motivated by a real computational bottleneck ($1/\sqrt{N}$ convergence). The speedup matters because Monte Carlo is everywhere, not because search is theoretically interesting.
- **The quadratic speedup is the most common quantum advantage.** Exponential speedups (Shor, simulation) get the headlines, but quadratic speedups (Grover, amplitude estimation) apply to a much broader class of problems. Anywhere you're sampling, searching, or optimising, a quadratic improvement is potentially available.
- **Connection to Unit 2.** QAE uses QPE, which uses the QFT; the same machinery from Shor's algorithm. The quantum ingredient is the same: extract a quantity (a period, an eigenvalue, an amplitude) encoded in the phase of a quantum state.
- **Further reading:**
  - Montanaro (2015). Quantum speedup of Monte Carlo methods. arXiv:1504.06987
  - Stamatopoulos, Egger, Sun, et al. (2020). Option pricing using quantum computers. Quantum 4:291
  - Suzuki, Uno, Raymond, et al. (2020). Amplitude estimation without phase estimation. Quantum Information Processing 19:75
  - Grover (1996). A fast quantum mechanical algorithm for database search. arXiv:quant-ph/9605043

- 3Blue1Brown (2023). Grover's algorithm visually explained (YouTube video)

Deep-Dive 5: Amplitude Estimation from Grover

This deep dive pairs with Unit 5 (Finance), which explained why Monte Carlo pricing is slow and how quantum amplitude estimation offers a quadratic speedup. Here we build the amplitude estimation circuit from Grover's algorithm.

In This Chapter

- **What you'll learn:** How Grover's algorithm works geometrically, how quantum amplitude estimation extracts probabilities, and why the quadratic speedup matters for Monte Carlo.
- **What you need:** From Deep-Dive 2, you know the controlled-powers-plus-inverse-QFT pattern that extracts eigenvalues from a unitary operator (that pattern is called **Quantum Phase Estimation**, or QPE). Here we apply QPE to a new operator — the Grover iterator — and the eigenvalue it extracts encodes a probability rather than a period.
- **Runnable version:** The companion notebook `05-finance.ipynb` demonstrates amplitude estimation on a cloud Quokka.

Grover's algorithm: the geometric picture

The search problem

You have a function $f : \{0,1\}^n \to \{0,1\}$ with M "marked" inputs (where $f(x) = 1$) among $N = 2^n$ total. Find a marked input.

Classically: $O(N/M)$ queries (check inputs one by one). Quantumly: $O(\sqrt{N/M})$ queries. Quadratic speedup.

Two subspaces

Define two states:

$$|\text{good}\rangle = \frac{1}{\sqrt{M}} \sum_{x:f(x)=1} |x\rangle, \quad |\text{bad}\rangle = \frac{1}{\sqrt{N-M}} \sum_{x:f(x)=0} |x\rangle$$

The initial uniform superposition lies in the plane spanned by these two states:

$$|s\rangle = \sin\theta\,|\text{good}\rangle + \cos\theta\,|\text{bad}\rangle$$

where $\sin\theta = \sqrt{M/N}$. For $M \ll N$ (a needle in a haystack), θ is small; the initial state is almost entirely $|\text{bad}\rangle$.

The Grover iterator as a rotation

The Grover iterator G consists of two reflections:

1. **Oracle reflection** S_f: flip the phase of marked states. $S_f|x\rangle = (-1)^{f(x)}|x\rangle$. This is phase kickback from Deep-Dive 2; the oracle marks solutions by flipping their phase.
2. **Diffusion** $S_0 = 2|s\rangle\langle s| - I$: reflect about the mean amplitude. Here $|s\rangle\langle s|$ is an **outer product** — it produces an operator (a matrix), not a number. Applied to any state $|\psi\rangle$, it gives $|s\rangle\langle s|\psi\rangle = \langle s|\psi\rangle \cdot |s\rangle$: project onto $|s\rangle$, scaled by the overlap. (The notation $\langle s|$ is called a **bra** — the row-vector partner of the ket $|s\rangle$, obtained by taking the adjoint.) So $2|s\rangle\langle s| - I$ doubles the component along $|s\rangle$ and subtracts the original — a reflection about $|s\rangle$. In circuit terms: $H^{\otimes n} \cdot (2|0\rangle\langle 0| - I) \cdot H^{\otimes n}$.

Two reflections make a rotation. $G = S_0 \cdot S_f$ rotates the state by angle 2θ toward $|\text{good}\rangle$ in the 2D plane:

$$G^k|s\rangle = \sin((2k+1)\theta)\,|\text{good}\rangle + \cos((2k+1)\theta)\,|\text{bad}\rangle$$

After $k_{\text{opt}} = \lfloor \pi/(4\theta) \rfloor$ iterations, the state is nearly $|\text{good}\rangle$. Measure → get a marked item with probability $\geq 1 - O(M/N)$.

> **The key insight:** Grover doesn't search. It rotates. The search space isn't explored one item at a time; the quantum state rotates continuously from "mostly bad" to "mostly good." The number of rotations needed is $O(1/\theta) = O(\sqrt{N/M})$.

The oracle circuit

For the financial application (Unit 5), the oracle marks states where the stock price exceeds the strike: $f(x) = 1$ if $\text{price}(x) > K$.

This is a **comparator circuit**: given a quantum register encoding a discretised price, flip an ancilla if the price exceeds a threshold. The ancilla, prepared in $|-\rangle$, converts the flip to a phase via kickback — the same trick from Deep-Dive 2.

The diffusion circuit

The diffusion operator $2|s\rangle\langle s| - I$ is implemented as:

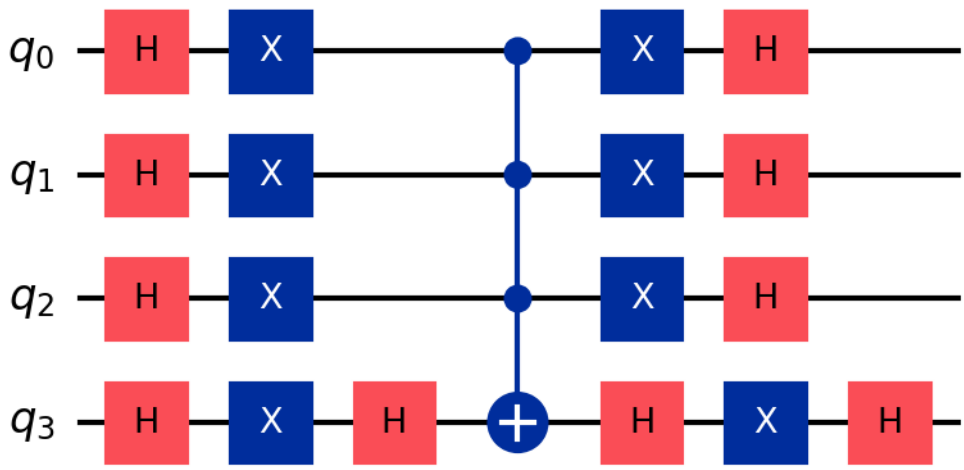

Figure 20. Grover diffusion circuit for 4 qubits

The pattern is: Hadamard all qubits, flip all qubits (X), apply a **multi-controlled** Z (which flips the phase only when all qubits are $|1\rangle$ — decomposed as H + multi-controlled X + H on the last qubit), then undo the X and H layers. This is the most gate-intensive part of Grover's algorithm, but it's a fixed overhead per iteration.

From Grover to amplitude estimation

The connection

Grover's algorithm uses G to find marked items. But the angle θ; which determines how many iterations are needed; also encodes the fraction of marked items: $\sin^2\theta = M/N$.

Quantum Amplitude Estimation (QAE) extracts θ directly, without finding any specific marked item. This is exactly what we need for Monte Carlo pricing: we don't want a specific market scenario, we want the probability of favourable scenarios (which encodes the expected payoff).

QPE on the Grover operator

In Deep-Dive 2, we used a pattern — controlled powers of a unitary, followed by the inverse QFT — to extract the period of $a^x \bmod N$. That pattern is **Quantum Phase Estimation** (QPE), and it works for any unitary operator: if $U|v\rangle = e^{i\phi}|v\rangle$, QPE extracts ϕ.

Here we apply QPE to the Grover iterator G, which has eigenvalues $e^{\pm 2i\theta}$. The recipe is identical:

1. Prepare ancilla qubits in $|+\rangle$ (via Hadamard)
2. Apply controlled-G^{2^k} operations (controlled by ancilla qubit k)
3. Apply inverse QFT to the ancillas
4. Measure → get an estimate of θ

From θ: compute $\tilde{a} = \sin^2\theta \approx M/N$. This is the amplitude (probability) we want.

The convergence advantage

With m ancilla qubits, QPE estimates θ to precision $O(1/2^m)$. This requires $O(2^m)$ applications of the Grover operator G.

Each application of G is one "query" to the oracle. So:

- **Precision** ϵ requires $O(1/\epsilon)$ queries
- **Classical Monte Carlo** requires $O(1/\epsilon^2)$ samples for the same precision

The quadratic advantage: same accuracy, quadratically fewer queries.

Target precision	Classical samples	Quantum queries	Speedup
10^{-1}	100	10	10×
10^{-3}	10^6	10^3	1,000×
10^{-6}	10^{12}	10^6	10^6×

The companion notebook runs amplitude estimation end-to-end — constructing the Grover oracle for option pricing, applying QPE, and comparing convergence against classical Monte Carlo.

→ **See notebook `05-finance.ipynb` for the runnable version.**

What you should take away

1. **Grover's algorithm is a rotation, not a search.** It rotates the quantum state from "mostly wrong" to "mostly right" in the 2D good/bad plane. The number of rotations is $O(\sqrt{N/M})$.
2. **Amplitude estimation = QPE on Grover.** The angle θ that controls Grover encodes the probability $\sin^2\theta = M/N$. QPE extracts θ with precision $O(1/2^m)$ using $O(2^m)$ queries.
3. **The quadratic speedup is $1/\epsilon$ vs. $1/\epsilon^2$.** This transforms Monte Carlo from "days on a cluster" to "seconds on a quantum computer" for high-precision estimates.
4. **Phase kickback appears again.** The oracle uses it (Deep-Dive 2). The Grover iterator uses it. QPE uses it. It's the same mechanism at every level.
5. **The circuit depth is the bottleneck.** QAE requires $O(1/\epsilon)$ sequential applications of the full Grover operator. Each application includes the oracle circuit. For useful precision, this means deep circuits; a fault-tolerant algorithm.

Unit 6: Supply Chains

10,000 Nurses, 50 Hospitals, 1 Schedule

The Hook

Every winter, Britain's National Health Service enters a scheduling crisis. Across 50 hospitals, 10,000 nurses need to be assigned to shifts; day, evening, night; across wards, specialties, and locations. Each nurse has qualifications (ICU-certified? paediatrics-trained?), contractual constraints (maximum hours, minimum rest), preferences (no nights, school pickup at 3 PM), and legal requirements (EU Working Time Directive: no more than 48 hours per week, 11 hours rest between shifts).

The NHS spends an estimated **£3 billion per year** on agency nurses; temporary staff hired at premium rates because the permanent workforce can't be scheduled efficiently. Not because there aren't enough nurses (though there often aren't), but because the scheduling problem is so complex that planners routinely produce suboptimal rosters, leaving gaps that agencies fill at 2–3× the cost.

This isn't a peculiarly British problem. Airlines schedule crews across thousands of flights. Manufacturing plants assign workers to production lines. Call centres staff agents across time zones. In every case, the mathematical structure is the same: assign N resources to M slots subject to constraints, minimising cost (or maximising fairness, or minimising overtime).

These are **constraint satisfaction problems**, and they are NP-hard. The nurse scheduling problem (NSP) was proven NP-hard by Osogami and Imai in 2000. No polynomial-time algorithm exists (unless P = NP). Classical solvers; integer linear programming (ILP), constraint programming (CP), simulated annealing; can handle instances with hundreds of nurses. At thousands, they struggle. At tens of thousands, they break.

The Bottleneck

The nurse scheduling problem has a distinctive mathematical structure. It's not a pure optimisation problem (like MaxCut); it's a **constrained** optimisation problem, with both hard constraints (must satisfy) and soft constraints (prefer to satisfy).

Hard constraints: - Each shift must be staffed by a qualified nurse - No nurse works more than 48 hours/week - Minimum 11 hours between consecutive shifts - No nurse works more than 5 consecutive days

Soft constraints (with penalties): - Honour nurse preferences (nights, weekends) - Distribute unpopular shifts fairly - Minimise travel between hospital sites - Balance workload across the team

The search space is enormous. With N nurses and S shifts, there are N^S possible assignments (ignoring constraints). For $N = 100$ nurses and $S = 500$ shifts across a week, that's $100^{500} \approx 10^{1000}$; absurdly beyond brute force.

The real difficulty is the **interaction structure**. Constraints are non-local: assigning Nurse A to Monday night affects whether Nurse B can take Tuesday morning (if they share qualifications and wards). These interactions create a web of dependencies that classical local-search methods struggle with; improving one part of the schedule can break another.

The Quantum Angle

QUBO: turning constraints into energy

The key idea is **Quadratic Unconstrained Binary Optimisation** (QUBO). We convert the constrained scheduling problem into an unconstrained minimisation of a quadratic function over binary variables; then map that to the ground state of a quantum Hamiltonian.

Each binary decision gets a variable: $x_{n,s} = 1$ if nurse n is assigned to shift s, 0 otherwise. The total number of binary variables is $N \times S$.

Hard constraints become **penalty terms**: add a large positive cost for any assignment that violates them. Soft constraints become **smaller penalties**, proportional to how undesirable the violation is.

The result is a cost function:

$$C(x) = \sum_i \sum_j Q_{ij} x_i x_j + \sum_i c_i x_i$$

where Q_{ij} encodes the interactions between binary decision variables and c_i encodes the individual preferences. This is the QUBO form. The optimal schedule is the binary string x^* that minimises $C(x)$.

From QUBO to Ising Hamiltonian

The substitution $x_i = \frac{1-Z_i}{2}$ converts each binary variable (0 or 1) to a spin variable ($+1$ or -1). The QUBO cost function becomes an **Ising Hamiltonian**:

$$H = \sum_{i<j} J_{ij} Z_i Z_j + \sum_i h_i Z_i + \text{const}$$

The ground state of this Hamiltonian is the optimal schedule. This is the same pattern as Unit 1 (MaxCut → cost Hamiltonian) and Unit 3 (molecule → molecular Hamiltonian); different problem, same mathematical structure.

Quantum annealing

There are two quantum approaches to finding the ground state of an Ising Hamiltonian:

Gate-based QAOA (Unit 1): apply alternating problem and mixer unitaries, optimise parameters variationally. Works on universal quantum computers.

Quantum annealing: a different paradigm entirely. Start in the ground state of a simple Hamiltonian H_{init} (whose ground state you know; typically all qubits in $|+\rangle$). Slowly transform H_{init} into the problem Hamiltonian H_{problem}:

$$H(t) = \left(1 - \frac{t}{T}\right) H_{\text{init}} + \frac{t}{T} H_{\text{problem}}$$

The **adiabatic theorem** guarantees: if you change the Hamiltonian slowly enough (large T), the system stays in the ground state throughout — it continuously adapts to the changing energy landscape rather than getting excited. At the end ($t = T$), you're in the ground state of H_{problem} — the optimal schedule.

Why quantum annealing might help

Classical **simulated annealing** explores the energy landscape by making random local moves — flip one variable and check if the cost improves. It occasionally accepts worse moves (uphill steps) with a probability that decreases over time, mimicking a cooling process. This randomness helps escape local minima, but the method gets stuck when the optimal solution is separated from the current state by a tall energy barrier — it has to go over the barrier.

Quantum annealing can tunnel through barriers. Quantum tunnelling; the same effect that allows radioactive decay; lets the quantum state traverse energy barriers without climbing them. Barriers that are tall but narrow are easy to tunnel through; barriers that are tall and wide are hard.

This gives quantum annealing a structural advantage for problems with specific landscape properties: many tall, narrow barriers. Whether real-world scheduling problems have this property is an empirical question; and a contested one.

Worked Example

Schedule 8 nurses across 4 shifts (Mon-day, Mon-night, Tue-day, Tue-night) in 2 wards, with constraints:

- Each shift/ward needs exactly 1 nurse
- No nurse works consecutive shifts (11-hour rest)
- Nurse preferences: 3 nurses prefer day shifts, 2 prefer nights

Formulate as QUBO: 8 nurses × 8 shift-slots = 64 binary variables (reducible with symmetries). Solve with simulated annealing (classical) and compare with a QAOA circuit.

→ **See notebook `06-supply-chains.ipynb` for the runnable version.**

→ Want to understand the algorithm in detail? Read the next chapter.

Reality Check

D-Wave. The company most associated with quantum annealing, D-Wave has built machines with over 5,000 qubits; far more than any gate-based machine. Their Advantage system can natively solve QUBO problems with thousands of variables.

But the qubits are noisy, the connectivity is limited (Pegasus graph), and embedding real problems onto the hardware graph introduces significant overhead.

The speedup debate. Despite two decades of effort, no definitive quantum speedup for annealing has been demonstrated on a practical problem. Rønnow et al. (2014) found no speedup over classical simulated annealing on random instances. King et al. (2023) showed advantage on specific crafted instances. The picture is nuanced: annealing may offer advantages for specific problem structures, but general-purpose speedup remains unproven.

Hybrid solvers. D-Wave's most practically useful tool is its **hybrid classical-quantum solver** (Leap), which uses classical heuristics for most of the work and the quantum annealer for specific subproblems. These hybrids outperform pure classical solvers on some logistics and scheduling benchmarks; but it's hard to attribute the advantage specifically to the quantum component.

Gate-based vs. annealing. QAOA on gate-based hardware offers more flexibility (arbitrary cost functions, tuneable depth) but currently operates on far fewer qubits. Annealing offers more qubits but less control. The two approaches may converge as hardware improves.

What would change the picture. A demonstration of quantum speedup on a real scheduling or logistics problem with practical constraints; not a crafted benchmark instance. This would require either significant improvements in annealer quality or gate-based machines with $O(10^3)$ logical qubits.

Chef's Notes

- **QUBO is the universal input format for constrained optimisation on quantum computers.** Any problem you can write as a quadratic function over binary variables can be solved (in principle) by quantum annealing or QAOA. Scheduling, routing, portfolio optimisation, resource allocation; they all have QUBO formulations. The D-Wave Ocean SDK has a library of pre-built QUBO templates for common problems.
- **The QUBO-to-Ising map is the bridge between classical OR and quantum computing.** If you know operations research, you already know how to formulate QUBO problems. The quantum part is "just" a different solver for the same formulation.
- **Quantum tunnelling vs. thermal hopping.** Simulated annealing hops over barriers using thermal energy. Quantum annealing tunnels through them. Neither is universally better; it depends on the barrier geometry. Tall-narrow barriers favour tunnelling; short-wide barriers favour hopping.
- **Connection to Unit 1.** QAOA and annealing are two paradigms for the same goal: finding the ground state of a cost Hamiltonian. Unit 1 introduced QAOA on MaxCut (a simple QUBO). This unit shows the QUBO formulation for a richer problem and introduces the annealing alternative.
- **Further reading:**
  - Lucas (2014). Ising formulations of many NP problems. Frontiers in Physics 2:5

- Farhi et al. (2001). A quantum adiabatic evolution algorithm applied to random instances of an NP-complete problem. Science 292:472–475 (arXiv:quant-ph/0001106)
- Rønnow et al. (2014). Defining and detecting quantum speedup. Science 345:420–424 (arXiv:1401.2910)
- King et al. (2023). Quantum critical dynamics in a 5,000-qubit programmable spin glass. Nature 617:61–66 (arXiv:2207.13800)
- Katzgraber, Hamze & Andrist (2014). Glued trees are not hard for quantum annealing: designing hard Ising instances. Physical Review X 4:021008 (arXiv:1404.6539)
- Katzgraber, Trebst, Huse & Troyer (2006). Feedback-optimized parallel tempering Monte Carlo. Journal of Statistical Mechanics P03018 (arXiv:cond-mat/0602085)
- Mandrà, Katzgraber & Thomas (2017). The pitfalls of planar spin-glass benchmarks. Quantum Science and Technology 2:038501 (arXiv:1703.00622)

Deep-Dive 6: QUBO Engineering

This deep dive pairs with Unit 6 (Supply Chains), which introduced nurse scheduling and constraint satisfaction. Here we show how to formulate real-world constraints as QUBO problems and solve them on quantum hardware.

In This Chapter

- **What you'll learn:** How to turn constraints into penalty terms, convert QUBO to an Ising Hamiltonian, and why quantum annealing tunnels through barriers that classical methods can't climb.
- **What you need:** From Deep-Dive 1, you know cost Hamiltonians and the ZZ gate. Here we extend the cost Hamiltonian concept to constrained problems — where not every binary string is a valid solution.
- **Runnable version:** The companion notebook `06-supply-chains.ipynb` solves a small scheduling QUBO on a cloud Quokka.

From constraints to energy

Real problems have rules

In Deep-Dive 1 (QAOA), our cost function was simple: count the cut edges. Every binary string was a valid colouring — there were no constraints, just better and worse solutions. Flip any node's colour, and you still have a colouring. Maybe a bad one, but a legal one.

Scheduling isn't like that. "Assign two nurses to the same shift" isn't a bad schedule — it's an invalid schedule. Real problems come with rules, and breaking a rule isn't just costly; it's forbidden.

QUBO (Quadratic Unconstrained Binary Optimisation) handles this by a trick you've already seen: turn everything into energy. Hard constraints become **penalty terms** — large positive costs added whenever a rule is broken. Preferences become small costs. Then "find the best valid schedule" becomes "find the lowest-energy state" — exactly the ground-state search from Unit 1. A solution with zero penalty satisfies all rules.

Encoding equality constraints

"Each shift must have exactly one nurse assigned."

Binary variables: $x_{n,s} = 1$ if nurse n takes shift s. The constraint for shift s:

$$\sum_n x_{n,s} = 1$$

Convert to penalty: $P \cdot \left(\sum_n x_{n,s} - 1\right)^2$ where P is large.

Expand (using $x_i^2 = x_i$ for binary variables):

$$P\cdot\left(\sum_n x_{n,s}^2 + 2\sum_{n<n'} x_{n,s}x_{n',s} - 2\sum_n x_{n,s} + 1\right) = P\cdot\left(-\sum_n x_{n,s} + 2\sum_{n<n'} x_{n,s}x_{n',s} + 1\right)$$

This is quadratic in the binary variables; it fits QUBO form. The minimum (zero penalty) occurs when exactly one $x_{n,s} = 1$.

Encoding inequality constraints

"No nurse works more than 3 shifts."

Equality constraints were clean: square the violation, get a quadratic penalty. Inequalities are trickier, because $\leq$ doesn't have a single "right answer" — any number from 0 to 3 is fine.

The simplest approach: **penalise every way the constraint can be violated.** If nurse n works 4 or more shifts, some group of 4 shifts must all be assigned. So penalise every possible group of 4:

$$P \cdot \sum_{s_1<s_2<s_3<s_4} x_{n,s_1}\, x_{n,s_2}\, x_{n,s_3}\, x_{n,s_4}$$

This penalty is zero when the nurse works 3 or fewer shifts (no group of 4 exists), and positive otherwise. It's conceptually clean — but there's a problem: it's a product of four variables. QUBO only allows products of two. We need everything to be quadratic.

There are standard tricks to reduce higher-order terms to quadratic form (introducing extra binary variables that represent pairs), but they add complexity. For small scheduling problems, the overhead is manageable. For large problems, specialised tools like D-Wave's `dwavebinarycsp` handle the reduction automatically.

The honest takeaway: equality constraints map to QUBO naturally. Inequality constraints require more work, and this engineering — choosing the right penalty structure, managing the extra variables — is where practical QUBO formulation gets interesting. It's part of why operations researchers still have jobs.

> **The QUBO art:** simple constraints (equalities, pairwise exclusions) map cleanly to quadratic penalties. Complex constraints (inequalities, conditional requirements) require auxiliary variables and careful penalty tuning. The penalty weight P must be large enough to enforce the constraint, but not so large that it distorts the optimisation landscape.

Soft constraints as linear costs

"Nurse A prefers day shifts."

Add a small cost for night assignments: $w \cdot x_{A,\text{night}}$ where $w \ll P$. The optimiser will avoid night shifts when possible, but will violate the preference if needed to

satisfy hard constraints (because the hard-constraint penalty P is much larger than the preference cost w).

This priority structure; hard constraints as large penalties, soft preferences as small costs; is what makes QUBO flexible enough for real scheduling problems.

From QUBO to Ising Hamiltonian

Once you have a QUBO cost function, turning it into a Hamiltonian is mechanical — and it lands you right back in familiar territory. Substitute $x_i = \frac{1-Z_i}{2}$ (so $x_i = 0$ when qubit i is $|0\rangle$ and $x_i = 1$ when it's $|1\rangle$):

$$x_i x_j = \frac{(1-Z_i)(1-Z_j)}{4} = \frac{1 - Z_i - Z_j + Z_i Z_j}{4}$$

Every QUBO term becomes a combination of: - $Z_i Z_j$ terms (the same interactions as MaxCut in Unit 1) - Z_i terms (local fields) - Constants (global energy offset)

The resulting Ising Hamiltonian $H = \sum_{ij} J_{ij} Z_i Z_j + \sum_i h_i Z_i + \text{const}$ is exactly the kind of operator we built QAOA circuits for in Deep-Dive 1. Same gates, same structure, just more terms.

Same gates, bigger circuit

The QAOA circuit for a QUBO problem uses the same building blocks as Deep-Dive 1: - ZZ gates (CNOT sandwich) for the J_{ij} terms - R_Z rotations for the h_i terms - R_X mixer rotations

The only difference from Deep-Dive 1 is scale: a QUBO with n variables and $O(n^2)$ interactions needs $O(n^2)$ CNOT sandwiches per QAOA layer. The circuit is wider and deeper, but the structure is identical.

Quantum annealing: a different paradigm

The adiabatic approach

QAOA (Deep-Dive 1) uses gate-based quantum computing: discrete operations applied in sequence. **Quantum annealing** takes a completely different approach: continuous time evolution.

Start in the ground state of a simple Hamiltonian H_0 (typically a **transverse field** $\sum_i X_i$ — an X operator on every qubit — whose ground state is $|+\rangle^n$, which we already know how to prepare from Deep-Dive 1). Slowly interpolate to the problem Hamiltonian H_1:

$$H(t) = \left(1 - \frac{t}{T}\right) H_0 + \frac{t}{T} H_1$$

The **adiabatic theorem** says: if the interpolation is slow enough, the system stays in the ground state throughout. At the end ($t = T$), you're in the ground state of H_1; the optimal solution.

How slow is slow enough?

"Slow enough" means $T \geq O(1/\Delta_{\text{min}}^2)$ where Δ_{min} is the minimum spectral gap; the smallest energy difference between the ground state and the first excited state during the anneal.

For easy problems, Δ_{min} is large and the anneal is fast. For hard problems, Δ_{min} can be exponentially small; the anneal time grows exponentially, and any speedup disappears.

The open question: for which problems is the spectral gap large enough that annealing is practical?

Quantum tunnelling: the potential advantage

Classical simulated annealing gets stuck at energy barriers; to escape a local minimum, it must thermally "hop" over the barrier, which requires the barrier to be low or the temperature to be high.

Quantum annealing can **tunnel through** barriers. Quantum tunnelling; the same physics that enables radioactive decay; lets the quantum state pass through energy barriers that are tall but narrow. The tunnelling rate depends on the barrier width, not its height.

For problems with many tall, narrow barriers: quantum annealing may explore the energy landscape more efficiently than classical annealing. For problems with wide barriers: classical annealing may be better.

> **The honest picture:** Despite two decades of effort, definitive quantum speedup for annealing on practical problems remains unproven. D-Wave's quantum annealers (5,000+ qubits) solve some crafted instances faster than classical methods, but general-purpose advantage has not been demonstrated. This is an active research frontier.

What you should take away

1. **QUBO is the universal language for constrained binary optimisation.** Any problem with binary decisions and quadratic interactions can be written as a QUBO. The map from constraints to penalties is the key engineering step.
2. **QUBO → Ising is mechanical.** The substitution $x_i = (1-Z_i)/2$ converts any QUBO to an Ising Hamiltonian that can be solved with QAOA or annealing.
3. **The circuit is the same as Deep-Dive 1, just bigger.** More variables → more qubits. More interactions → more ZZ gates. Same structure.
4. **Annealing is a different paradigm.** Continuous time evolution instead of discrete gates. The adiabatic theorem provides the guarantee. Tunnelling provides the (potential) advantage.

5. **Penalty weight matters.** P must be large enough that violating a hard constraint is always worse than any feasible solution. But too large, and the energy landscape becomes dominated by the constraint surface, making the optimisation harder.

Unit 7: Materials Science

The Room-Temperature Superconductor Problem

The Hook

A room-temperature superconductor would be one of the most transformative discoveries in the history of technology. Superconductors carry electrical current with **zero resistance**; no energy lost to heat, no wasted power. Today's superconductors require cooling to near absolute zero (below −140°C for "high-temperature" superconductors, below −250°C for conventional ones). The cooling infrastructure is expensive, bulky, and energy-intensive.

If you could superconduct at room temperature, you could: - **Transmit electricity with zero loss.** The U.S. electrical grid loses about 5% of generated power to transmission resistance; roughly $20 billion per year. - **Build MRI machines without liquid helium.** A hospital MRI costs $1–3M, largely because of the superconducting magnets that need continuous helium cooling. - **Enable practical fusion reactors.** Tokamak fusion requires superconducting magnets to confine plasma. Room-temperature superconductors would simplify the engineering dramatically. - **Revolutionise computing.** Superconducting circuits (already the basis of most quantum computers) could operate without cryogenics.

The problem is that we can't predict which materials will superconduct at high temperatures. The physics of superconductivity in complex materials involves **strongly correlated electrons** — quantum systems where the interactions between electrons are so strong that the **mean-field** approximation (treating each electron as if it moves in the average field of all the others — the same simplification we saw fail in Unit 3) breaks down completely. We can't simulate these systems classically.

The **Hubbard model**; the simplest model that captures the essential physics of strongly correlated electrons; has been studied for over 60 years. Its phase diagram in two dimensions remains unsolved. We literally do not know whether the 2D Hubbard model supports high-temperature superconductivity. This is not a matter of computational effort; it's a matter of computational impossibility with classical methods.

The Bottleneck

The Hubbard model

The Hubbard model describes electrons hopping on a lattice, with an energy cost for double occupation. If you've read Deep-Dive 3 (VQE), the notation will be familiar — the same creation operators $c^\dagger$ and annihilation operators c from the molecular Hamiltonian, now on a lattice:

$$H = -t \sum_{\langle i,j \rangle, \sigma} (c_{i\sigma}^\dagger c_{j\sigma} + \text{h.c.}) + U \sum_i n_{i\uparrow} n_{i\downarrow}$$

Here $c^\dagger_{i\sigma}$ creates an electron with spin σ (up or down) at site i. The symbol $\langle i, j\rangle$ means the sum runs over neighbouring sites on the lattice. The notation "h.c." (Hermitian conjugate) means "add the dagger of the preceding term" — it ensures the Hamiltonian is physically valid. And $n_{i\sigma} = c^\dagger_{i\sigma} c_{i\sigma}$ is the **number operator** — it counts whether site i is occupied by a spin-σ electron (0 or 1).

Two parameters: t (the hopping amplitude — kinetic energy) and U (the on-site repulsion — interaction energy). The ratio U/t controls the physics:

- **Small U/t:** electrons hop freely, metal
- **Large U/t:** electrons localise to avoid repulsion, insulator (Mott insulator)
- **Intermediate U/t:** neither picture works; strongly correlated regime; this is where superconductivity may live

Why classical methods fail

The Hilbert space of N electrons on L sites with spin has dimension $\binom{2L}{N}$. For a modest 10×10 lattice with 100 electrons: $\binom{200}{100} \approx 10^{58}$. No classical computer can store or diagonalise a matrix of this size.

Density functional theory (DFT) — the workhorse of solid-state physics — approximates the many-electron problem by working with the electron density rather than the full wavefunction (the same approach we met in Unit 3). It's a mean-field method: it handles weakly correlated materials well but systematically fails for strongly correlated systems.

Quantum Monte Carlo (QMC) — the best classical method for many-body quantum systems — works for bosons but suffers the **sign problem** for fermions: the antisymmetry of the fermionic wavefunction (the same minus sign that requires fermion-to-qubit encodings) causes Monte Carlo sampling to produce catastrophic cancellations, making the calculation exponentially expensive at low temperatures. Troyer and Wiese (2005) proved the sign problem is NP-hard in general.

Exact diagonalisation — exact but limited to ~20 sites. Physically relevant results require hundreds of sites.

DMRG (Density Matrix Renormalization Group) — an approach that represents the wavefunction as a chain of tensors (a **matrix product state**), capturing correlations efficiently in one dimension. Excellent for 1D systems, but the entanglement structure of 2D strongly correlated systems exceeds what matrix product states can represent.

The 2D Hubbard model at intermediate U/t is in a computational no-man's land. No classical method can solve it reliably.

The Quantum Angle

Quantum Phase Estimation (QPE)

In Unit 3 (Drug Discovery), we used VQE; a variational method that gives energy upper bounds. VQE is a NISQ algorithm: shallow circuits, resilient to some noise, but limited by measurement overhead and ansatz quality.

Quantum Phase Estimation (QPE) is the fault-tolerant alternative. It gives the exact energy eigenvalue (to arbitrary precision) without variational optimisation or measurement overhead. The tradeoff: it requires much deeper circuits and full error correction.

QPE works like this:

1. **Prepare a trial state** $|\psi\rangle$ with non-zero overlap with the ground state
2. **Apply controlled time evolution** e^{-iHt} to the trial state, controlled by an ancilla register
3. **Apply the inverse QFT** to the ancilla register
4. **Measure** the ancilla → the result is the energy eigenvalue (encoded as a binary fraction)

The key ingredient is step 2: implementing e^{-iHt}; the time evolution operator under the Hubbard Hamiltonian; as a quantum circuit.

Hamiltonian simulation via Trotterisation

The Hamiltonian $H = H_{\text{hop}} + H_{\text{int}}$ is a sum of non-commuting terms. We can't implement e^{-iHt} directly, but we can approximate it using the **Trotter-Suzuki decomposition**:

$$e^{-iHt} \approx \left(e^{-iH_{\text{hop}}\Delta t} \cdot e^{-iH_{\text{int}}\Delta t}\right)^{t/\Delta t}$$

Each factor ($e^{-iH_{\text{hop}}\Delta t}$ and $e^{-iH_{\text{int}}\Delta t}$) is easy to implement as a circuit because the individual terms within each part do commute. The error is $O(\Delta t^2)$ per step, so $O(t \cdot \Delta t)$ total. Use more Trotter steps (smaller Δt) for better accuracy; at the cost of deeper circuits.

Higher-order Trotter formulas reduce the error further. The encodings book (Chapter 15, Trotter Formulas) covers this in full detail.

Resource estimation: how big does the quantum computer need to be?

This is the critical question. Babbush et al. (2018) estimated the resources for QPE on the 2D Hubbard model:

Lattice size	Qubits needed	T-gates	Estimated wall time
4×4	~100 logical	10^8	Minutes
8×8	~400 logical	10^{11}	Hours
16×16	~1,600 logical	10^{14}	Days

(A **T-gate** is a specific rotation gate that, together with the Clifford gates, enables universal quantum computation. T-gates are the most expensive operation in fault-tolerant computing because they require resource-intensive distillation procedures, so the T-gate count is the standard cost metric.)

With **surface code** error correction (the leading scheme, which encodes one logical qubit using ~1,000 physical qubits arranged on a 2D grid, at a physical error rate of 10^{-3}), the 8×8 lattice needs about 400,000 physical qubits — well beyond current hardware, but within the scope of what might be built in 10–15 years.

The Pinnacle architecture (Unit 2) could dramatically reduce these numbers if quantum LDPC codes prove applicable to Hamiltonian simulation.

Worked Example

Ground state of the **2-site Hubbard model** via QPE. Sweep the U/t ratio from 0 (free electrons) to 10 (strongly interacting) and observe the **Mott insulator transition**.

The 2-site Hubbard model with 2 electrons has a 6-dimensional Hilbert space, representable with 4 qubits after Jordan-Wigner encoding. Small enough to run, large enough to show real physics:

- At $U/t = 0$: ground state is delocalised (metallic)
- At $U/t \gg 1$: electrons localise, one per site (insulating)
- The transition between them is visible in the ground-state energy curve

→ **See notebook `07-materials-science.ipynb` for the runnable version.**

→ Want to understand the algorithm in detail? Read the next chapter.

Reality Check

The 2D Hubbard model is the "grand challenge" of quantum simulation. It has been called the "standard model of condensed matter physics"; the simplest model that might explain high-temperature superconductivity in cuprate materials. Solving it is widely regarded as the problem most likely to yield the first practical quantum advantage in simulation.

What's been demonstrated. Small Hubbard models (2–4 sites) have been simulated on quantum hardware using VQE. Google's 2020 experiment on a 2×2 plaquette was a notable milestone. But these sizes are trivially classical; the value was in demonstrating the pipeline, not in producing new physics.

The VQE stopgap. Until fault-tolerant QPE is available, VQE on the Hubbard model is an active area of research. Problem-specific ansätze (like the Hamiltonian variational ansatz) can capture Hubbard physics better than generic hardware-efficient circuits. But VQE is fundamentally limited by measurement overhead and barren plateaus at scale.

What would change the picture. A fault-tolerant quantum computer capable of running QPE on a 10×10 Hubbard lattice. This would produce new physics; results that cannot be obtained any other way. The resource estimates suggest $\sim 10^5$ physical qubits with error rates $< 10^{-3}$. This is the most concrete milestone for "quantum utility" in simulation.

Chef's Notes

- **QPE = QFT + controlled time evolution.** QPE was previewed in Unit 2 (Shor uses it for period-finding). Here it appears in its natural habitat: extracting energy eigenvalues from a physical Hamiltonian. The QFT, phase kickback, and controlled unitaries are the same machinery; applied to a different problem.
- **Trotterisation bridges Units 3, 7, and 8.** In Unit 3 (VQE), we prepared trial states and measured energy variationally. Here and in Unit 8, we implement actual time evolution (e^{-iHt}), which requires decomposing the Hamiltonian into a Trotter product. The encodings book (Chapters 14–16) covers this in detail.
- **The sign problem is deep.** The reason QMC fails for fermions is intimately connected to the antisymmetry of the fermionic wavefunction; the same physics that requires fermion-to-qubit encodings. Quantum computers bypass the sign problem by representing the fermionic state directly (via encodings), rather than sampling from it.
- **Further reading:**
  - Hubbard (1963). Electron correlations in narrow energy bands. Proceedings of the Royal Society A 276:238–257
  - Babbush, Wiebe, McClean, et al. (2018). Low-Depth Quantum Simulation of Materials. Physical Review X 8:011044 (arXiv:1711.04789)
  - Troyer and Wiese (2005). Computational Complexity and Fundamental Limitations to Fermionic Quantum Monte Carlo Simulations. Physical Review Letters 94:170201 (arXiv:cond-mat/0407066)
  - Qin, Schäfer, Andergassen, Corboz, Gull (2022). The Hubbard Model: A Computational Perspective. Annual Review of Condensed Matter Physics 13:275–302 (arXiv:2103.12097)

Deep-Dive 7: QPE and Trotterisation

This deep dive pairs with Unit 7 (Materials Science), which explained why the Hubbard model defeats classical methods and how QPE could solve it. Here we build QPE and the Trotter time-evolution circuit from the components introduced in earlier deep dives.

In This Chapter

- **What you'll learn:** How QPE extracts exact energy eigenvalues, how Trotterisation approximates time evolution as a circuit, and how to estimate the resources needed for real materials simulation.
- **What you need:** From Deep-Dive 2 (Shor), you know the QFT and phase kickback. From Deep-Dive 1 (QAOA), you know the ZZ gate and CNOT sandwich. Here we combine them for Hamiltonian simulation.
- **Runnable version:** The companion notebook `07-materials-science.ipynb` runs QPE on a 2-site Hubbard model on a cloud Quokka.

QPE: from phases to eigenvalues

Where we are

Let's take stock of what we already have. From Deep-Dive 1, we know how to build cost Hamiltonians from ZZ gates and encode optimisation problems as quantum operators. From Deep-Dive 2, we know how to extract a hidden number from the phase of a quantum state — that's what the controlled-powers-plus-inverse-QFT pattern did for Shor's algorithm. From Deep-Dive 3, we know how to encode a molecular Hamiltonian as a sum of Pauli operators.

QPE puts these together. The question it answers: given a physical system described by a Hamiltonian H, what are its energy levels?

Here's the idea in words before we touch a formula. If you let a quantum system evolve under its Hamiltonian H for time t, each energy eigenstate picks up a phase proportional to its energy — states with higher energy rotate faster. QPE measures that rotation speed. The QFT (from Deep-Dive 2) converts the rotation into a binary number you can read out. Phase kickback (from Deep-Dive 2) is the mechanism that transfers the phase information from the system to the measurement qubits.

That's it. Same tools, new application. Let's see how they fit together.

The setup

QPE involves two registers. The **system register** holds the quantum state whose energy we want to measure. The **ancilla register** — m helper qubits used as a measurement instrument — will hold the answer.

If the system is in an eigenstate $|$
psi
$rangle$ of the Hamiltonian with energy E, then time evolution gives $e^{-iHt}|$

psi
$rangle = e^{-iEt}|$
psi
$rangle$. The phase
$phi = -Et/(2$
$pi)$ encodes the energy. QPE extracts
phi.

The circuit

The circuit has four steps — all of which reuse pieces you've seen before:

1. **Hadamard** all ancillas → superposition
2. **Controlled-U^{2^k}** for each ancilla qubit k → phase accumulation
3. **Inverse QFT** on the ancillas → converts phases to a binary number
4. **Measure** the ancillas → read out ϕ

Steps 1, 3, and 4 we've seen before. The inverse QFT is the QFT from Deep-Dive 2, run backwards (swap the order of gates, negate the rotation angles). Step 2 is the new ingredient.

Controlled-U^{2^k}

Ancilla qubit k controls the application of U^{2^k} to the system register. If the system is in eigenstate $|\psi\rangle$:

$$\text{c-}U^{2^k}\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)|\psi\rangle = \frac{|0\rangle + e^{2\pi i \cdot 2^k \phi}|1\rangle}{\sqrt{2}}|\psi\rangle$$

The ancilla picks up a phase proportional to $2^k\phi$. After all m controlled operations, the ancilla register holds:

$$\frac{1}{\sqrt{2^m}}\sum_{k=0}^{2^m-1} e^{2\pi i k\phi}|k\rangle$$

This is exactly the QFT of $|\phi\rangle$. Applying the inverse QFT yields ϕ (or its best m-bit approximation).

> **Connection to Shor:** In Deep-Dive 2, QPE was implicit — the controlled modular exponentiation + QFT was QPE applied to the modular multiplication operator. Here we make it explicit and apply it to a physically motivated Hamiltonian.

Trotterisation: implementing e^{-iHt}

The problem

QPE requires controlled applications of $U = e^{-iHt}$. But H is a sum of terms that don't commute:

$$H = H_1 + H_2 + \cdots + H_L$$

We can implement $e^{-iH_k t}$ for each individual term (because each term has a simple structure; products of Pauli operators). But $e^{-iHt} \neq e^{-iH_1 t} \cdot e^{-iH_2 t} \cdots e^{-iH_L t}$ because the terms don't commute.

First-order Trotter

The **Trotter formula** says: the product of exponentials approximates the exponential of the sum if the time step is small:

$$e^{-iHt} \approx \left(e^{-iH_1 \Delta t} \cdot e^{-iH_2 \Delta t} \cdots e^{-iH_L \Delta t}\right)^N$$

where $\Delta t = t/N$. The error is $O(L^2 t^2 \Delta t)$; it goes to zero as $\Delta t \to 0$ (more Trotter steps).

Second-order Trotter (Suzuki)

A better approximation symmetrises the product:

$$e^{-iHt} \approx \left(e^{-iH_1 \Delta t/2} \cdot e^{-iH_2 \Delta t/2} \cdots e^{-iH_L \Delta t} \cdots e^{-iH_2 \Delta t/2} \cdot e^{-iH_1 \Delta t/2}\right)^N$$

The error drops to $O(L^3 t^3 \Delta t^2)$; much better for the same circuit depth. Higher-order formulas exist but add circuit complexity.

Implementing each term

For the Hubbard model:

Hopping terms $c^\dagger_{i\sigma} c_{j\sigma} + \text{h.c.}$: after Jordan-Wigner encoding (Deep-Dive 3), these become $\frac{1}{2}(X_i X_j + Y_i Y_j) \cdot Z_{\text{string}}$. The time evolution under this term is:

$$e^{-i\theta(X_i X_j + Y_i Y_j)/2}$$

This requires 2 CNOTs mixed with single-qubit rotations — a known decomposition that generalises the ZZ sandwich from Deep-Dive 1.

Interaction terms $n_{i\uparrow}n_{i\downarrow}$: after encoding, each becomes $\frac{(1-Z_{i\uparrow})(1-Z_{i\downarrow})}{4}$. This is diagonal; the time evolution is a ZZ phase gate, exactly the CNOT sandwich from Deep-Dive 1:

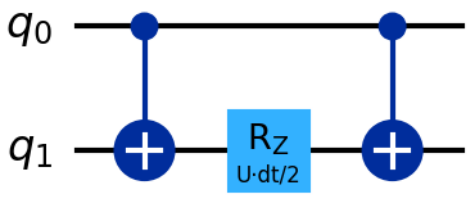

Figure 21. Trotter ZZ interaction term: CNOT–Rz(U·dt/2)–CNOT

Every piece of the Trotter circuit is built from gates we already know.

Resource estimation

Gate count

For an $L \times L$ Hubbard lattice with $2L^2$ spin-orbitals:

Component	Gate count
Hopping terms per Trotter step	$O(L^2)$ CNOTs
Interaction terms per Trotter step	$O(L^2)$ CNOTs
QFT (for QPE)	$O(m^2)$ controlled rotations
Total	$O(m \cdot N_{\text{Trotter}} \cdot L^2)$

For a 10×10 lattice with **chemical-precision** QPE ($m \approx 20$ bits, giving ~1 milli-Hartree accuracy — the threshold from Unit 3 where energy differences become chemically meaningful, $N_{\text{Trotter}} \approx 10^3$): roughly 10^7 gates on ~ 200 qubits.

Physical qubits

With **surface code** error correction (the leading scheme from Unit 2's Reality Check; physical error rate 10^{-3}, **code distance** ~20 — the code distance controls how many physical errors can be corrected, with each logical qubit requiring roughly $2d^2 \approx 800$ physical qubits for distance $d = 20$): The 10×10 Hubbard model needs:

$$200 \text{ logical} \times 800 \text{ physical/logical} \approx 160{,}000 \text{ physical qubits}$$

This is in the same ballpark as the Pinnacle architecture's estimate for RSA-2048 (Unit 2: 100,000 physical qubits). Both are ambitious but plausible targets for the next decade of hardware development.

What you should take away

1. **QPE = controlled time evolution + inverse QFT.** It extracts energy eigenvalues to arbitrary precision. The QFT (from Deep-Dive 2) and phase kickback (from Deep-Dive 2) do the heavy lifting.
2. **Trotterisation is the bridge between Hamiltonians and circuits.** It breaks e^{-iHt} into a product of simple operations — each implemented with the gates from Deep-Dive 1 (ZZ sandwich for diagonal terms) and generalisations for off-diagonal terms.
3. **The circuit is deep but structured.** Every gate in a Trotter circuit has a physical meaning: it simulates one interaction in the Hamiltonian for one time step. More accuracy → more Trotter steps → deeper circuit.
4. **Resource estimates are concrete.** For the 2D Hubbard model: ~200 qubits, ~10^7 gates, ~10^5 physical qubits with error correction. These numbers define the engineering targets.
5. **Everything connects.** QPE reuses the QFT from Deep-Dive 2 (Shor). Trotterisation reuses the ZZ gate from Deep-Dive 1 (QAOA). The fermionic encoding comes from Deep-Dive 3 (VQE). This is where the threads converge.

Beyond Trotter: qubitization

Trotterisation is not the only way to implement e^{-iHt} for QPE. A fundamentally different approach called **qubitization** (Low and Chuang, 2019) avoids Trotter error entirely by encoding the Hamiltonian as a block of a larger unitary — a technique called a **block encoding**. Instead of approximating e^{-iHt} with a product of simple gates (introducing Trotter error that must be controlled), qubitization constructs a unitary operator whose eigenvalues are exact functions of the Hamiltonian's eigenvalues.

The practical consequence: qubitization-based algorithms like **quantum signal processing** (QSP) and **quantum singular value transformation** (QSVT) achieve optimal query complexity — they use the minimum number of oracle calls provably necessary to estimate eigenvalues to a given precision. For Hamiltonian simulation, this means asymptotically fewer gates than any Trotter formula.

The tradeoff: qubitization requires more ancilla qubits and a more complex circuit structure (the block encoding of H). For near-term and early fault-tolerant devices, Trotterisation may still be more practical due to its simpler circuits. But for the long-term fault-tolerant regime, qubitization is the theoretically optimal approach.

- Low and Chuang (2019). Hamiltonian Simulation by Qubitization. Quantum 3:163 (arXiv:1610.06546)
- Gilyen, Su, Low, Wiebe (2019). Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. STOC 2019 (arXiv:1806.01838)

Unit 8: Climate & Energy

Designing a Better Catalyst for Carbon Capture

The Hook

In 2024, the concentration of CO_2 in Earth's atmosphere passed 427 parts per million; the highest in at least 4 million years. To limit global warming to 1.5°C, we need to remove approximately **10 gigatonnes of CO_2 per year** from the atmosphere by 2050, on top of drastic emissions reductions. Current direct air capture (DAC) technology removes about 10,000 tonnes per year. The gap is six orders of magnitude.

The bottleneck isn't engineering; it's chemistry. Every DAC process depends on a **catalyst** or **sorbent** that captures CO_2 molecules from air. The best current sorbents are expensive, degrade quickly, and require too much energy to regenerate. Better catalysts exist in principle; the chemical space of possible materials is vast; but finding them requires predicting **reaction pathways** on catalyst surfaces: which molecules bind, how strongly, through what transition states, with what activation energy.

This is a quantum chemistry problem. The catalyst's active site; the few atoms where the CO_2 molecule actually binds and reacts; involves **strongly correlated electrons** (just like the Hubbard model in Unit 7) embedded in a much larger environment (the metal surface, the solvent, the substrate). You need quantum-level accuracy for the active site and classical efficiency for everything else.

No classical method handles both simultaneously. DFT is fast but inaccurate for strongly correlated active sites. **CCSD(T)** (the "gold standard" classical method from Unit 3, scaling as $O(N^7)$) is accurate but can't handle the full environment. **Full CI** (exact but exponentially expensive) is impossible at scale.

This is where the quantum computing story comes together.

The Bottleneck

Surface chemistry: the accuracy-size tradeoff

A realistic catalyst simulation involves three scales:

1. **The active site** (~10–50 atoms, involving transition metals with partially filled **d-orbitals** — the outermost electron orbitals of metals like iron and copper, which give these atoms their complex chemistry): strongly correlated, requires quantum accuracy
2. **The local environment** (~50–200 atoms of the support material and nearby solvent): weakly correlated, treatable classically
3. **The bulk** (the rest of the material): can be described by a mean-field or continuum model

The challenge is that the active site's electronic structure depends on the environment, and the environment's response depends on the active site. They're coupled.

Classical methods pick one scale and approximate the others: - **DFT**: handles everything at the same (insufficient) level of accuracy - **Multiscale QM/MM**: use quantum mechanics (QM) for the active site and classical molecular mechanics (MM, a ball-and-spring model) for the environment — but the QM part is still classical DFT, which fails for strong correlation - **Embedded CCSD(T)**: classical high-accuracy method for the active site, DFT for the environment — better, but CCSD(T) still scales as $O(N^7)$ and breaks down for **open-shell** systems (molecules with unpaired electrons) and **multi-reference** states (systems where no single electron arrangement dominates — multiple configurations contribute equally to the wavefunction), which are common in catalysis

What we need: a method that gives quantum-accurate results for the active site while scaling efficiently with the total system size.

The Quantum Angle

Quantum embedding: the best of both worlds

The solution is **quantum embedding**; use a quantum computer for the hard part (the active site) and a classical computer for the easy part (the environment). This is not a compromise; it's the computationally optimal strategy.

The mathematical framework:

1. **Define the active space**: identify the orbitals (typically 20–50) where strong correlation lives; the partially filled d-orbitals of the transition metal, the π-orbitals of the reacting molecule
2. **Embed**: use a classical method (DFT or Hartree-Fock) to compute the effect of the environment on the active space, producing an **effective Hamiltonian** that includes the environment's influence as a potential
3. **Solve the active space quantumly**: run VQE (Unit 3) or QPE (Unit 7) on the effective Hamiltonian; a much smaller problem than the full system
4. **Self-consistently update**: the active-space solution modifies the environment, which modifies the effective Hamiltonian, iterate until convergence

This is known as **DMET** (Density Matrix Embedding Theory) or **active-space VQE/QPE**, depending on the specific formulation.

The full pipeline

This unit is the culmination of the book. The complete quantum simulation pipeline draws on everything we've built:

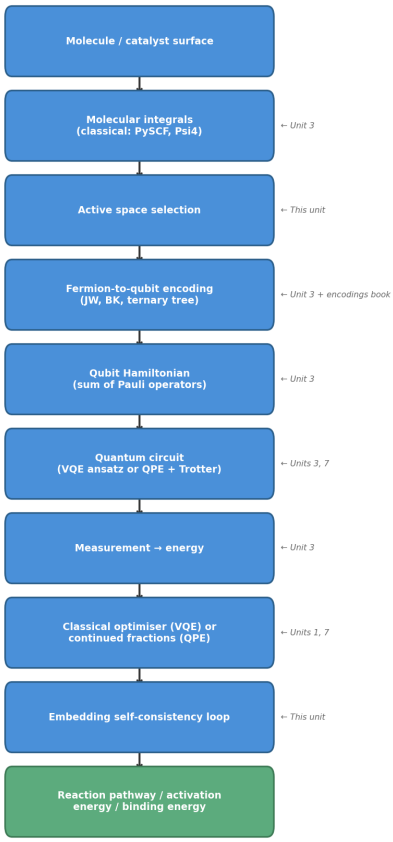

Figure 22. The complete quantum simulation pipeline for catalyst modelling, with cross-references to earlier units

Every step in this pipeline has been introduced in an earlier unit. This chapter shows how they connect into a single, end-to-end workflow.

Why the active space matters

The active space is where the computational savings come from; and where the physical insight enters.

For CO_2 capture on a metal oxide surface, the active site might be an iron or copper atom with its nearest-neighbour oxygen atoms; perhaps 6 metal d-orbitals and 10 ligand orbitals: 16 active orbitals. After encoding and tapering, this requires ~12–16 qubits and circuits of moderate depth; feasible on near-term or early fault-tolerant hardware.

The key point: **you don't need to simulate the whole surface.** The quantum computer handles the 16 orbitals where strong correlation matters. The classical computer handles the other 500 orbitals where mean-field methods are perfectly adequate. This division of labour is what makes quantum-classical embedding practical.

Worked Example

Active-space VQE for **CO adsorption on a small iron cluster** (a simplified model of an iron oxide catalyst surface).

Setup: - Fe_2O_3 cluster with CO adsorbate - Active space: 6 Fe d-orbitals + 4 CO **π/π*** orbitals (the bonding and antibonding orbitals formed by sideways overlap of atomic p-orbitals) = 10 active orbitals, 10 active electrons - After Jordan-Wigner encoding: 20 qubits (or fewer with tapering) - Environment: remaining orbitals treated with DFT embedding

The notebook computes the binding energy of CO on the cluster using: 1. Full DFT (fast, inaccurate for the binding site) 2. Active-space VQE (quantum accuracy for the binding site, DFT for the rest) 3. Exact active-space diagonalisation (classical benchmark)

The difference between DFT and active-space results is the **correlation energy** in the active site; the part that classical methods get wrong and quantum computers get right.

→ The next chapter builds the quantum embedding pipeline from scratch, and shows you the code.

→ Want to understand the algorithm in detail? Read the next chapter.

Reality Check

Microsoft and PNNL's nitrogen fixation estimate. In 2022, Microsoft and Pacific Northwest National Laboratory published resource estimates for simulating the FeMo cofactor of nitrogenase; the enzyme that fixes atmospheric nitrogen. This is a biologically critical catalyst with a strongly correlated active site. Their estimate: ~4 million physical qubits for a useful simulation using QPE with surface code error correction. With the Pinnacle architecture (Unit 2), this could potentially drop to $\sim 200,000$ physical qubits.

What's been demonstrated. Active-space VQE has been demonstrated for small molecules (H_2, LiH, BeH_2) on real hardware. No catalyst system has been simulated quantumly at a scale that produces new chemistry; the gap between "toy demonstration" and "useful catalyst screening" remains large.

The classical competition. DMRG (Density Matrix Renormalization Group) has made remarkable progress on catalyst active sites in recent years. Chan and co-workers have applied DMRG to problems with up to ~100 active orbitals in 1D-like geometries. For 2D active sites (common in surface catalysis), DMRG struggles; this is where quantum advantage is most likely.

The timeline. Useful quantum catalyst screening requires ~50 active orbitals → ~100 qubits (after encoding and tapering) → ~10^5 physical qubits with error correction. This is the same order of magnitude as the Hubbard model estimates from Unit 7. A generous estimate: 10–15 years.

What's real today. The pipeline is real; define active space, compute integrals, encode, optimize. The quantum chemistry software (PySCF, OpenFermion, the encodings library) exists and works. What's missing is a quantum computer large enough and quiet enough to run the circuits.

Chef's Notes

- **This chapter is the capstone.** Every concept introduced in the book; qubits as binary variables (Unit 1), superposition and interference (Unit 2), fermions and encodings (Unit 3), variational methods (Units 1, 3), amplitude estimation (Unit 5), QPE and Trotterisation (Unit 7); comes together here in a single end-to-end pipeline.
- **Active-space methods are the near-term strategy for quantum chemistry.** They reduce the quantum resource requirements by orders of magnitude compared to full-system simulation. The classical insight (where does strong correlation live?) combines with the quantum capability (solve the correlated subsystem exactly).
- **The encodings book covers the encoding step in depth.** This unit's pipeline includes "encode the active-space Hamiltonian as a qubit operator"; the step where From Molecules to Qubits begins. Readers who want to understand why Jordan-Wigner produces long operator strings, or how tapering reduces qubit count, should start there.
- **Catalyst design is a killer app for quantum computing.** It's economically critical (carbon capture, fertiliser production, hydrogen fuel cells), scientifically well-defined (compute the activation energy of a reaction pathway), and classically intractable at the relevant accuracy. If quantum computers prove their worth anywhere in chemistry, it's here.
- **Further reading:**
  - Reiher, Wiebe, Svore, Wecker, Troyer (2017). Elucidating Reaction Mechanisms on Quantum Computers. PNAS 114:7555–7560 (arXiv:1704.05018)
  - Lee, Berry, Gidney, et al. (2021). Even More Efficient Quantum Computations of Chemistry Through Tensor Hypercontraction. PRX Quantum 2:030305 (arXiv:2012.09265)
  - Li, Otten, Isaev, et al. (2022). Toward Practical Quantum Embedding Simulation of Realistic Chemical Systems on Near-Term Quantum Computers. Chemical Science 13:8953–8962
  - Azariah et al. From Molecules to Qubits. github.com/johnazariah/encodings-book

Deep-Dive 8: Quantum Embedding Methods

This deep dive pairs with Unit 8 (Climate & Energy), which explained why catalyst design requires quantum accuracy for the active site and classical efficiency for the environment. Here we show how to combine both.

In This Chapter

- **What you'll learn:** How to select an active space, embed it in a classical environment, solve the active space with VQE or QPE, and iterate to self-consistency.
- **What you need:** This is the capstone deep dive. It draws on VQE (Deep-Dive 3), QPE (Deep-Dive 7), and fermion-to-qubit encodings (Deep-Dive 3). If you've followed the deep dive path, you have everything you need.
- **Runnable version:** The companion notebook `08-climate-energy.ipynb` demonstrates active-space VQE on a cloud Quokka.

The active space idea

Why you don't simulate the whole molecule

A catalyst system has hundreds of orbitals; carbon scaffolding, solvent molecules, bulk metal. But the chemistry happens at just a few: the metal d-orbitals where CO binds, the π* orbitals where electrons transfer.

Strong correlation; the kind that defeats classical methods; lives in these few orbitals. The rest are either doubly occupied (core electrons: inert, boring) or empty (high-energy virtual orbitals: irrelevant at normal temperatures). Classical methods handle these "boring" orbitals perfectly well.

The strategy: **use a quantum computer only for the hard part.** Select the 10–50 orbitals where classical methods fail (the "active space"), and solve those exactly with VQE (Deep-Dive 3) or QPE (Deep-Dive 7). Let a classical computer handle the other 450 orbitals with standard DFT or Hartree-Fock.

How to choose the active space

This is part science, part art:

- **The metal d-orbitals.** Transition metals (Fe, Cu, Mn, Co) have partially filled d-shells. These orbitals are close in energy and strongly coupled; the definition of strong correlation.
- **The adsorbate frontier orbitals.** For CO_2 capture: the CO_2 π^* orbital that accepts electrons from the metal.
- **Natural orbital analysis.** Compute approximate natural orbital occupation numbers from a cheap classical calculation (CASSCF — Complete Active Space Self-Consistent Field, a classical method that optimises both orbitals and electron configurations within a small active space — or MP2, the perturbative correction from Unit 3's hierarchy). Orbitals with occupations significantly different from 0 or 2 are strongly correlated; include them.

- **Entropy-based selection.** Compute the single-orbital entropy from an approximate wavefunction. High-entropy orbitals are strongly correlated.

For a typical catalyst active site: 6 metal d-orbitals + 4–8 ligand orbitals = 10–14 active orbitals. After Jordan-Wigner encoding (Deep-Dive 3): 20–28 qubits. After tapering (using point-group symmetry and particle-number conservation to reduce qubits): possibly 12–20 qubits. Feasible on near-term hardware.

DMET: Density Matrix Embedding Theory

The mathematical framework

DMET (Knizia and Chan, 2012) is a clean embedding approach. It works by recognising that the environment's effect on the fragment can be captured by a small **bath**; a set of orbitals that encode the entanglement between the fragment and the rest of the system.

The procedure:

1. **Solve the full system cheaply.** Run Hartree-Fock on all 500 orbitals. This gives a mean-field **density matrix** ρ_{MF} — a mathematical object that describes the quantum state of a system in terms of probabilities and correlations (a generalisation of the outer product $|\psi\rangle\langle\psi|$ from Deep-Dive 5, extended to handle statistical mixtures of states).
2. **Build the bath.** From ρ_{MF}, extract the entanglement between the fragment (active site) and the environment. The bath has the same number of orbitals as the fragment; so a 10-orbital fragment gets a 10-orbital bath. The 20-orbital embedded problem captures the fragment-environment entanglement exactly (at the mean-field level).
3. **Build the embedded Hamiltonian.** Project the full Hamiltonian onto the fragment + bath space. The result is a small Hamiltonian; 20 orbitals instead of 500; that includes the environment's effect as an effective potential.
4. **Solve the embedded problem.** Run VQE (Deep-Dive 3) or QPE (Deep-Dive 7) on the 20-orbital embedded Hamiltonian. This gives the exact energy and density matrix for the fragment, in the presence of the environment.
5. **Self-consistency.** Update the mean-field solution with the quantum result. The new mean-field changes the bath, which changes the embedded Hamiltonian, which changes the quantum solution. Iterate until convergence (typically 5–10 cycles).

What the quantum computer does

The quantum computer appears exactly once per self-consistency cycle: step 4. Everything else is classical linear algebra. The quantum computer solves a small, strongly correlated problem; the fragment + bath; that classical methods can't handle.

The pipeline is solver-agnostic. Today, use VQE on NISQ hardware. Tomorrow, use QPE on fault-tolerant hardware. The embedding framework doesn't change; only the quantum subroutine.

The complete pipeline

This chapter ties together every deep dive in the book:

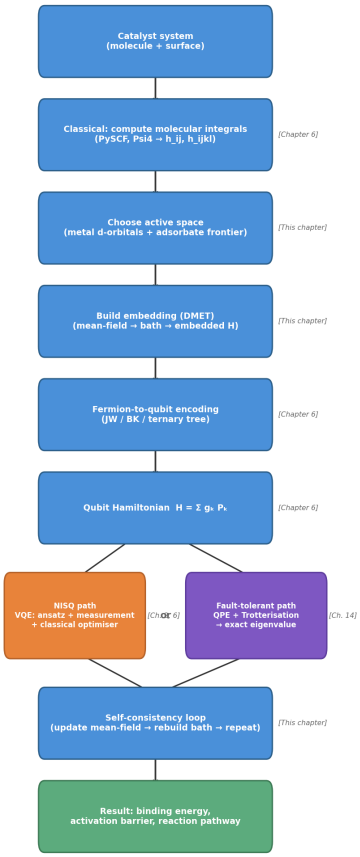

Figure 23. The complete quantum-classical embedding pipeline, showing NISQ and fault-tolerant paths with chapter cross-references

Every box references a chapter where the concept was introduced. A reader who followed the deep dive path (Chapters 2 → 4 → 6 → 8 → 10 → 12 → 14 → 16) has the tools to understand every step. A reader who skipped the deep dives can see the pipeline and know where to look for details.

The companion notebook demonstrates active-space VQE for a simplified catalyst model, running the full embed → solve → iterate pipeline.

→ **See notebook `08-climate-energy.ipynb` for the runnable version.**

What you should take away

1. **The active space is the key insight.** You don't need a quantum computer for the whole molecule; just for the 10–50 orbitals where classical methods fail. This reduces the quantum resource requirement by orders of magnitude.

2. **DMET provides the framework.** It tells you how to build the bath, how to embed the fragment, and how to iterate to self-consistency. The quantum computer plugs in as the solver for the embedded problem.
3. **Everything connects.** The qubit Hamiltonian comes from Deep-Dive 3 (VQE pipeline). The solver is either VQE (Deep-Dive 3) or QPE (Deep-Dive 7). The ZZ gates are from Deep-Dive 1 (QAOA). The QFT is from Deep-Dive 2 (Shor). Phase kickback drives QPE. The cost Hamiltonian pattern from Unit 1 reappears as the molecular Hamiltonian.
4. **This is the pipeline for quantum utility in chemistry.** Not toy demonstrations on H_2, but real catalyst screening on industrially relevant systems. The engineering target: 50 active orbitals → 100 qubits → 10^5 physical qubits with error correction. Ambitious but concrete.
5. **For the encoding step,** see From Molecules to Qubits. That book covers exactly the transformation at the heart of this pipeline.

The Elephant in the Room: Quantum Error Correction

Every Reality Check in this book has ended at the same wall: the hardware isn't ready yet. The algorithms work. The mathematics is settled. But running them on real quantum computers — at the scale where they'd actually beat classical methods — requires something we don't yet have: **fault-tolerant quantum computing**.

This chapter explains why, and what it will take to get there.

Why quantum computers make mistakes

A classical computer stores a bit as a voltage: high or low. The voltage can drift, but as long as it stays on the right side of the threshold, the bit is correct. Digital error correction is essentially free — every logic gate refreshes the signal.

Qubits are different. A qubit's state is a point on the Bloch sphere — a continuous value, not a discrete one. Any interaction with the environment (stray electromagnetic fields, thermal vibrations, cosmic rays) nudges the state slightly. This is **decoherence**: the qubit's quantum information leaks into the environment, turning a precise superposition into a noisy mess.

Gate operations aren't perfect either. Every gate applied to a qubit introduces a small rotation error. On current hardware, the error rate per gate is roughly 10^{-3} — one mistake per thousand operations. That sounds small, but Shor's algorithm for RSA-2048 requires $\sim 10^{10}$ gates. With a 10^{-3} error rate, the computation would drown in noise after a few thousand gates.

The core problem: **quantum information is fragile, and the computations we want to run are deep.**

The basic idea: redundancy without looking

Classical error correction is straightforward: make copies. Store three copies of each bit; if one flips, take the majority vote. Simple, effective, and used everywhere from hard drives to deep-space communication.

Quantum error correction can't copy. The **no-cloning theorem** says you cannot make an exact copy of an unknown quantum state. And you can't just check a qubit's value to see if it's correct — measuring it destroys the superposition.

So quantum error correction uses a different kind of redundancy. Instead of copying the state, you **spread it across many physical qubits** in a way that lets you detect and correct errors without ever learning what the state is.

A **logical qubit** — one error-protected unit of quantum information — is encoded in many **physical qubits**. The encoding is designed so that small errors (a single physical qubit flipping or drifting) produce detectable syndromes: patterns in the correlations between physical qubits that reveal which error occurred, without revealing the encoded state itself.

Measure the syndromes. Identify the error. Apply a correction. The logical qubit is restored — and you never learned what state it was in. This is the trick that makes quantum error correction possible despite the no-cloning theorem and the fragility of measurement.

Surface codes: the leading approach

The most developed error-correction scheme is the **surface code**. Physical qubits are arranged on a 2D grid. Each logical qubit is encoded in a patch of $d \times d$ physical qubits, where d is the **code distance** — roughly, the number of physical errors that must occur simultaneously to corrupt the logical qubit.

Key properties:

- **Threshold**: if the physical error rate is below ~1%, the surface code can correct errors faster than they accumulate. Current hardware is near or just below this threshold.
- **Overhead**: each logical qubit requires $\sim 2d^2$ physical qubits. For $d = 20$ (enough for many algorithms): ~ 800 physical qubits per logical qubit.
- **Locality**: syndrome measurements involve only nearest-neighbour interactions on the 2D grid — a good match for superconducting and ion-trap hardware.

The cost: a computation that needs 200 logical qubits (like QPE on the Hubbard model, Unit 7) requires $\sim 160{,}000$ physical qubits with surface codes.

LDPC codes: doing more with less

Quantum LDPC codes (Low-Density Parity-Check codes) are a newer family that encode more logical qubits per physical qubit than surface codes. The Pinnacle architecture (Unit 2's Reality Check) uses LDPC codes to reduce the cost of breaking RSA-2048 from 20 million physical qubits to under 100,000 — a 200× improvement.

The tradeoff: LDPC codes require longer-range connectivity between qubits (not just nearest neighbours), which is harder to build in hardware. Whether current architectures can support LDPC codes at scale is an active engineering challenge.

Error mitigation: the NISQ bridge

Fault-tolerant quantum computing with full error correction is a future technology. Today's machines — the **NISQ** (Noisy Intermediate-Scale Quantum) era — have too few qubits and too much noise for surface codes.

Error mitigation is the pragmatic alternative: classical post-processing techniques that reduce the impact of noise without the full overhead of error correction. Key approaches:

- **Zero-noise extrapolation**: run the circuit at several artificially increased noise levels, then extrapolate back to zero noise.

- **Probabilistic error cancellation**: decompose noisy gates into combinations of ideal gates, using randomness and classical post-processing to cancel errors statistically.
- **Symmetry verification**: discard measurement results that violate known symmetries of the problem (like electron number conservation in chemistry).

Error mitigation extends the useful depth of NISQ circuits by a modest factor — enough for VQE on small molecules (Unit 3) and small QAOA instances (Unit 1), but not enough for Shor's algorithm or large-scale QPE.

Where we are and where we're going

Milestone	Qubits	Status
NISQ demonstrations	50–1,000 physical	**Now** (2024–2026)
First logical qubit (below break-even)	~1,000 physical → 1 logical	**Demonstrated** (Google, 2024)
Early fault-tolerant (useful QEC)	10^4–10^5 physical → 10^2 logical	**5–10 years**
Cryptographically relevant	10^5–10^6 physical → 10^3 logical	**10–15 years**
Full-scale quantum chemistry	10^5–10^6 physical → 10^3 logical	**10–15 years**

Google's 2024 result — demonstrating a logical qubit with a lower error rate than any of its constituent physical qubits — was the first time a quantum error-correcting code operated below break-even. It used a surface code with distance $d = 5$ (49 physical qubits per logical qubit). Scaling to $d = 20$ and hundreds of logical qubits is an engineering challenge, not a physics one. The path is clear; the question is how fast we walk it.

Every resource estimate in this book — 100,000 qubits for RSA-2048, 160,000 for the Hubbard model, 100,000 for quantum chemistry — assumes that this engineering succeeds. The algorithms are waiting.

What Quantum Computers Are Actually For

We started with a UPS driver and 20 stops. We end with a catalyst and 50 orbitals. Between them, eight problems — and one recurring pattern.

Every quantum algorithm in this book does the same thing: it encodes a hard problem as a quantum operator, manipulates phases through interference, and extracts the answer through measurement. The operator changes — a cost Hamiltonian in Unit 1, a modular exponentiation oracle in Unit 2, a molecular Hamiltonian in Unit 3, a Grover iterator in Unit 5, a lattice Hamiltonian in Unit 7. The interference mechanism changes — the QAOA mixer, the QFT, the VQE ansatz, Grover's diffusion operator. But the architecture is always the same three acts: encode, interfere, measure.

This is not a coincidence. It's the structure of quantum mechanics itself. Quantum systems carry information in phases. Phases interfere. Measurement collapses the interference pattern into a classical answer. Every quantum algorithm is an interference machine designed to make the right answer loud and the wrong answers quiet.

What's real, what's next, what's unknown

Real today. The algorithms work. Shor's algorithm factors numbers. VQE computes molecular energies. QAOA optimises combinatorial problems. Quantum kernels classify data. These are mathematical facts, verified on simulators and demonstrated on small quantum hardware. The question has never been whether the algorithms work — it's whether the hardware can run them at useful scale.

Next. The hardware gap is closing. Resource estimates for breaking RSA-2048 have dropped from 20 million physical qubits to under 100,000 in the space of five years. Early fault-tolerant devices — machines with a few thousand logical qubits — could arrive within a decade. The first applications to cross the utility threshold will likely be quantum chemistry (Units 3 and 8), where the problems are natively quantum and the active-space trick keeps qubit counts manageable.

Unknown. Whether QAOA will beat classical optimisation at scale. Whether quantum kernels will find natural datasets where they outperform classical methods. Whether the quadratic speedup of amplitude estimation justifies the overhead of error correction. These are open research questions, not marketing talking points. The honest answer is: we don't know yet. This book has tried to give you the tools to follow the answers as they emerge.

The one thing to remember

If you take away a single idea from this book, let it be this: quantum computing is not about parallelism. It's not about "trying all answers at once." It's about **interference** — the ability to make wrong answers cancel and right answers reinforce, using the phase structure of quantum mechanics. Every algorithm, every speedup, every advantage traces back to this one mechanism.

The problems are real. The algorithms are sound. The hardware is coming. What remains is engineering — and the quiet, persistent work of turning mathematical possibility into practical capability.

Where to go from here

If you want to build circuits. Our companion book, the Quokka Cookbook, contains runnable recipes for every algorithm in this book — QAOA, Shor's period-finding, VQE, quantum kernels, amplitude estimation, and more — designed for the Quokka cloud platform. This book told you why; the Cookbook shows you how.

If you want to understand encodings. The transformation from molecules to qubits — fermion-to-qubit encodings — is the subject of From Molecules to Qubits. It covers Jordan-Wigner, Bravyi-Kitaev, parity, ternary tree, and compact encodings, with the algebraic structure behind all of them.

If you want the full mathematical treatment. Nielsen and Chuang's Quantum Computation and Quantum Information (2000) remains the standard reference. It covers everything in this book and more, but starts from qubits rather than problems. Szabo and Ostlund's Modern Quantum Chemistry (1996) is the classical reference for the computational chemistry foundations behind Units 3, 7, and 8.

If you want to follow the research. The arXiv preprint server (arxiv.org) is where quantum computing research appears first. The annotated bibliography below is a starting point; every paper links to its arXiv version where available.

If you want to contribute. This is an open-source book. The manuscript, notebooks, and build tooling are at github.com/johnazariah/problem-down. Issues, corrections, and pull requests are welcome.

Annotated Bibliography

The references below are the most important papers cited in this book, grouped by topic. Each annotation explains what the paper contributes and why it matters. The full citation list for each unit appears in that unit's Chef's Notes.

Quantum optimisation (Units 1, 6)

- Farhi, Goldstone, Gutmann (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028. The paper that introduced QAOA — the starting point for quantum combinatorial optimisation. Short, readable, and still the best introduction to the algorithm.
- Basso, Farhi, Marwaha, Villalonga, Zhou (2021). The Quantum Approximate Optimization Algorithm at High Depth for MaxCut on Large-Girth Regular Graphs and the Sherrington-Kirkpatrick Model. arXiv:2110.14206. Showed how to compute exact QAOA performance at high depth via tree tensor networks — the framework that enabled precise comparison with other algorithms.
- Lucas (2014). Ising formulations of many NP problems. Frontiers in Physics 2:5. The reference for encoding combinatorial problems as QUBO/Ising Hamiltonians. Covers graph problems, scheduling, satisfiability, and more.

- Farhi et al. (2001). A quantum adiabatic evolution algorithm applied to random instances of an NP-complete problem. Science 292:472–475. The foundational paper on quantum annealing as an algorithmic paradigm.

Cryptography and period-finding (Unit 2)

- Shor (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. arXiv:quant-ph/9508027. The paper that launched quantum computing as a practical field. Still worth reading for its clarity.
- Gidney and Ekerå (2021). How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. arXiv:1905.09749. The definitive resource estimate for breaking RSA with a quantum computer — the benchmark that every subsequent estimate improves on.
- Webster, Berent, Chandra, Hockings et al. (2026). The Pinnacle Architecture. arXiv:2602.11457. Reduced the RSA-2048 cost to under 100,000 physical qubits using quantum LDPC codes — a 200× improvement over surface-code estimates.

Quantum chemistry and simulation (Units 3, 7, 8)

- Peruzzo et al. (2014). A variational eigenvalue solver on a photonic quantum processor. Nature Communications 5:4213. The first experimental demonstration of VQE — the algorithm that makes quantum chemistry possible on near-term hardware.
- McArdle, Endo, Aspuru-Guzik, Benjamin, Yuan (2020). Quantum computational chemistry. Reviews of Modern Physics 92:015003. The most comprehensive review of quantum algorithms for chemistry. Start here for the full landscape.
- Low and Chuang (2019). Hamiltonian Simulation by Qubitization. Quantum 3:163. Introduced qubitization — a fundamentally different approach to Hamiltonian simulation that achieves optimal query complexity. The theoretical foundation for next-generation quantum simulation algorithms.
- Reiher, Wiebe, Svore, Wecker, Troyer (2017). Elucidating Reaction Mechanisms on Quantum Computers. PNAS 114:7555–7560. The first concrete resource estimate for quantum chemistry on an industrially relevant molecule (FeMoCo). Established the engineering targets for fault-tolerant quantum chemistry.
- Babbush, Wiebe, McClean, et al. (2018). Low-Depth Quantum Simulation of Materials. Physical Review X 8:011044. Resource estimates for QPE on the 2D Hubbard model — the numbers in Unit 7's Reality Check.

Quantum machine learning (Unit 4)

- Havlíček, Córcoles, Temme, et al. (2019). Supervised learning with quantum-enhanced feature spaces. Nature 567:209–212. The experimental demonstration of quantum kernel methods — the algorithm from Unit 4 running on real hardware.

- Tang (2019). A quantum-inspired classical algorithm for recommendation systems. STOC 2019. The dequantisation result that narrowed the territory for quantum ML advantage. Essential reading for anyone claiming quantum speedups in ML.

Quantum finance (Unit 5)

- Montanaro (2015). Quantum speedup of Monte Carlo methods. arXiv:1504.06987. The theoretical foundation for quantum amplitude estimation applied to Monte Carlo — the quadratic speedup that Unit 5 is built on.
- Grover (1996). A fast quantum mechanical algorithm for database search. arXiv:quant-ph/9605043. Grover's search algorithm, which underlies amplitude estimation. The original quadratic speedup.

Textbooks

- Nielsen and Chuang (2000). Quantum Computation and Quantum Information. Cambridge University Press. The standard reference for the field. Covers algorithms, error correction, and the mathematical foundations. Dense but comprehensive.
- Szabo and Ostlund (1996). Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory. Dover. The classical reference for computational chemistry. Essential background for Units 3, 7, and 8.